系统镜像存储、docker镜像存储技术方案

1. 需求分析

云平台镜像存储是云计算基础设施的核心组件之一，直接关系到虚拟机（VM）和容器的启动速度、分发效率以及平台整体的可靠性和可用性。

1.1 需求概述

1.1.1 核心目标

a. 文件格式支持：支持二进制文件格式和docker镜像、ISO。

b. 全生命周期管理：支持镜像上传与拉取、版本控制、镜像清理删除等。

c. 多租户与权限：支持租户隔离和权限控制。

d. 多后端存储支持：支持本地文件存储、S3兼容对象存储、NAS存储等。

1.1.2 详细需求描述

1.1.2.1 功能需求

a. 文件格式支持

- 二进制文件格式：支持 二进制文件格式存储，如ISO、快照等二进制文件格式。
- 容器镜像格式：支持 OCI（Open Container Initiative）标准，支持 Docker 镜像的分层存储机制。

b. 全生命周期管理

- 上传与拉取：支持分片上传、断点续传、多线程并发拉取。
- 版本控制：支持镜像多版本共存，支持标签和基于时间戳的版本回滚。
- 删除与清理：支持硬删除与软删除，具备“孤儿层”自动回收机制。

c. 多租户与权限

- 租户隔离：镜像不可跨租户默认可见。
- 支持对接企业 LDAP/AD、OIDC（如 Keycloak）进行统一身份认证。
- 权限：支持公开项目和私有项目，私有项目需授权访问。

d. 多后端存储支持

- 本地文件系统：支持本地文件系统，可以配置挂载磁盘或卷。
- 兼容S3对象存储：支持对接S3兼容对象存储。
- NAS：支持对接第三方NAS存储，并支持常见NAS协议。

1.1.2.2 非功能需求

a. 性能需求

- 拉取性能：单镜像拉取速度在千兆网络环境下不低于 100MB/s。
- 推送性能：推送 1GB 镜像（含 10 层）到仓库，总耗时不超过 2分钟。
- 并发能力：单仓库实例支持至少 500 并发推送/拉取请求。

b. 可靠性需求

- 高可用：仓库服务无单点故障，支持多副本或EC模式部署。
- 数据一致性：保证推送、删除、复制操作的最终一致性，避免出现层丢失或引用错误。
- 故障恢复：支持元数据（数据库）的自动备份与恢复。

c. 安全性需求

- 访问控制：支持基于 RBAC 的认证授权，细粒度权限策略。
- 镜像漏洞扫描：支持集成第三方镜像漏洞扫描。
- 传输加密：支持服务端启用 TLS加密。
- 存储加密：支持后端存储的静态加密，敏感镜像数据落盘加密。

d. 可扩展性需求

- 存储容量可水平扩展。
- 兼容其他文件格式，如QCOW2、VMDK等。

2. 架构设计

2.1 技术调研

针对同时管理容器镜像、系统镜像ISO、系统软件包（APT/YUM）、各类库文件（Maven/npm/PyPI等） 的复杂场景，结合上述需求，将可用的免费开源软件做了如下技术选型对比（JFrog Artifactory OSS 开源版阉割到没法用不做对比，FoLib开源版功能阉割也较多，且稳定性和社区支持较差也不进行对比）

对比维度	Sonatype Nexus	Harbor
制品格式覆盖	30+种，原生支持 APT/YUM、Maven、npm、PyPI、Docker、NuGet、Go 等	核心为 OCI 制品（容器镜像 Chart、CNAB、SBOM、M 通过 OCI 规范理论上可扩展 文件，但不支持原生 APT/YU
容器镜像能力	✅ 支持 Docker Registry V2 API，但功能深度不如 Harbor	✅ 原生设计，功能最强：镜 名、漏洞扫描（Trivy）、不 回收策略等
ISO/APT/YUM 支持	✅ 原生支持	❌ 不支持，无 APT/YUM 仓
Maven/npm/PyPI 支持	✅ 原生支持	❌ 不支持
安全能力	✅ 开源版不自带漏洞扫描，但可以与第三方集成	✅ 容器原生安全最强：Tri 描、镜像签名（Cosign/No RBAC、审计日志
租户支持	⚠️支持LDAP，但开源版不支持SAML和用户Token认证	✅支持
信创适配	❌ 非原生强项	⚠️ 社区版无原生信创适配 多架构支持工作组，但非产 配
社区/生态	✅ 社区成熟，文档丰富	✅ CNCF 毕业项目，容器生

neuxs社区版功能

https://help.sonatype.com/en/nexus-repository-feature-matrix.html

Formats	Community Edition (CE)	Professional (PRO)
APT	✓	✓
CocoaPods	✓	✓
Conan	✓	✓
Conda	✓	✓
Docker	✓	✓
Git Large File Storage (LFS)	✓	✓
Go Modules	✓	✓
Gradle	✓	✓
Helm	✓	✓
Maven	✓	✓
npm	✓	✓
NuGet	✓	✓
p2	✓	✓
Pub	✓	✓
PyPI	✓	✓
R	✓	✓
Rust Cargo	✓	✓
RubyGems	✓	✓
Swift	✓	✓
Terraform	✓	✓
Yum	✓	✓

Community Features	Community Edition (CE)	Professional (PRO)
AWS S3 Blob Stores	✓	✓
Component Search	✓	✓
Custom Access Controls	✓	✓
External PostgreSQL Support	✓	✓
Improved Backup & Restore	✓	✓
LDAP	✓	✓
Comprehensive API coverage	✓	✓
Remote Auth Token (RUT)	✓	✓
Repository Health Check	✓	✓
Repo Targets / Content Selectors	✓	✓
Routing Rules	✓	✓
REST	✓	✓
Unlimited Nexus Repository Deployments	✓	✓
Upload Third-Party Artifacts via UI	✓	✓

Enterprise Features	Community Edition (CE)	Professional (PRO)
Azure Blob Store	✗	✓
Change Repository Blob Store	✗	✓
Component Tagging	✗	✓
Content Replication	✗	✓
Crowd	✗	✓
Deployment to Group Repositories	✗	✓
Docker Subdomain Connector	✗	✓
Google Blob Store	✗	✓
Export Assets	✗	✓
High Availability Deployment Options	✗	✓
Import External Files	✗	✓
Repository Health Check Detailed Report	✗	✓
Resilient Deployment Options	✗	✓
SAML	✗	✓
Staging & Build Promotion	✗	✓
User Token Support	✗	✓

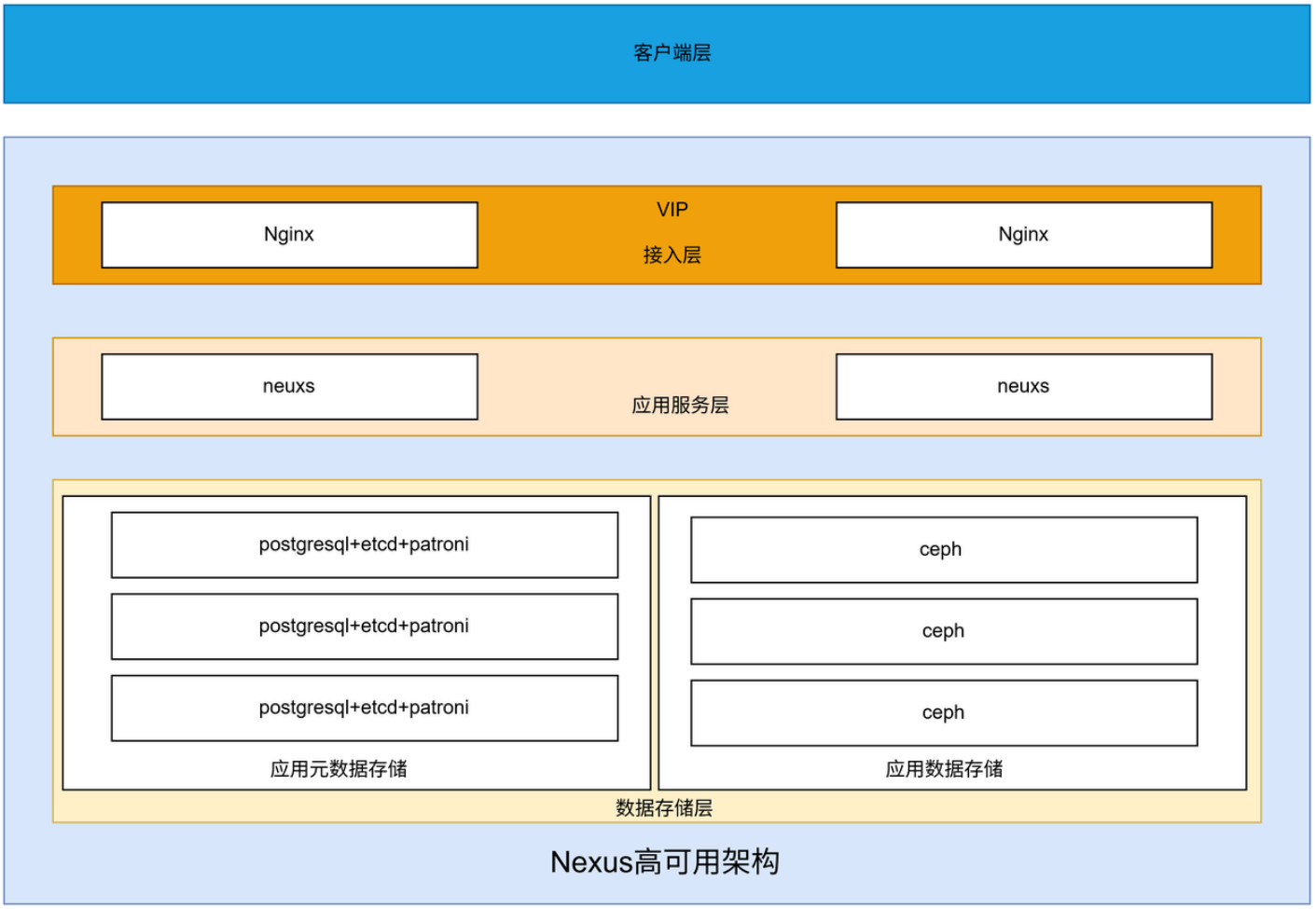
logo和样式可调整能力

定制项	Nexus OSS	Nexus Pro
替换 Logo	✅ 手动放文件	✅ 界面配置
自定义 CSS	✅ 注入文件	✅ 界面配置
修改文案 / 版权	✅ 配置文件	✅ 界面配置
调整菜单顺序	❌ 需改源码	✅ 界面配置
增删页面模块	❌ 需改源码	✅ 插件扩展
拖拽式布局	❌	✅

根据需求和上述对比，选择开源版本基础功能支持全面nexus来作为镜像管理工具。

2.2 概要设计

基于Nexus实现高可用制品仓库，避免制品服务宕机导致客户业务服务受影响、内部团队开发中断、CI/CD 流水线阻塞。通过多节点负载均衡提升并发下载性能，加速构建；同时实现数据冗余保护，防止因磁盘故障导致数据丢失。高可用架构如下图所示



接入层：Keepalived+Nginx 主备，VIP 自动漂移，单节点宕机不影响入口访问。

应用服务层：Nexus 主备部署，Nginx 将流量转发到本机后端的neuxs服务，当Nginx 或neuxs被检测到有一个异常时则进行主备切换，保证流量正常转发。

数据存储层：包括应用元数据存储和应用数据存储。应用元数据存储由Patroni + etcd + PostgreSQL完成，PostgreSQL负责实际的元数据存储、查询、事务处理，它自己不知道谁是主谁是从，需要外部来告诉它；etcd 分布式配置与状态存储，记录当前集群的拓扑状态（谁是主、谁是从、各自的连接信息、切换历史等）；Patroni 高可用控制器（核心大脑），主库宕机时，协调各节点通过 etcd 完成新主选举，告诉本地 PG “你是主"或"你是从”，从库挂了自动拉起，主库挂了自动切换。应用数据存储ceph集群完成，多副本（或纠删码）存储制品文件，单节点故障不影响数据读写。

2.3 详细设计

2.3.1 角色分工

组件	节点（host1）	节点（host2）	节点（host3）	主要职责
Nexus	✓（实例1）	✓（实例2）	-	提供制品仓库服务
Nginx	✓（主）	✓（备）	-	负载均衡，通过 VIP 分
Keepalived	✓（主）	✓（备）	-	管理虚拟IP（VIP），实
PostgreSQL	✓（主）	✓（从）	✓（从）	存储 Nexus 的配置、数
Patroni	✓	✓	✓	自动管理 PostgreSQL 集
etcd	✓	✓	✓	为 Patroni 提供分布式
Ceph	✓	✓	✓	提供共享存储，存储实

2.3.2 部署前准备

1. 主机与网络配置
   - 修改主机名：在每台服务器执行 `sudo hostnamectl set-hostname host{X}`，如 `host1`。
   - 配置域名解析：在三台主机的 `/etc/hosts` 中添加对方的主机名和IP。
   - 配置免密登录：配置从 `host1` 到其他所有主机的 SSH 免密登录，便于后期管理。
2. 系统初始化

代码块

```
# 更新系统并安装基础工具
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget gnupg2 software-properties-common python3-pip
python3-venv openssl ntp

# 启用并同步时间
sudo systemctl enable --now ntp
```

2.3.3 部署高可用 Ceph 存储集群

1. 安装 `cephadm`：在所有三台服务器上，安装 Ceph 集群管理工具。

代码块

```
sudo apt install -y cephadm podman lvm2
```

2. 引导初始 Monitor 节点：在 `host1` 上，引导并创建 Ceph 集群的首个 Monitor 节点。`{host1_ip}` 需替换为 `host1` 的实际 IP。

代码块

```
sudo cephadm bootstrap --mon-ip {host1_ip} --allow-fqdn-hostname
```

3. 添加其余主机：在 `host1` 上，将 `host2` 和 `host3` 添加到集群中。

代码块

```
# 添加 SSH 公钥
sudo cephadm shell -- ceph cephadm get-ssh-config > ~/ceph_ssh_config
ssh-copy-id -f -i /etc/ceph/ceph.pub root@host2
ssh-copy-id -f -i /etc/ceph/ceph.pub root@host3

# 在集群中添加主机
sudo cephadm shell -- ceph orch host add host2
sudo cephadm shell -- ceph orch host add host3
```

🍰 *提示：执行命令后，可使用 `sudo cephadm shell -- ceph orch host ls` 验证主机是否成功加入。*

4. 部署 OSD（对象存储设备）：在 `host1` 上，为每台已添加未格式化磁盘（如 `/dev/sdb`）的服务器部署 OSD。

代码块

```
# 为 host1 添加 OSD（替换 /dev/sdb 为实际磁盘）
sudo cephadm shell -- ceph orch daemon add osd host1:/dev/sdb
# 为 host2 和 host3 重复此操作
```

5. 部署 RADOS Gateway（RGW）：在 `host1` 上，为每个节点部署 RGW 服务实例，以对外提供 S3 接口。

代码块

```
```

```
sudo cephadm shell -- ceph orch apply rgw nexus-s3 --placement="3 host1 host2 host3"
```

6. 配置 S3 访问
   - 创建用户与凭证：在 `host1` 上创建 S3 用户并获取访问密钥。

```
# 创建用户
sudo radosgw-admin user create --uid=nexus-user --display-name="Nexus S3 User"
# 记录输出中的 "access_key" 和 "secret_key"
```

   - 创建存储桶：安装 `s3cmd` 工具并通过它创建用于 Nexus 的存储桶。

```
sudo apt install -y s3cmd
# 配置 s3cmd（需提供上一步的 access_key, secret_key 以及 RGW 地址）
s3cmd --configure
# 创建存储桶
s3cmd mb s3://nexus-bucket
```

2.3.4 部署高可用 PostgreSQL 数据库集群

2.3.4.1 二进制常规安装

1. 安装 PostgreSQL：在所有三台服务器上，安装数据库，并暂停默认服务。

```
# 添加官方仓库并安装
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /usr/share/keyrings/postgresql.gpg
echo "deb [signed-by=/usr/share/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt/ $(lsb_release -cs)-pgdg main" | sudo tee /etc/apt/sources.list.d/postgresql.list
sudo apt update
sudo apt install -y postgresql-16 postgresql-16-dev postgresql-client-16
sudo systemctl stop postgresql && sudo systemctl disable postgresql
```

2. 安装并配置 etcd 集群

- 安装 etcd：在所有节点下载并安装。

代码块

```
# 下载并安装二进制文件
ETCD_VERSION="3.5.10"
wget https://github.com/etcd-io/etcd/releases/download/v${ETCD_VERSION}/etcd-v${ETCD_VERSION}-linux-amd64.tar.gz
tar -xzf etcd-v${ETCD_VERSION}-linux-amd64.tar.gz
sudo mv etcd-v${ETCD_VERSION}-linux-amd64/etcd* /usr/local/bin/
```

- 配置 etcd 集群：在各节点创建配置文件 `/etc/etcd/etcd.conf`，内容大体一致，注意替换 `{name}`、`{local_ip}` 等占位符：

代码块

```
name: '{host1}' # 根据主机名修改
data-dir: /var/lib/etcd
initial-cluster-token: 'nexus-ha-etcd-cluster'
initial-cluster: 'host1=http://{host1_ip}:2380,host2=http://{host2_ip}:2380,host3=http://{host3_ip}:2380'
initial-cluster-state: 'new'
listen-peer-urls: 'http://{local_ip}:2380'
listen-client-urls: 'http://{local_ip}:2379,http://127.0.0.1:2379'
advertise-client-urls: 'http://{local_ip}:2379'
initial-advertise-peer-urls: 'http://{local_ip}:2380'
```

- 创建 systemd 服务并启动：在每台服务器上创建 `/etc/systemd/system/etcd.service` 服务文件，然后启动 etcd。

代码块

```
sudo systemctl daemon-reload
sudo systemctl enable --now etcd
```

验证集群：`etcdctl --endpoints=http://127.0.0.1:2379 member list` 应在任一节点显示三个成员。

3. 安装并配置 Patroni：在所有节点上，安装 Patroni 并创建配置文件。

代码块

```
# 安装
```

```
sudo pip3 install patroni[etcd] psycopg2-binary
# 创建配置文件 /etc/patroni.yml（需根据节点信息修改 name, host, etcd hosts 等）
```

验证集群：创建 Patroni service 文件并启动后，执行 `patronictl -c /etc/patroni.yml list` 应显示集群状态。

4. 安装 HAProxy（可选，作为数据库访问入口）：在所有节点上，安装并配置 HAProxy，为数据库访问提供一个稳定的入口。配置文件 `/etc/haproxy/haproxy.cfg` 示例：屏蔽 HAProxy 的管理接口。

代码块

```
# 安装
sudo apt install -y haproxy
```

验证代理：配置完成后，可使用 `psql -h 127.0.0.1 -p 5000 -U postgres` 测试连接。

2.3.4.2 k8s安装（未测试）

PostgreSQL跑在容器或K8s里，稳定性和生产级可用性已经被充分验证，其性能的关键则不在于容器本身，而更多取决于存储和网络方案的选型。k8s容器部署可以按现有组件方案部署，但更好的最佳实践方案是用PostgreSQL Operator，它封装了 Patroni 的逻辑，并提供了更高级的自动化功能。

方案1：Bitnami PostgreSQL Helm Chart（其内部使用 Patroni + etcd）

1. 添加 Bitnami 仓库

helm repo add bitnami https://charts.bitnami.com/bitnami

helm repo update

2. 部署一个 3 节点的 HA PostgreSQL 集群

helm install my-postgres bitnami/postgresql-ha

方案2：PostgreSQL Operator

CloudNativePG (推荐)，这是目前 CNCF 社区最火、最符合 K8s 原生哲学的 Operator。

优点:

无外部依赖: 不需要 Etcd，直接利用 K8s 原生的 API 进行选主。

全生命周期管理: 自动处理安装、备份、在线扩容、版本升级。

自愈能力: 故障切换极快（通常在 10s 内）。

声明式 API: 只需提交一个 `Kind: Cluster` 的 YAML 即可。

部署参考

代码块

```
cluster.yaml
apiVersion: postgresql.cnpg.io/v1
kind: Clustermetadata:
    name: pg-ha-cluster
spec:
    instances: 3                    # 包含 3 个实例的高可用集群
    storage:    size: 128Gi
```

2.3.5 配置高可用 Nexus 应用集群

1. 准备共享存储 (S3 Blob Store)：已通过 Ceph 的 S3 接口实现，后续在 Nexus 中配置。
2. 部署 Nexus Docker 实例：
   - 在每台服务器上，使用 Docker 启动 Nexus 实例，注意：每个实例需使用相同的配置。

   代码块

   ```
   sudo mkdir -p /opt/nexus/data
   sudo chown -R 200:200 /opt/nexus/data
   docker run -d -p 8081:8081 --name nexus \
     -v /opt/nexus/data:/nexus-data \
     -e INSTALL4J_ADD_VM_PARAMS="-Xms4g -Xmx4g -XX:MaxDirectMemorySize=2g" \
     sonatype/nexus3
   ```

   - -p 8081:8081: 将容器的8081端口映射到主机的8081端口。
   - -v /opt/nexus/data:/nexus-data: 关键步骤，将主机上的数据目录挂载到容器内部，用于测试环境下模拟持久化存储。
   - -e INSTALL4J_ADD_VM_PARAMS：设置 Nexus 的 JVM 参数，用于调整内存大小和控制性能。建议根据服务器实际可用内存调整 -Xms 和 -Xmx 的值。
3. 配置 Nexus 实例：
   - 首次登录：访问任一实例的 `http://{host_ip}:8081`，获取初始密码。

```
docker exec -it nexus cat /nexus-data/admin.password
```

- 配置 S3 Blob Store：导航至 “设置” -> “Blob Stores”，创建类型为 `S3` 的存储，填入 Ceph 的 S3 凭证信息。
- 配置数据库连接：导航至 “设置” -> “System” -> “Database”，选择 `External PostgreSQL`，填入数据库集群信息（如 `{db_host}:5432`）、数据库名 `nexus` 及用户名、密码。
- 重复配置：在任何一台 Nexus 实例上完成以上配置后，其他实例只需启动 Docker 容器并挂载相同的数据目录即可。

2.3.6 部署高可用 Nginx 负载均衡器

1. 安装软件：在 `host1` 和 `host2` 服务器上，安装 Nginx 和 Keepalived。

代码块

```
sudo apt install -y nginx keepalived
```

2. 配置 Nginx：在 `host1` 和 `host2` 上，创建统一的配置文件 `/etc/nginx/sites-available/nexus`，内容示例如下：

代码块

```
upstream nexus_backend {
    # 后端 Nexus 实例列表
    server {host1_ip}:8081 max_fails=3 fail_timeout=30s;
    server {host2_ip}:8081 max_fails=3 fail_timeout=30s;
    server {host3_ip}:8081 max_fails=3 fail_timeout=30s; # 如果有三个实例
}
server {
    listen 80;
    server_name nexus.yourdomain.com;
    location / {
        proxy_pass http://nexus_backend;
        # ... 其他标准的 proxy 设置 ...
    }
}
```

配置后通过软链接启用 Nginx 站点：

```
sudo ln -s /etc/nginx/sites-available/nexus /etc/nginx/sites-enabled/nexus
sudo nginx -t && sudo systemctl restart nginx
```

3. 配置 Keepalived：
   - 在 `host1` (主) 编辑 `/etc/keepalived/keepalived.conf`：

代码块

```
vrrp_instance VI_1 {
    state MASTER
    interface enp0s8 # 替换为实际的网卡名
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1234
    }
    virtual_ipaddress {
        192.168.1.100/24 # VIP地址
    }
    track_script {
        chk_nginx
    }
}
```

   - 在 `host2` (备) 上修改配置：`state BACKUP` 和 `priority 90`。
   - 创建健康检查脚本：

代码块

```
# 创建脚本 /usr/local/bin/nginx-health-check.sh
#! /bin/bash
counter=$(ps -C nginx --no-heading|wc -l)
if [ "${counter}" = "0" ]; then
    exit 1
fi
exit 0
# 确保脚本有执行权限
chmod +x /usr/local/bin/nginx-health-check.sh
```

2.3.7 验证高可用方案

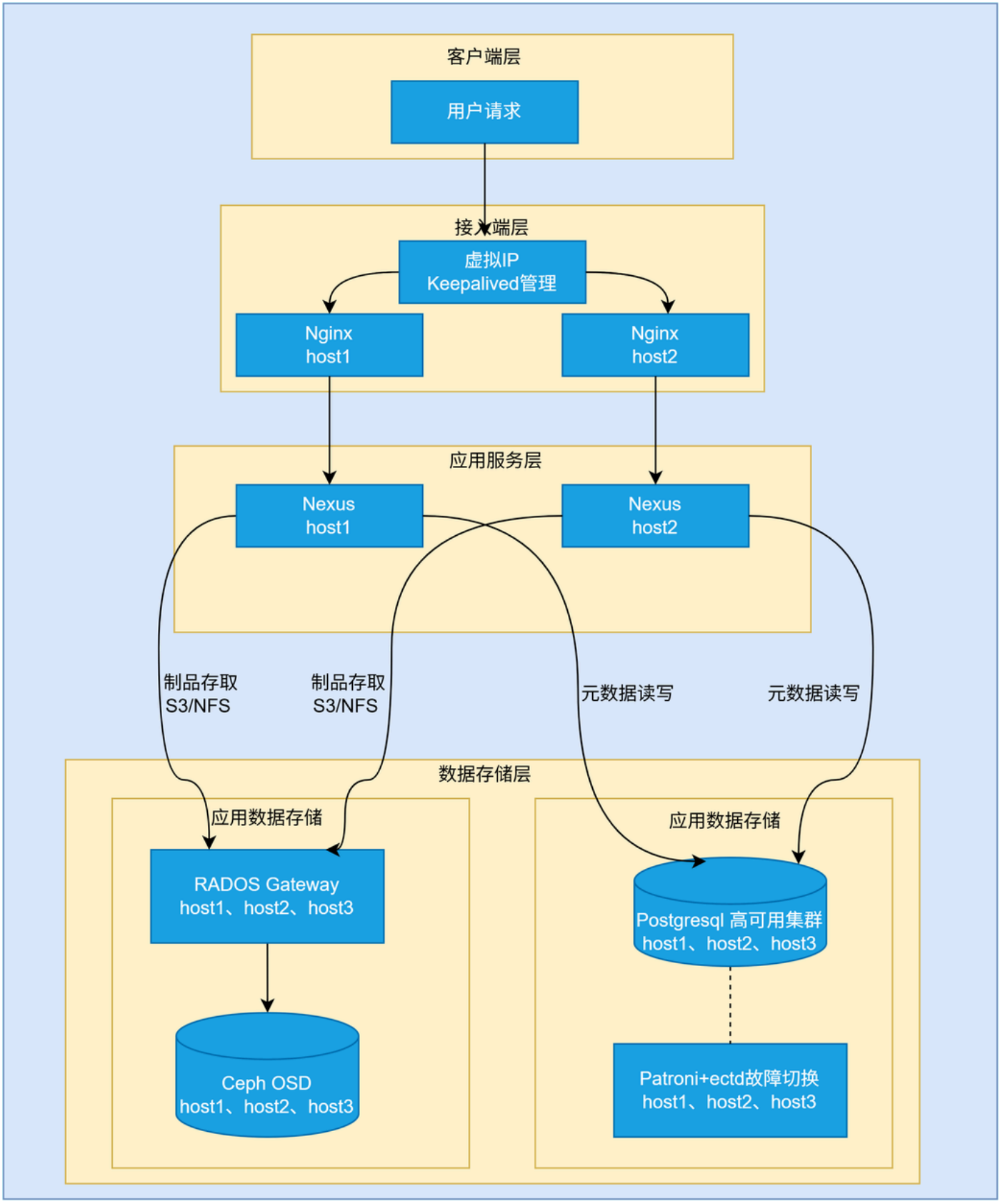
完成部署后，进行以下关键功能验证：

- 访问验证：通过虚拟IP（VIP）或解析到该IP的域名访问 Nexus。
- 单点故障模拟：
  - 模拟 Nexus 实例故障：`docker stop nexus` 其一实例，服务应正常。
  - 模拟 Nginx 节点故障：在 `host1` 执行 `systemctl stop keepalived`，VIP 应立即漂移到 `host2`。
  - 模拟 数据库主节点故障：使用 `patronictl -c /etc/patroni.yml switchover` 观察切换。
  - 模拟 Ceph 存储节点故障：观察文件上传下载是否正常。
- 性能与功能测试：上传下载大文件，观察是否出现错误。

3. 业务实现

3.1 模块间数据流交互

3.1.1 整体数据流图

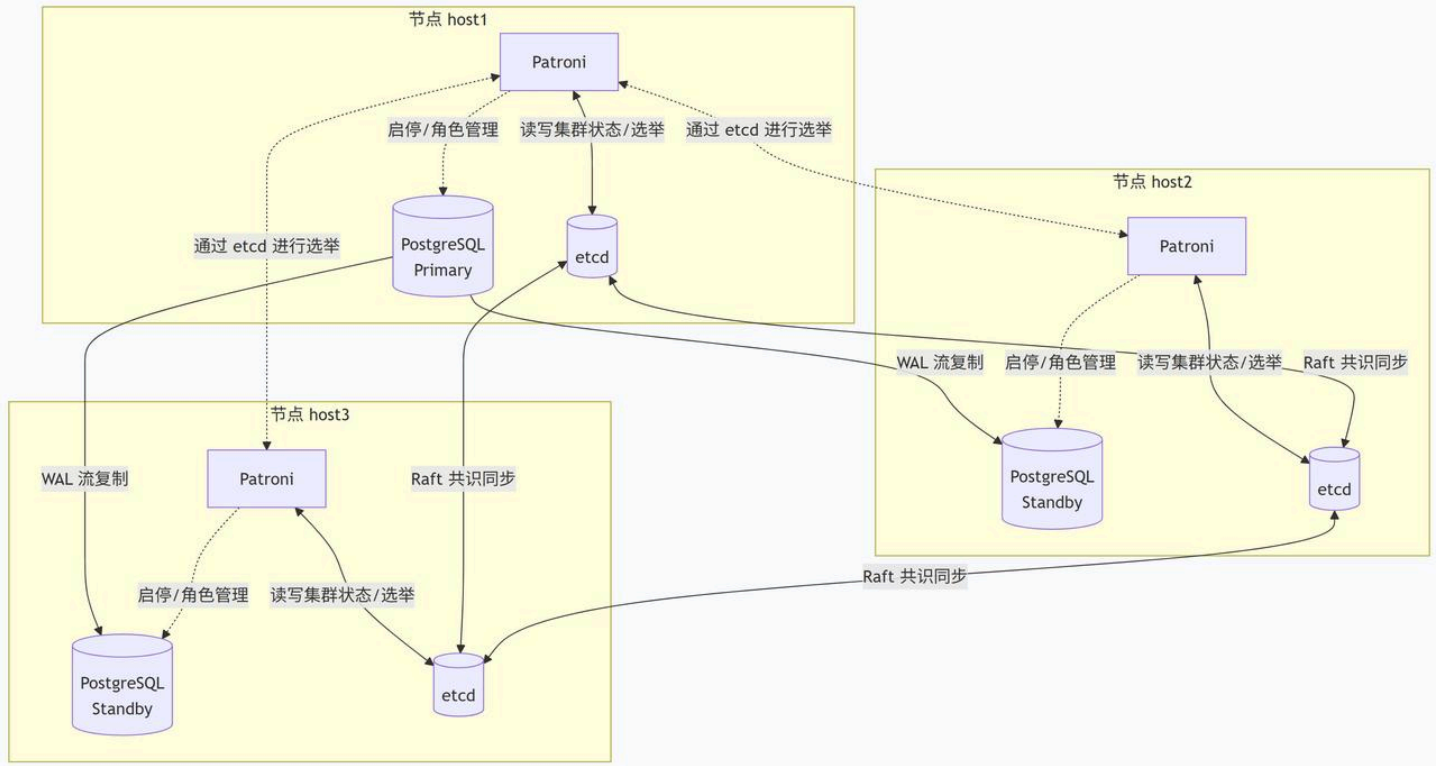


分层说明：

- 客户端层：外部用户请求。
- 接入层：由 Keepalived 管理虚拟 IP，高可用 Nginx 实现流量接入与分发。
- 应用服务层：Nexus 实例处理业务逻辑。

- 数据存储层：分为“元数据存储”（PostgreSQL + Patroni + etcd）与“应用数据存储”（Ceph 对象存储集群），分布在三台主机上，实现数据共享和一致。

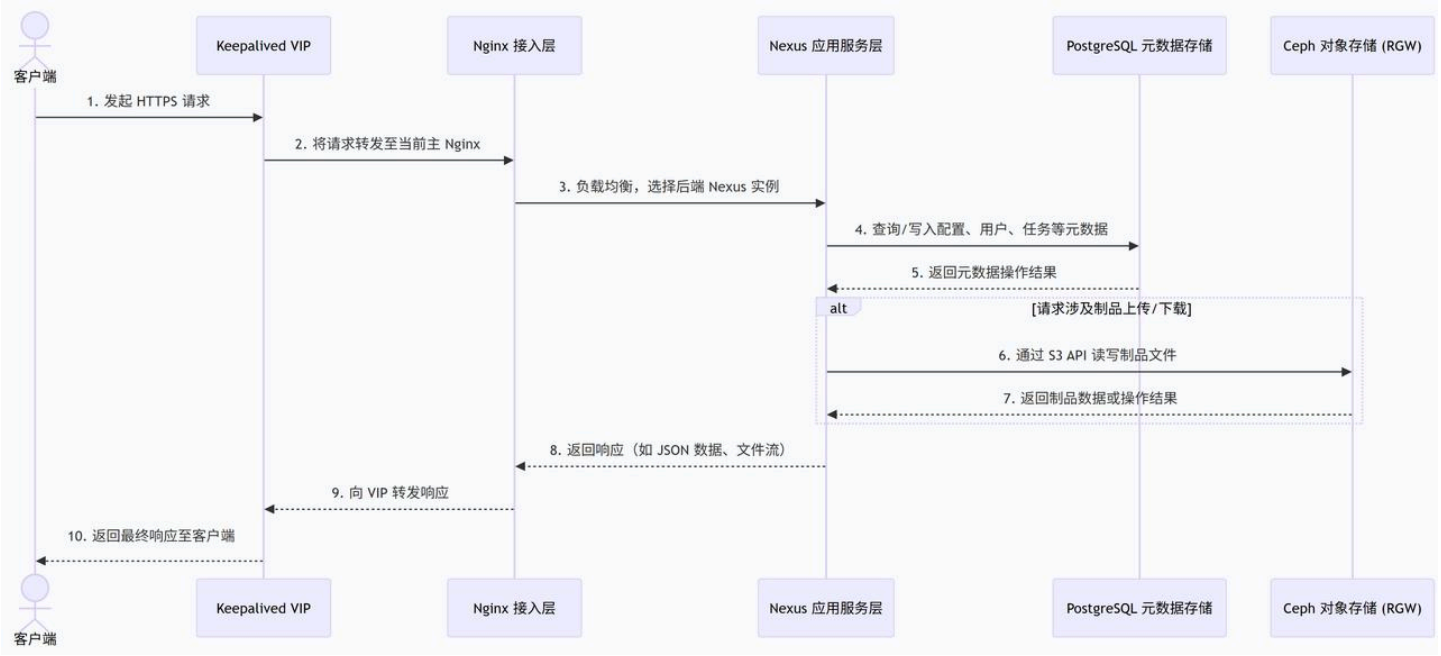
3.1.2 PostgreSQL + Patroni + etcd 三个核心组件之间的内部数据流图



数据流说明：

1. WAL 流复制（橙色实线）
   Primary 节点将预写日志实时传输到所有 Standby 节点，实现数据同步。
2. Patroni ↔ etcd（紫色双向箭头）
   每个节点上的 Patroni 直接读写本地的 etcd 实例，以获取或更新集群状态、领导者信息及配置。
3. etcd 集群间 Raft 同步（紫色双箭头）
   etcd 节点通过 Raft 协议保持数据强一致，确保集群状态信息在任意时刻全局统一。
4. Patroni → PostgreSQL 管理（灰色虚线）
   Patroni 根据 etcd 中的状态，对本地 PostgreSQL 进行启停、角色提升或降级等生命周期控制。
5. Patroni 间间接通信（灰色虚线箭头）
   Patroni 实例之间不直接通信，而是通过 etcd 共享的状态进行领导者选举与故障检测。

3.2 请求处理时序图



1. 接入高可用：用户始终访问由 Keepalived 管理的 VIP，即使某一 Nginx 宕机，VIP 也会自动漂移，确保接入层高可用。
2. 代理转发：主Nginx 将请求分发至健康的 Nexus 后端实例。
3. 元数据分离：Nexus 将用户、权限、任务、仓库配置等全部结构化数据存放于高可用的 PostgreSQL 集群，保证一致性。
4. 制品存储分离：所有二进制制品文件直接通过 S3 兼容 API 存入 Ceph 对象存储，利用其高可用和冗余能力。
5. 故障无感：若任一 Nexus 或nginx实例故障， Nexus 和Nginx 自动切换至另一个实例，且因为数据库和存储均为共享，服务可无缝恢复。

3.3 基本功能

3.3.1 登录与初始化

1. 浏览器打开：`http://<服务器IP>:8081`（默认端口 8081）

http://192.168.2.100:8081/

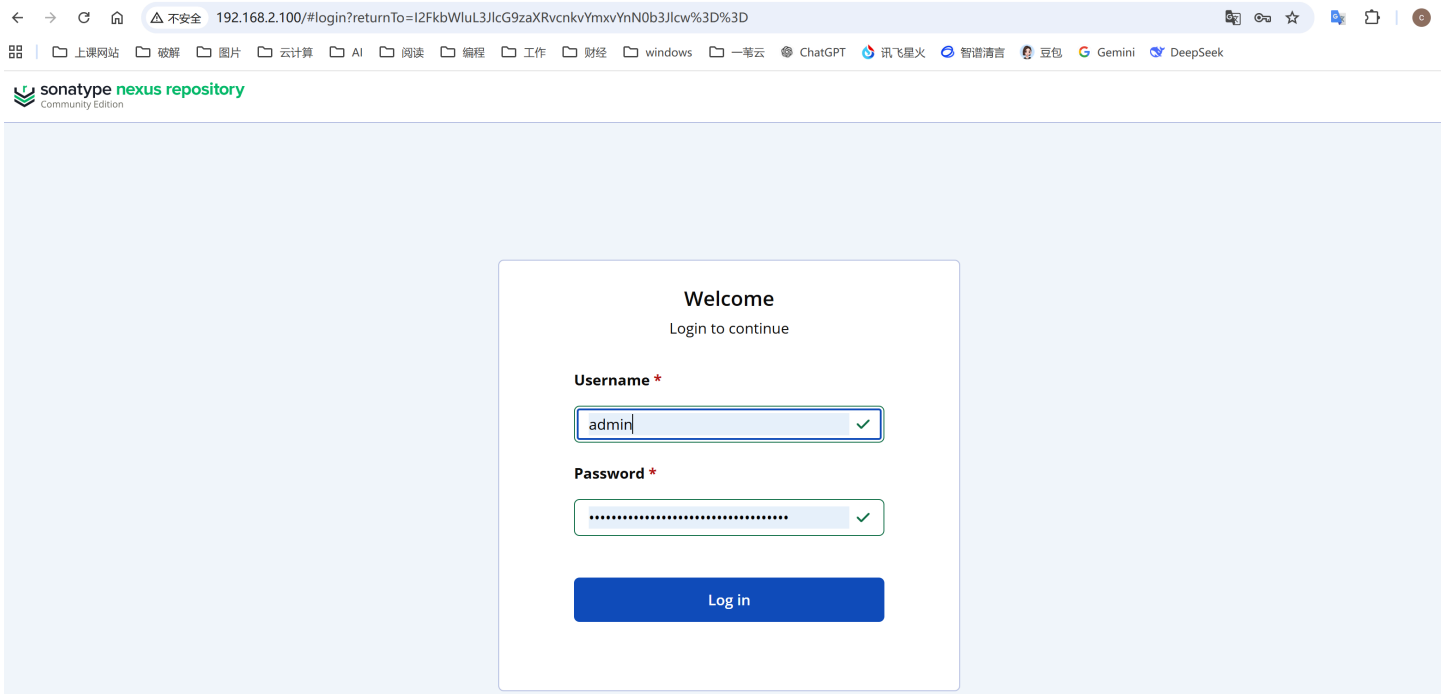
2. 初始密码获取（Nexus 3.x）
   - Docker 部署：`docker exec -it nexus cat /nexus-data/admin.password`。
   - 物理机部署：`sonatype-work/nexus3/admin.password`。

代码块

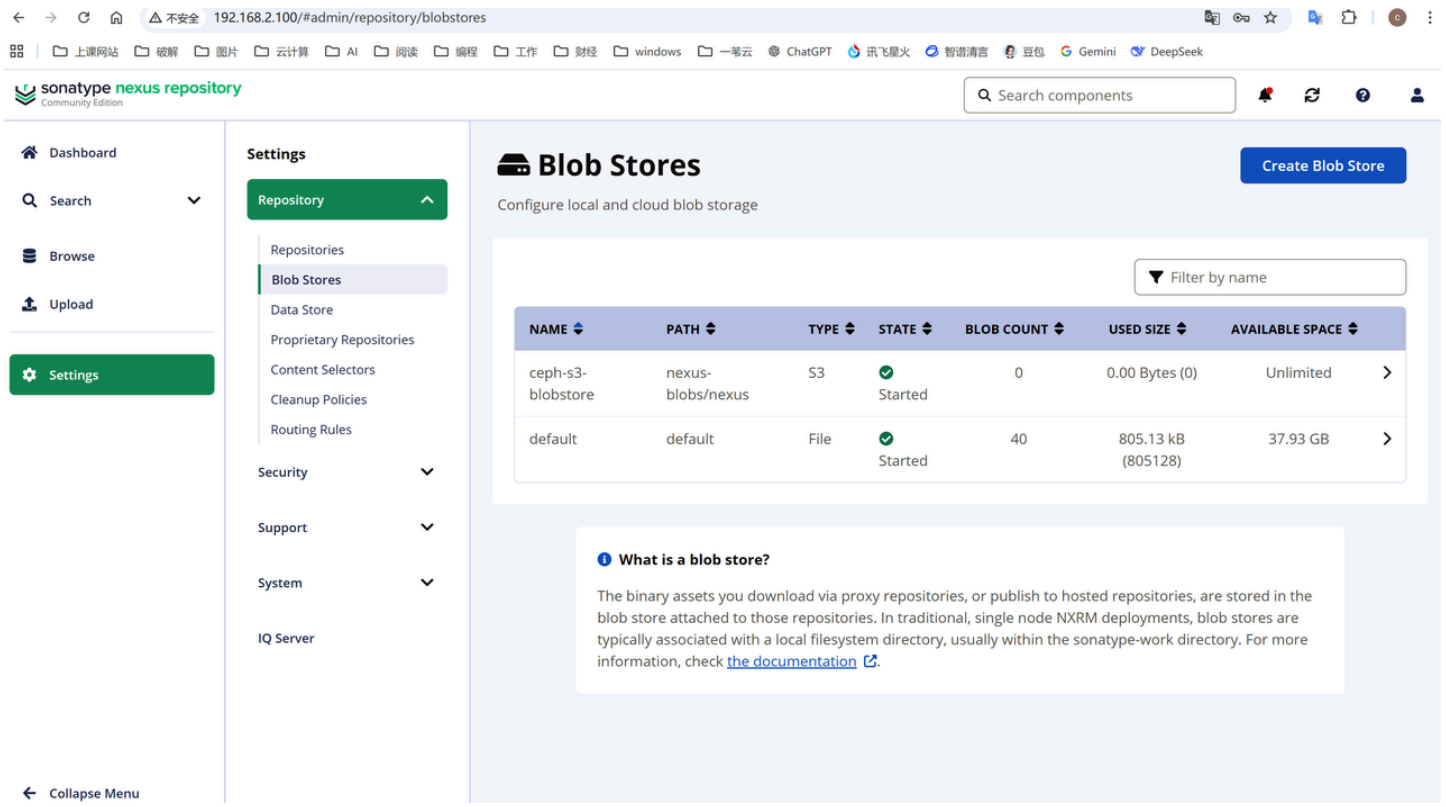
```
cat /opt/sonatype-work/nexus3/admin.password
```

```
2    304f3b76-9301-4705-bfa8-14c0593b80fc
```

在登录界面输入用户名密码，登录



登录



3.3.2 用户接入和认证 （未测试）

Nexus OSS社区版所支持的核心认证方式，并补充了其与Nexus Pro专业版或云端版的区别：

认证方式	简介	适用场景
本地认证	Nexus内置的用户名/密码认证，用户信息存储在Nexus内部数据库中。	小规模团队、测 回退方案。
LDAP集成	对接企业现有的LDAP目录服务（如OpenLDAP、Microsoft Active Directory）进行集中认证，并可同步目录中的用户组用于权限控制。	需要统一身份认 用的集中认证方
远程用户令牌 (RUT)	前置一个反向代理（如Nginx、Apache）处理用户认证，并通过HTTP头（Header）将已验证的用户信息传递给Nexus。	与现有基于PKI、
匿名访问	允许用户在不登录的情况下访问资源，通常为只读权限。	公开的软件仓库 景。
其他外部认证	Nexus还支持与Atlassian Crowd、SAML SSO、OIDC对接。	需要使用特定SS
用户令牌 (User Token)	允许用户生成一对令牌替代用户名/密码，用于Maven等客户端工具。	提升客户端认证 佳实践。

远程用户令牌 (RUT) 的完整工作流程与集成步骤：

核心原理：反向代理 + HTTP头

RUT认证的核心流程如下：

1. 用户请求所有流量 → Nginx、Apache等反向代理
2. 反向代理（对接LDAP、Kerberos、PKI等外部身份系统）负责完成用户的认证
3. 反向代理在每个请求到达Nexus前，动态增加一个自定义的HTTP头（如 `X-SSO-USER` ），其值为认证后的用户名
4. Nexus不再有登录界面，直接信任反向代理传递过来的HTTP头，将该值作为已认证的用户，并继续执行后续的授权步骤

🔄 核心关键环节

第一步：在 Nexus 中激活 RUT 认证

这是让 Nexus “认识”RUT并知道该信任哪个HTTP头：

- 在 管理界面 (UI) -> 设置 (System) -> 能力 (Capabilities) -> 点击创建能力 (Create Capability) -> 选择 `Rut Auth` 。
- 在 "HTTP Header Name" 字段，填入你在反向代理中设置的那个HTTP头名称。两者必须保持完全一致。

- 在 设置 (Settings) -> 安全 (Security) -> 域 (Realms) -> 将 `Rut Auth Realm` 移到激活列表并置顶（或确保它在 `Local Authenticating Realm` 之上）。

第二步：补充 Nexus 中的用户授权

Nexus完成了认证，但还需要知道用户能做什么。有三种推荐方式：

- 使用LDAP进行外部角色映射（最优选）：用户认证由代理完成，但授权可依赖Nexus已集成的LDAP。Nexus将外部LDAP组映射到其内部角色，RUT认证的用户名和组信息匹配后自动获得相应权限。
- 配合API自动创建用户（备选方案）：若无法使用LDAP，可采用此方案。编写脚本在用户第一次RUT认证时调用Nexus的REST API在本地数据库中动态创建用户。
- 在Nexus中手动创建同名用户（体验差，不推荐）：虽然最简单的办法是在Nexus中创建用户名完全相同的用户，再由管理员手动分配角色权限，但它违背了RUT自动化的初衷，难以用于日常环境。

🏗️ 典型集成场景与配置示例

RUT 的最佳实践是基于不同身份管理系统，在反向代理层面进行认证，例如：

- LDAP / Active Directory：Apache可配置 `mod_authnz_ldap` 模块，对用户进行LDAP基本认证。认证成功后，代理添加 `X-SSO-USER` 头，并完成转发。
- PKI / 客户端证书：Apache可配置 `SSLVerifyClient require`，要求客户端提供证书。认证成功后提取证书中的通用名(CN)，将其值赋给 `X-SSO-USER` 头。
- Kerberos / SPNEGO：在Apache中配置 `mod_auth_kerb` 模块来处理Kerberos认证。认证成功后，代理通过 `mod_headers` 设置 `X-SSO-USER` 头。
- Keycloak / OIDC：使用Nginx部署Keycloak网关服务(如 `keycloak-gatekeeper`)在Nexus之前。网关处理OIDC流程后将登录用户信息设置在HTTP头中（例如 `X-Auth-Email` 或 `preferred_username`），由Nexus的RUT Auth能力消费。

⚠️ 注意事项与最佳实践

在规划RUT方案时，有几点需要特别留意：

- 安全风险：必须严格控制反向代理和Nexus之间的网络访问，确保用户无法绕过代理直接访问Nexus端口（`127.0.0.1`），否则任何人都可以伪造HTTP头冒充他人。
- 版本限制：从 Nexus 3.78.0 开始，RUT Auth Realm 在OSS（社区版）中被标记为“不可用”，使用前请务必检查版本（建议使用3.77.x及以下版本）。
- 数据同步问题：RUT本身不处理用户信息的同步，Nexus不会自动创建用户或同步头像等。用户资料的增删改查需要额外通过API或手动操作来完成。
- **RUT版本支持情况：3.78.0–3.79.0 OSS，**RUT 被官方临时移除 / 隐藏，不可用；**3.79.1 及之后（含 3.91.1）OSS，**官方在 3.79.1 恢复了 RUT Auth Realm，之后的社区版一直可用。

3.3.2.1 支持OIDC和SAML协议开发工作量评估

3.3.2.1.1 现有代码库状态

- 基于代码库分析，以下是详细的技术评估报告。

现有代码库状态（Nexus Repository Manager 3.91.1-04）

已有基础能力

模块	状态	位置
OAuth2 前端配置 UI	✅ 已实现	nexus-coreui-plugin/.../OAuth2Configuration/
SSO 登录按钮	✅ 已实现	nexus-ui-plugin/.../login/SsoLogin.jsx
OAuth2Realm 注册引用	⚠️仅桩代码	RealmToSource.java 枚举引用
SamlRealm 注册引用	⚠️仅桩代码	RealmToSource.java 枚举引用
JWT 基础设施	✅ 已实现	nexus-base/.../jwt/
Apache Shiro 框架	✅ 已集成	全局安全框架
LDAP Realm	✅ 完整实现	可作为参考实现

核心问题： OAuth2Realm 和 SamlRealm 的实际实现在私有闭源模块（/private）中，公开代码库只有接口和枚举引用。

3.3.2.1.2 OIDC协议支持

OIDC 协议支持 — 需要新增模块

模块 1: OIDC Discovery & Provider 配置

功能：解析 /.well-known/openid-configuration，自动填充端点

组件	文件	估算行数
OidcProviderDiscoveryService	Java 服务	~200 行
OidcProviderConfig（数据模型）	Java DTO	~120 行
OidcConfigurationStore	持久化	~150 行
数据库迁移脚本	SQL	~50 行
小计		**~520 行**

模块 2: Authorization Code Flow + PKCE

功能：标准 OIDC 授权码流程，支持 PKCE（S256）

组件	文件	估算行数
OidcAuthorizationFilter	Shiro Filter	~280 行
OidcCallbackServlet	回调处理	~250 行
PkceGenerator / Verifier	安全工具	~100 行
StateNonceManager（CSRF防护）	状态管理	~180 行
OidcTokenExchangeService	Token 交换	~220 行
小计		**~1,030 行**

模块 3: Token 验证与 UserInfo

功能：ID Token 验证 (JWT/JWKS)，UserInfo 端点集成

组件	文件	估算行数
IdTokenValidator	JWT 验证	~300 行
JwksKeyResolver	JWKS 动态密钥	~200 行
OidcUserInfoService	UserInfo 获取	~150 行
ClaimMappingService	声明映射	~250 行
小计		**~900 行**

模块 4: Shiro Realm 实现 (OAuth2Realm)

功能：将 OIDC 集成进 Shiro 安全框架

组件	文件	估算行数
OAuth2Realm (Shiro Realm)	核心 Realm	~400 行
OAuth2UserManager	用户管理	~300 行
OAuth2AuthenticationToken	Token 封装	~100 行
OidcSessionManager	Session 管理	~200 行
小计		**~1,000 行**

模块 5: RP-Initiated Logout & Token Refresh

功能：OIDC 登出，Refresh Token 静默续期

组件	文件	估算行数
OidcLogoutFilter	登出处理	~180 行
TokenRefreshService	续期逻辑	~220 行
小计		**~400 行**

模块 6: REST API 端点（新增/扩展）

功能：OIDC 配置管理 API

组件	文件	估算行数
OidcConfigApiResource	REST 接口	~200 行
OidcStatusApiResource	状态/测试接口	~150 行
小计		**~350 行**

OIDC 后端总计：~4,200 行 Java

OIDC 前端补充（已有 UI 框架，需完善）

组件	估算行数
OIDC Discovery URL 自动填充逻辑	~150 行
PKCE 配置选项	~80 行
Token 状态展示/调试面板	~200 行
配置验证/测试连接	~120 行
前端小计	**~550 行**

3.3.2.1.3 SAML协议支持

SAML 2.0 协议支持 — 需要新增模块

模块 1: SP 元数据管理

功能：生成/导出 SP Metadata XML，上传 IdP Metadata

组件	文件	估算行数
SamlSpMetadataGenerator	元数据生成	~300 行
SamlIdpMetadataParser	IdP 解析	~250 行
SamlConfigurationStore	配置持久化	~200 行
SamlMetadataApiResource	元数据 REST API	~150 行
小计		**~900 行**

模块 2: SAML AuthnRequest / Response 处理

功能：创建 AuthnRequest，解析并验证 SAMLResponse

组件	文件	估算行数
SamlAuthnRequestBuilder	请求构建	~300 行
SamlResponseParser	响应解析	~350 行
SamlAssertionValidator	断言验证	~400 行
XmlSignatureService	XML 签名验证	~300 行
SamlEncryptionService	断言解密	~250 行
小计		**~1,600 行**

模块 3: Shiro Realm 实现 (SamlRealm)

功能：将 SAML 集成进 Shiro 框架

组件	文件	估算行数
SamlRealm (Shiro Realm)	核心 Realm	~400 行
SamlUserManager	用户管理	~300 行
SamlAttributeMapper	属性映射	~250 行
SamlAuthenticationToken	Token 封装	~100 行
SamlFilter	Shiro Filter	~280 行
小计		**~1,330 行**

模块 4: SAML 登录流程 (SP-Initiated & IdP-Initiated)

功能：两种 SSO 发起方式

组件	文件	估算行数
SamlLoginServlet	登录入口	~200 行
SamlCallbackServlet	ACS 端点	~280 行
RelayStateManager	状态管理	~150 行
小计		**~630 行**

模块 5: SAML Single Logout (SLO)

功能: SP/IdP 发起的全局登出

组件	文件	估算行数
SamlSloFilter	SLO 过滤器	~250 行
SamlLogoutRequestBuilder	登出请求	~200 行
SamlLogoutResponseHandler	登出响应	~180 行
小计		**~630 行**

模块 6: 前端 SAML 配置 UI (全新)

功能: SAML 配置管理界面 (当前代码库无此 UI)

组件	估算行数
SamlConfiguration.jsx	~350 行
SamlConfigurationMachine.js	~200 行
SamlStrings.js	~100 行
元数据上传/下载组件	~150 行
SP 元数据展示面板	~120 行
前端小计	**~920 行**

SAML 后端总计: ~5,090 行 Java

3.3.2.1.4 依赖库评估及代码量汇总

依赖库评估

```xml
<!-- OIDC 所需新增 -->
<dependency>
  <groupId>com.nimbusds</groupId>
  <artifactId>oauth2-oidc-sdk</artifactId>  <!-- OIDC 标准实现 -->
</dependency>
<dependency>
  <groupId>com.nimbusds</groupId>
  <artifactId>nimbus-jose-jwt</artifactId>  <!-- JWKS/JWT 处理 -->
</dependency>

<!-- SAML 所需新增（二选一）-->
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-impl</artifactId>  <!-- 推荐，标准完整 -->
</dependency>
<!-- 或 -->
<dependency>
  <groupId>com.onelogin</groupId>
  <artifactId>java-saml</artifactId>  <!-- 轻量级替代 -->
</dependency>
```

汇总估算

协议	后端 Java	前端 JS/JSX	测试代码	**合计**
OIDC	~4,200 行	~550 行	~2,000 行	**~6,750 行**
SAML	~5,090 行	~920 行	~2,500 行	**~8,510 行**
共用基础设施	~500 行	~200 行	~300 行	**~1,000 行**
总计	**~9,790 行**	**~1,670 行**	**~4,800 行**	**~16,260 行**

开发时间估算

团队配置假设

- 后端工程师 2 名（熟悉 Java/Shiro/Spring）
- 前端工程师 1 名（熟悉 React/XState）
- QA 工程师 1 名

分阶段计划

```
Phase 1 — OIDC 核心（6 周）
├── W1-2: Discovery、Config 存储、DB 迁移
├── W3-4: Auth Code Flow + PKCE + Callback
├── W5:   Token 验证 + UserInfo + Claim 映射
└── W6:   OAuth2Realm 实现 + 集成测试

Phase 2 — OIDC 完善（2 周）
├── W7:   Logout + Token Refresh
└── W8:   前端完善 + E2E 测试

Phase 3 — SAML 核心（7 周）
├── W9-10:  SP/IdP Metadata 处理
├── W11-12: AuthnRequest/Response + XML 签名
├── W13:    SamlRealm + 属性映射
├── W14:    ACS/SLO Servlet
└── W15:    前端 SAML UI

Phase 4 — 集成与加固（3 周）
├── W16:  安全审计（签名绑定、重放攻击防护）
├── W17:  多 IdP 测试（Keycloak/Okta/Azure AD）
└── W18:  文档、性能测试、上线准备
```

场景	时间	备注
仅 OIDC	**8 周**	2 后端 + 1 前端
仅 SAML	**10 周**	2 后端 + 1 前端
OIDC + SAML 同时	**16-18 周**	部分可并行
仅 OIDC（资深团队）	**5-6 周**	有 OpenSAML/Nimbus 经验

3.3.2.1.5 关键风险点

关键风险点

1. **XML 签名验证复杂性** — SAML 的 XML Canonicalization 是高频踩坑区，建议使用 OpenSAML 而非自行实现
2. **IdP 兼容性测试** — Okta/Azure AD/Keycloak 行为差异较大，需各自验证
3. **Session 并发安全** — Shiro + OIDC/SAML Session 共存时的竞态条件
4. **现有私有模块冲突** — RealmToSource 中已有 OAuth2/SAML 引用，需确认私有模块边界避免重复实现
5. **PKCE 与旧 IdP 兼容** — 部分企业 IdP 不支持 PKCE，需降级兼容策略

✻ Baked for 4m 19s

› 如果用大模型claude或codex 开发周期多久

• **使用 Claude / Codex 辅助开发的周期评估**

核心前提：LLM 是加速器，不是替代者

LLM 在代码生成上效率极高，但以下环节仍强依赖人工：
- 架构决策与模块边界划定
- 与现有 Shiro/Spring 框架的深度集成调试
- 安全边界验证（签名绑定、重放攻击）
- 跨 IdP 兼容性测试

各阶段 LLM 加速效果分析

任务类型	传统开发	LLM 辅助	加速倍数	说明
样板代码（DTO/Config/Store）	高	极低	**5-8x**	Claude 一次生成
标准协议实现（PKCE/JWKS/XML签名）	高	低	**3-5x**	有大量训练数据
Shiro Realm 集成	中	中	**2-3x**	需理解现有框架上下文
前端 React/XState 组件	中	低	**3-4x**	参考现有组件风格生成
安全审计与边界测试	低	低	**1.2-1.5x**	人工不可替代
IdP 兼容性调试	极低	极低	**1.1-1.3x**	运行时问题，LLM 帮助有限
单元测试生成	高	极低	**5-8x**	Claude 擅长

3.3.2.1.6 用大模型实现工作量预估

重新估算开发周期

团队配置（LLM 辅助模式）：
- 1 名后端工程师（主导，LLM 为副驾）
- 1 名前端工程师（兼职，50% 时间）
- 1 名 QA（后期介入）

OIDC 实现

```
传统：8 周 (2 后端 + 1 前端)
LLM:  3-4 周 (1 后端 + 0.5 前端)

Week 1: Discovery + Config + DB迁移 + 模型层
        └─ Claude 生成 ~80% 样板，工程师审查集成
Week 2: Auth Code Flow + PKCE + Callback Servlet
        └─ 参考 RFC 7636，Claude 直接生成实现
Week 3: Token验证 + JWKS + Claim映射 + OAuth2Realm
        └─ Shiro集成是难点，需人工调试
Week 4: Logout + Refresh + 前端完善 + 集成测试
        └─ E2E调试，Claude辅助写测试用例
```

OIDC: 3-4 周

SAML 实现

```
传统：10 周 (2 后端 + 1 前端)
LLM:  5-6 周 (1 后端 + 0.5 前端)

Week 1-2: Metadata解析/生成 + 配置存储
          └─ OpenSAML API 调用 Claude 熟悉
Week 3-4: AuthnRequest/Response + XML签名验证
          └─ 这是最复杂部分，Claude能生成框架
            但 XML Canonicalization 细节需人工验证
Week 5:   SamlRealm + 属性映射 + ACS Servlet
Week 6:   SLO + 前端UI + 多IdP测试
          └─ Okta/Azure AD 兼容问题是纯运行时，LLM帮助最小
```

综合对比

方案	团队规模	OIDC	SAML	OIDC+SAML	人力成本对比
传统开发	3人	8周	10周	18周	基准
LLM 辅助	1.5人	3-4周	5-6周	8-10周	**约 1/4**
LLM 重度使用*	1人	5-6周	7-8周	12-14周	**约 1/3**

> **重度单人模式：工程师经验不足时，理解和验证 LLM 输出反而变慢*

LLM 工作流建议（实操）

最高效的 Claude Code 使用方式：

```
1. 架构设计（人工）
   └ 画出模块边界、接口定义、数据流

2. 接口先行（Claude）
   └ "基于以下接口定义和现有 Shiro 框架，生成 OAuth2Realm 实现"
   └ 提供 NexusAuthenticatingRealm 作为参考上下文

3. 批量生成 + 审查（Claude + 人工）
   └ 一次性生成 DTO / Store / Migration / Test
   └ 工程师只做 Code Review，不从零写

4. 集成调试（人工主导，Claude 辅助）
   └ 运行时错误粘贴给 Claude 分析
   └ 但最终判断靠工程师

5. 测试覆盖（Claude）
   └ "为以下 IdTokenValidator 生成边界测试用例"
   └ 安全场景：过期token、错误签名、重放攻击
```

关键结论

结论	说明
最快落地 OIDC	1 名熟悉 Shiro 的工程师 + Claude Code，**3.5 周**可上线
SAML 不能大幅压缩	XML 签名 + IdP 兼容是运行时问题，LLM 解决不了
最大价值在测试生成	LLM 生成安全边界测试用例，比人工写快 6-8 倍
风险在上下文窗口	本项目代码量大，需要精心设计 prompt 上下文，避免 LLM 生成与现有框架不兼容的代码
推荐工具组合	Claude Code（架构理解+生成）+ GitHub Copilot（行级补全）

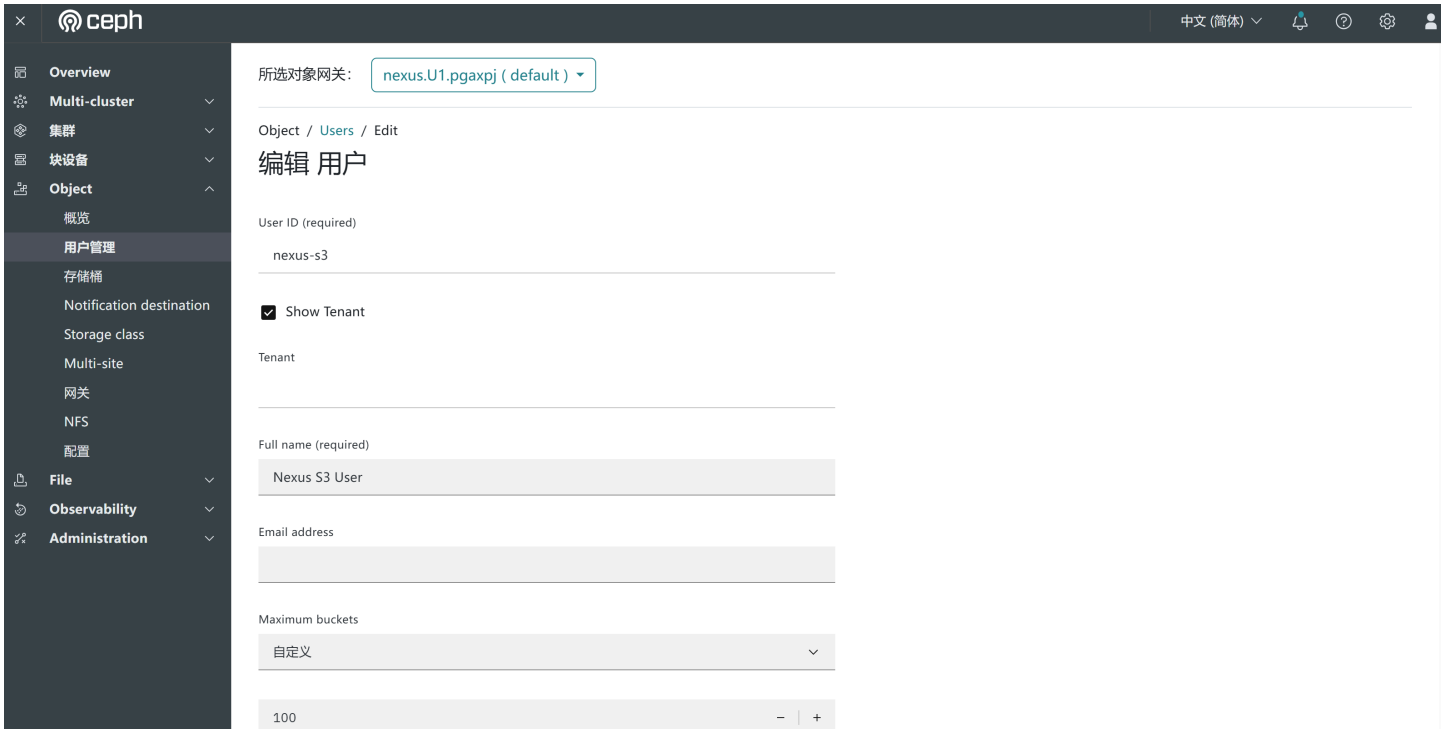
3.3.3 nexus配置后端存储

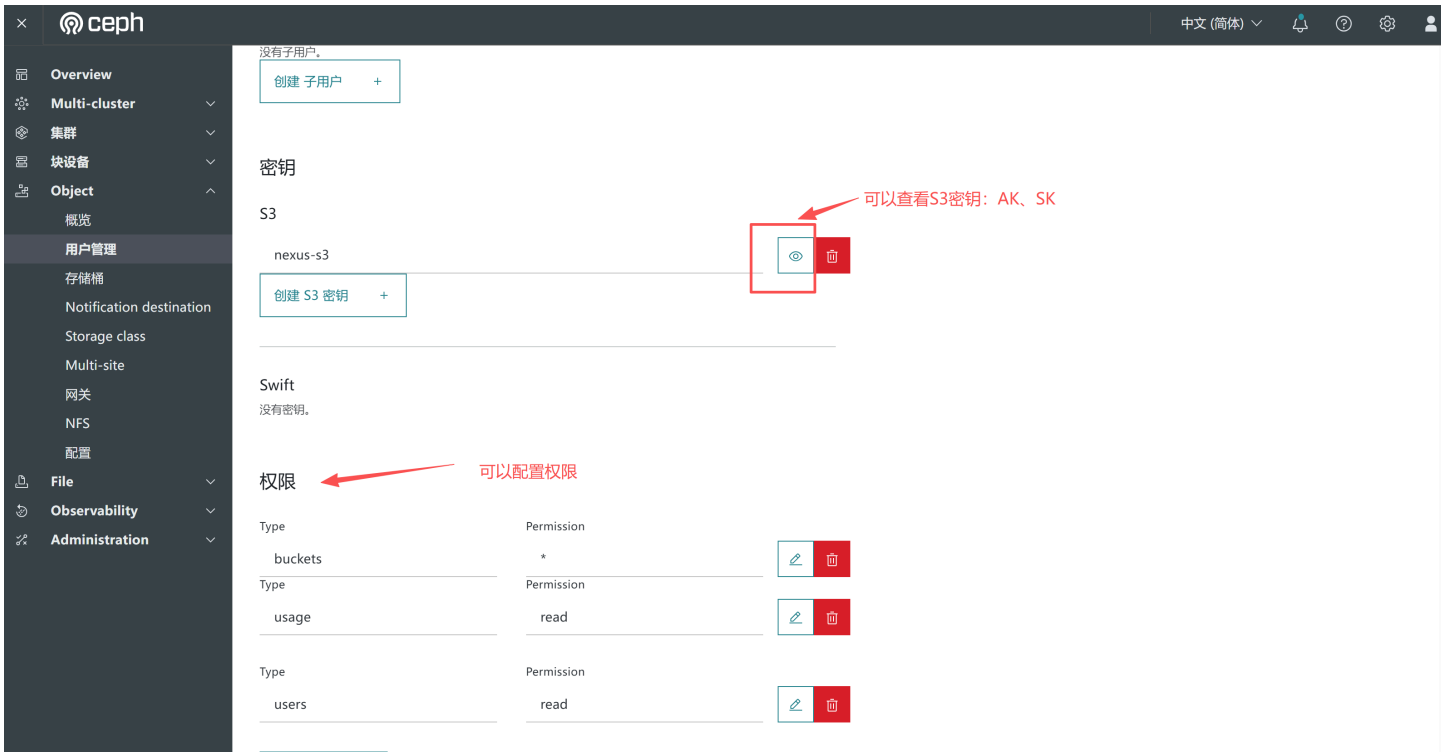
3.3.3.1 nexus配置S3对象存储

1. Ceph 创建S3 对象存储

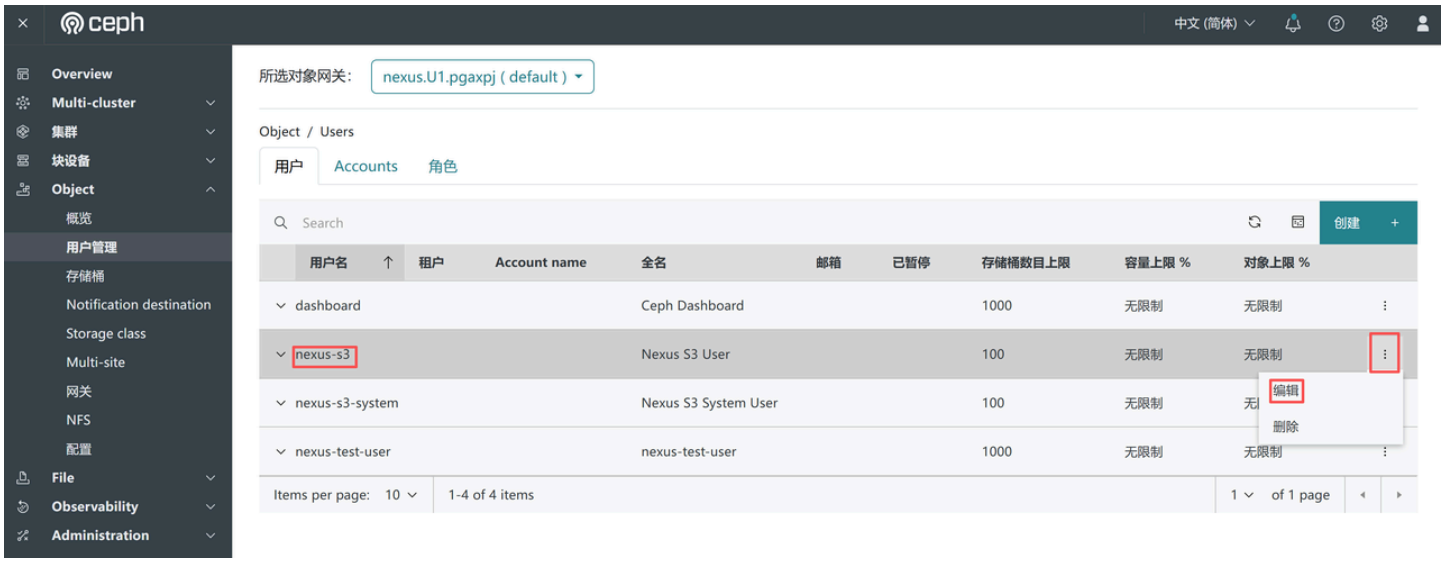
   a. 创建S3兼容用户

在ceph dashboard中， Object-->用户管理创建用户



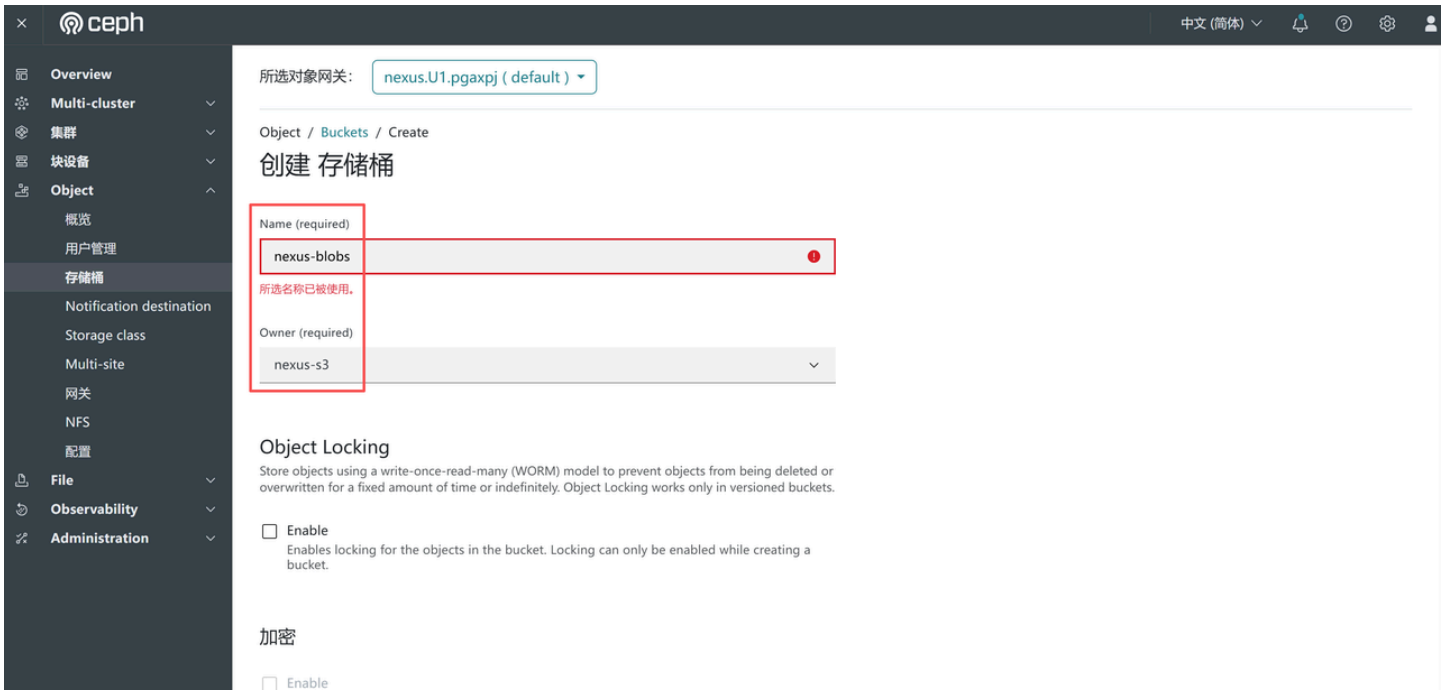


创建完用户后，想要再次查看用户密钥，找到用户可以通过“编辑”进入查看



b. 创建存储桶（Bucket）

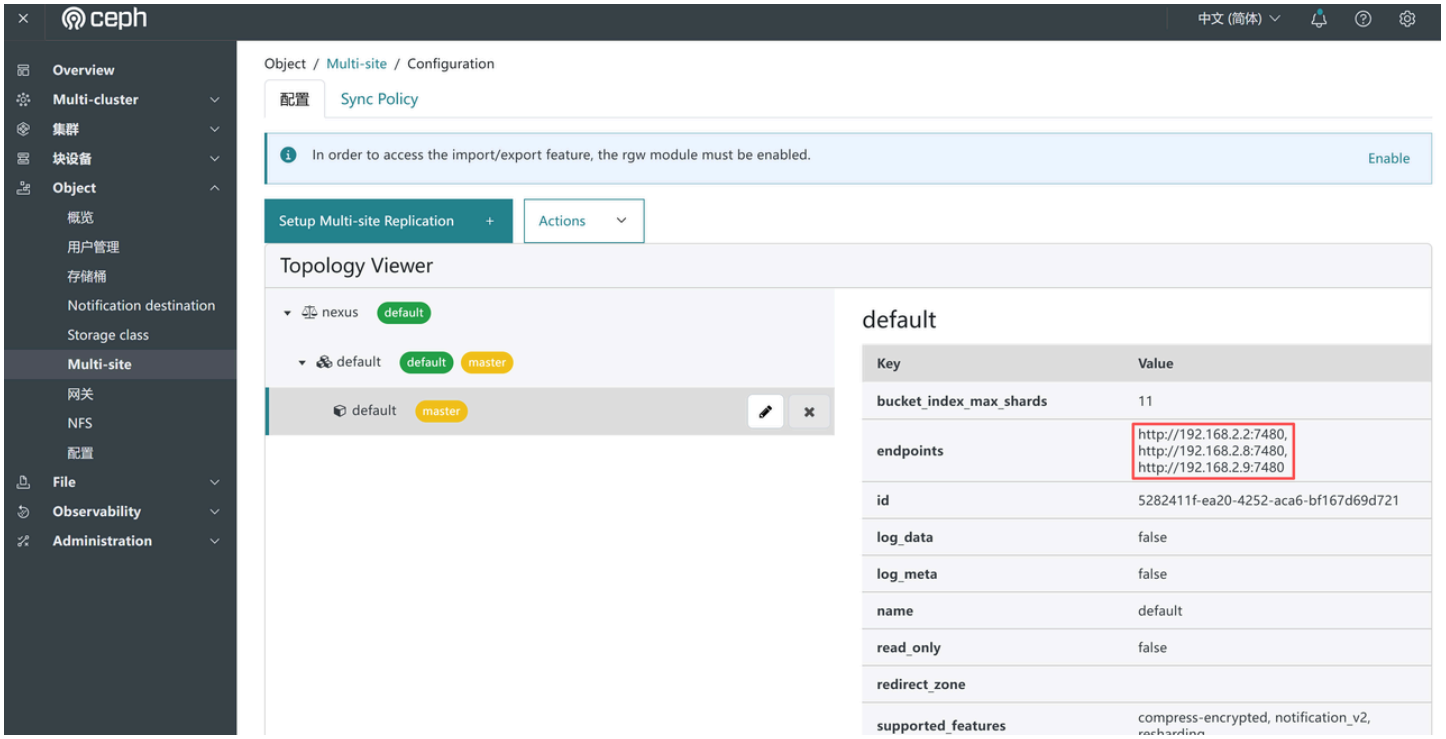
在dashboard中，点击存储桶进行存储桶的创建，填写存储桶名称和选择所属用户，进行创建





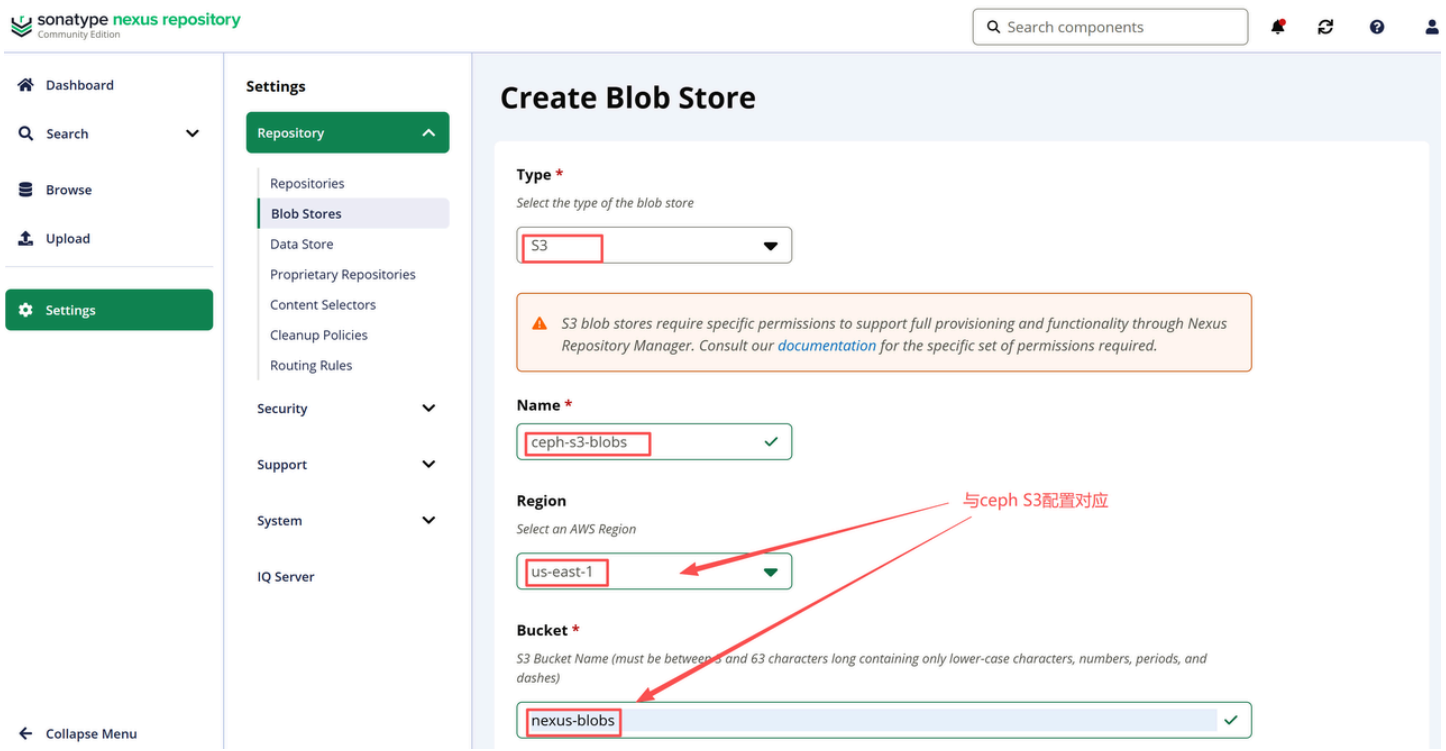
c. 获取RGW服务访问地址

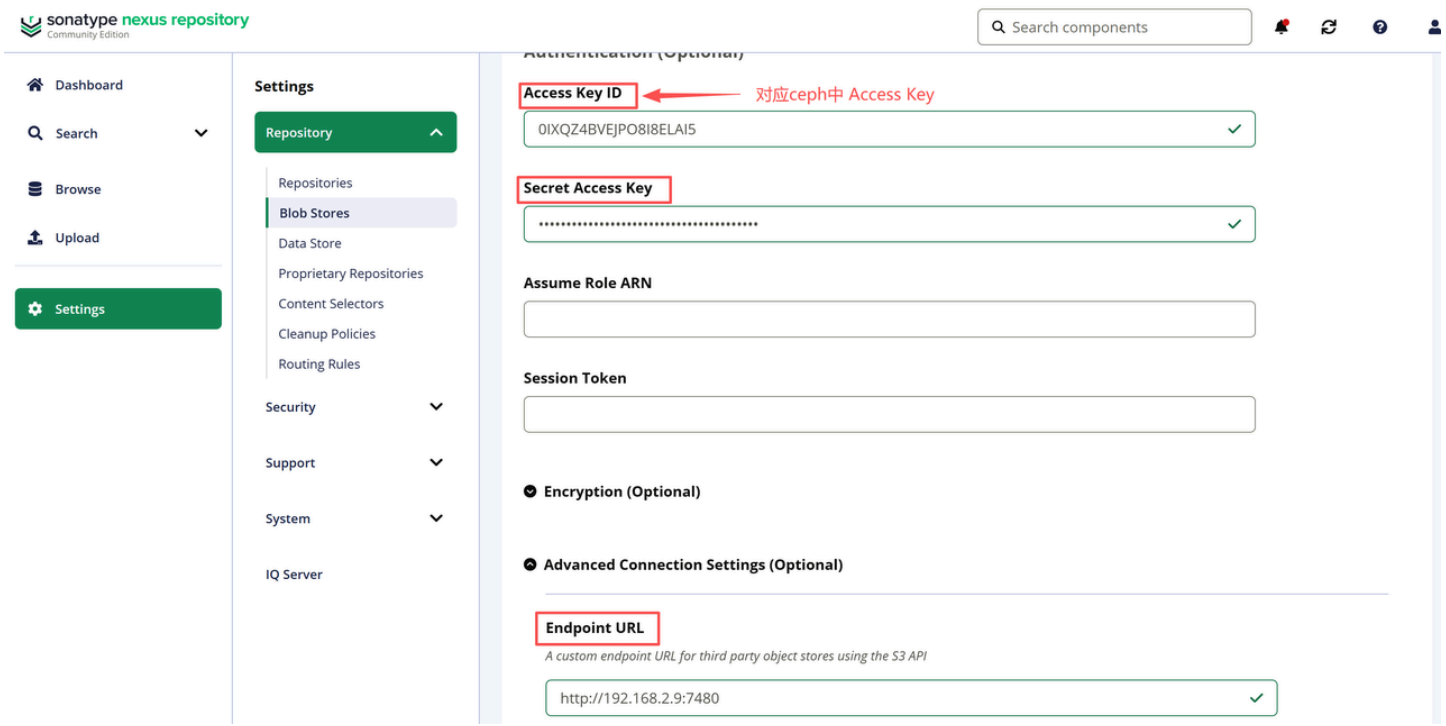
在dashboard点击Multi-site，在用侧查看endpoints，找到dashboard登录一致的endpoints并记录



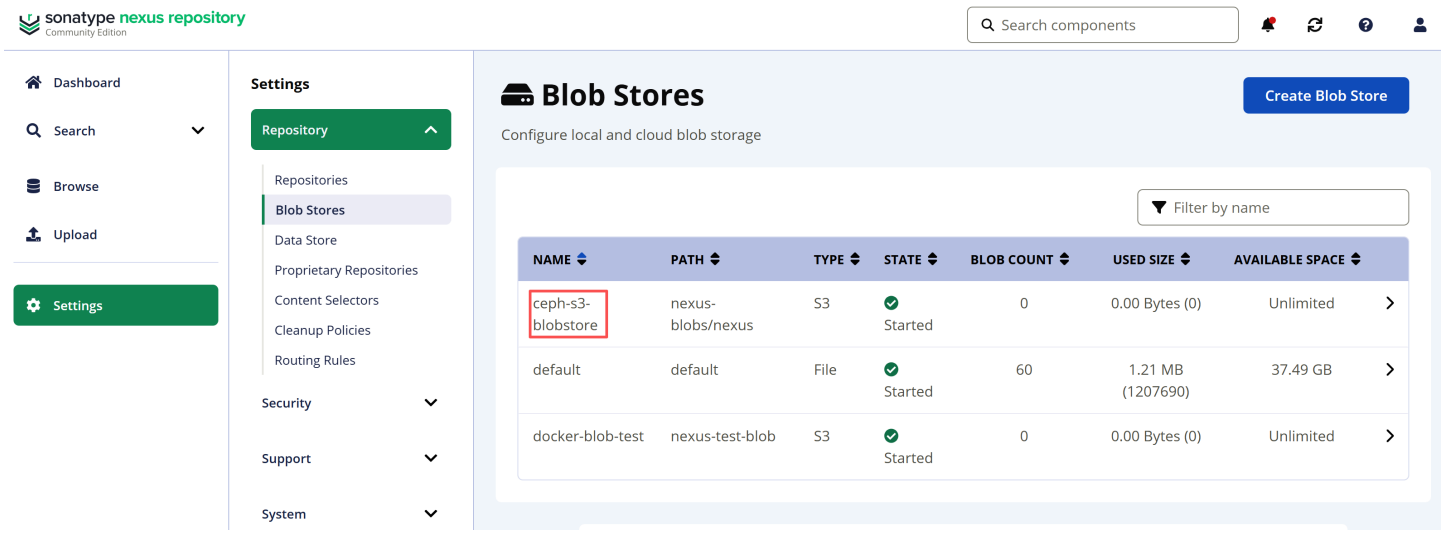
2. neuxs配置ceph S3对象存储

在neuxs dashboard 点击settings 选择Blob Stores创建Blob Store，如下图，类型选择S3，填写存储名字，选择ceph 中网关区域





配置好后创建Blob。



3.3.3.2 nexus配置NAS后端存储

第一步：配置前准备

1. 创建操作系统用户
   建议为Nexus创建一个专用的系统用户（例如 `nexus`），并确保该用户对NAS挂载点有完全的读写权限，这是后续所有操作成功的基础。
2. 安装并启动Nexus
   NFS是二进制文件的存储最佳选择，但请注意 `nexus` 用户的UID/GID必须与NFS服务器端设置一致（如需匿名访问，映射到nobody/nogroup即可）。完成安装后，在浏览器通过 `http://<你的服务器IP>:8081` 访问Nexus管理界面。

第二步：挂载NAS存储并创建Blob Store

1. 挂载NAS文件系统
   登录Ubuntu 24.04服务器，按顺序执行以下命令。
   - 安装NFS客户端（若使用NFS） / SMB客户端（若使用SMB）：

```
代码块
# 如果NAS提供NFS共享
sudo apt update
sudo apt install nfs-common -y

# 如果NAS提供SMB/CIFS共享
sudo apt update
sudo apt install cifs-utils -y
```

   - 创建挂载点并挂载共享文件夹：

```
代码块
# 创建一个目录作为挂载点
sudo mkdir -p /mnt/nexus_nas_data

# 如果是NFS共享，执行以下命令（请替换为你的NAS IP和共享路径）
sudo mount -t nfs <NAS_IP_ADDRESS>:/path/to/nas/share /mnt/nexus_nas_data

# 如果是SMB/CIFS共享，执行以下命令（请替换为你的NAS IP、共享名、用户名和密码）
sudo mount -t cifs //<NAS_IP_ADDRESS>/share-name /mnt/nexus_nas_data -o
username=<你的NAS用户名>,password=<你的密码>,uid=$(id -u nexus),gid=$(id -g
nexus),vers=3.0
```

为确保服务器重启后挂载不会丢失，强烈建议将挂载配置写入 `/etc/fstab` 文件以实现持久化挂载。例如，可以添加如下NFS配置：

`<NAS_IP_ADDRESS>:/nexus /opt/sonatype-work nfs defaults 0 0`。

   - ⚠️ 性能优化关键（以 NFS v4.1 为例）：官方和社区经验都强调，为了获得最佳性能，挂载时应使用特定的参数：

```
代码块
# 推荐的挂载选项，能显著提升I/O性能
```

```
2   sudo mount -t nfs -o
    defaults,noatime,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,vers=4.1
    <NAS_IP_ADDRESS>:/path/to/nas/share /mnt/nexus_nas_data
```

参数解释🔧：

`noatime`：读取文件时不更新访问时间，减少磁盘写入次数，提升性能。

`rsize=1048576, wsize=1048576`：将读写缓冲区大小设置为1MB，能有效提升网络吞吐性能。

`hard`：当NFS服务器无响应时，客户端操作会一直等待直至恢复，确保数据不丢失。

`timeo=600, retrans=2`：设置NFS请求超时时间（60秒）与重传次数，适用于高延迟网络环境。

`vers=4.1`：Nexus官方强制要求使用NFS v4或更高版本，不支持更老的v3版本。

2. 在Nexus中创建外部Blob Store
   挂载完成后，就可以在Nexus的Web界面中将该目录添加为存储空间了。
   - 登录Nexus控制台：使用管理员账号登录，通常在 `http://<your-server-ip>:8081`。
   - 进入Blob Stores配置页面：在顶部菜单栏中，依次点击 `Administration`（服务器齿轮图标） -> `Repository` -> `Blob Stores`。
   - 创建新的Blob Store：
     i. 点击页面右侧的 `Create Blob Store` 蓝色按钮。
     ii. 在弹出的窗口中，确保 Type 选择为 `File`。
     iii. 在 Name 字段为其起一个清晰的名字，例如 `nas-storage`。
     iv. 在 Path 字段，填入你在Ubuntu上创建的NAS挂载点路径，例如 `/mnt/nexus_nas_data`。
     v. （可选）设置一个 Soft Quota（软配额），例如 `100 GB`。当存储空间超过此阈值时，系统会在日志和管理界面发出警告，有助于避免因磁盘写满导致的数据损坏。
     vi. 点击 `Create` 按钮完成创建。

第三步：将Nexus仓库数据迁移到NAS

新创建的Blob Store默认是空的，你需要将现有仓库的数据迁移过去或将其分配给新建的仓库。

- 为现有仓库迁移数据：
  - 官方推荐使用 `nexus-blobstore-move` 任务。

> - 注意：创建仓库时选择的Blob Store存储后无法修改。迁移并非直接更改设置，而是通过任务完成。

    - 操作步骤：
        i. 前置操作：进入 `Administration` → `System` → `Capabilities`，启用 `Storage Quota` 并设置阈值（例如500GB）以建立监控机制。
        ii. 在Nexus后台中，导航至 `Administration` → `System` → `Tasks`。
        iii. 点击 `Create Task`，任务类型选择 `nexus-blobstore-move`。
        iv. 在任务配置中，选择需要移动数据的 Source Blob Store (当前存储) 和目标 Destination Blob Store (`nas-storage`)。
        v. 根据需要选择 Source Repository（指定某个仓库或全部仓库），然后保存并运行此任务。
        vi. 系统会将选中的仓库的所有二进制文件从旧存储迁移到新的NAS Blob Store。
- 为未来新建的仓库分配存储：
  在你创建任何新的 `Hosted`、`Proxy` 或 `Group` 类型仓库时，配置向导的 `Storage` 部分会有一个 Blob store 下拉选项。届时，直接选择 `nas-storage` 即可让该仓库的数据直接存入NAS。

通过以上步骤，可以将Nexus的后端存储配置到NAS上。

📌 关键注意事项

- 网络与环境：
    - 低延迟：确保Nexus服务器与NAS之间的网络延迟极低。
    - 稳定性：Nexus的生产环境不支持SMB/CIFS协议，强烈建议使用NFS v4.1或更高版本，并确保网络稳定。
    - 用户权限：在Nexus服务器和NAS上，运行Nexus进程的用户必须拥有对挂载目录的读写权限。
    - 防火墙：确保服务器的防火墙允许NFS（端口2049）或SMB（端口445等）的通信流量。

3.3.4 仓库管理

Nexus中主要的仓库类型：

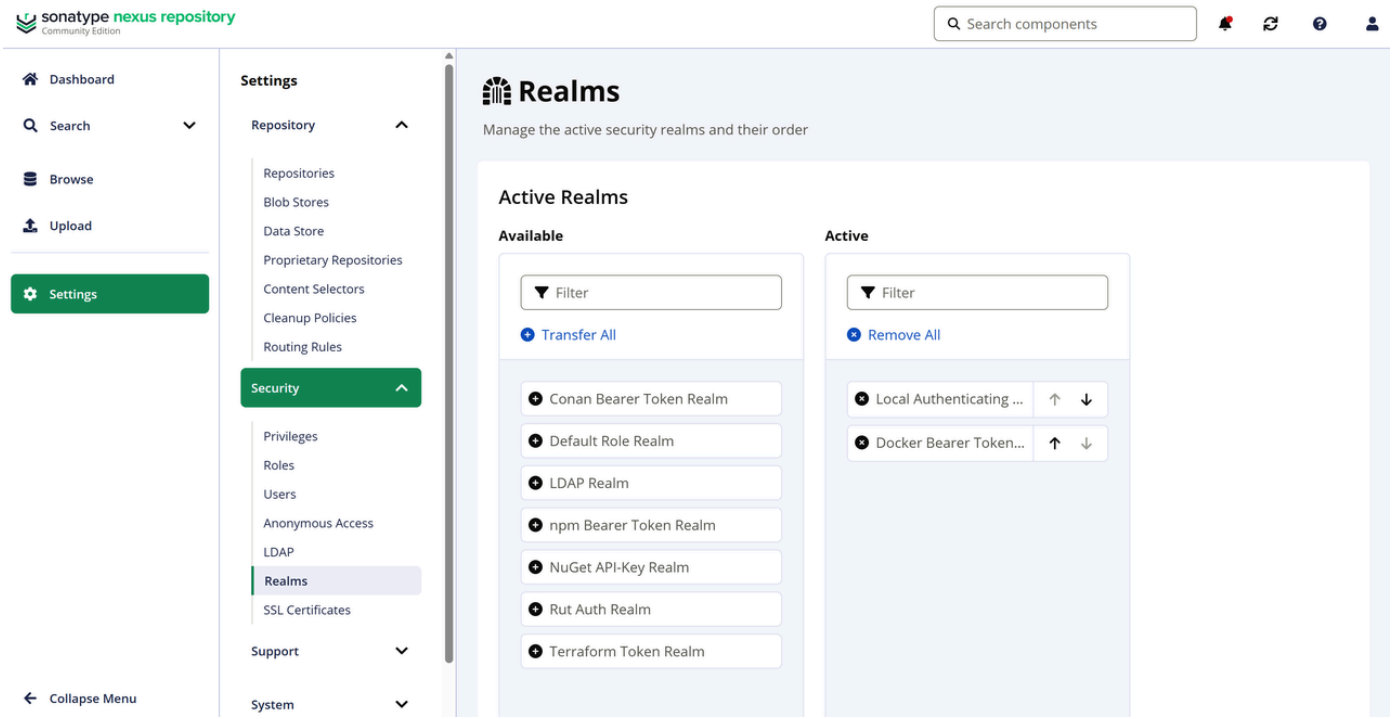
- hosted (私有仓库)
    - 用途：存储自己构建、不公开的私有镜像，提供了一个安全的私有存储空间。
- proxy (代理仓库)
    - 用途：代理并缓存外网（如 `registry-1.docker.io` ）的镜像。拉取过的镜像会缓存在内网，能有效加速后续拉取。
- group (聚合仓库)
    - 用途：将 `hosted` 和 `proxy` 仓库组合成一个统一地址。拉取镜像时，会优先在 `hosted` 查找，没有再通过 `proxy` 代理，兼顾了私有与效率。

3.3.4.1 容器镜像管理

1. nexus配置

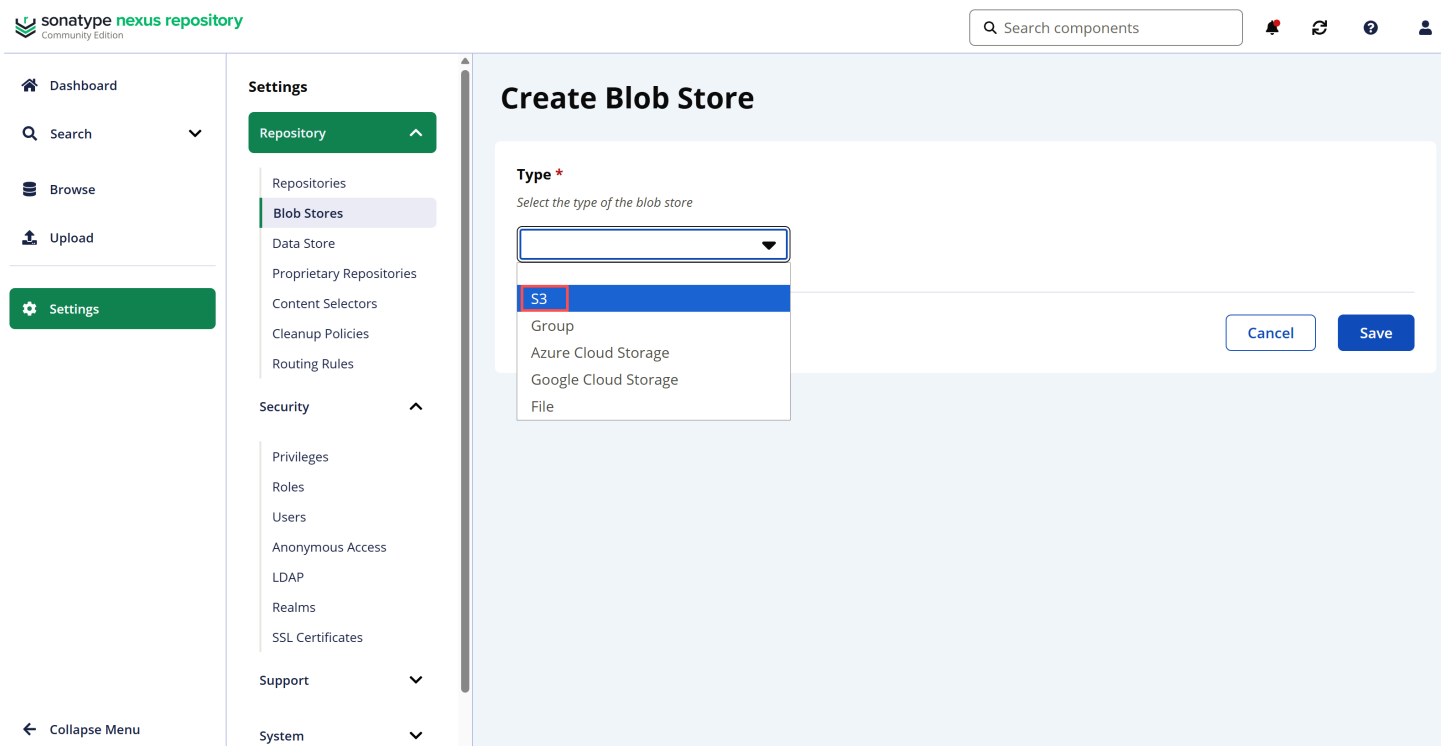
   a. 在 Nexus 中启用 Docker 认证 Realm

   - 登录 Nexus，点击左侧菜单的 Security (安全) → Realms (领域)。
   - 右侧的 Active (可用列表) 中，检查是否有 Docker Bearer Token Realm。
   - 如果没有，从左侧的 Available (可用领域) 中选中并移至右侧，点击 Save (保存)



   b. 创建存储空间 (Blob Store)

   - 进入 Administration (管理) → Blob Stores (存储)。
   - 点击 Create Blob Store (创建存储) → 类型选择S3。
   - 填写 Name (名称)，如 `docker-blobs` ，点击 Create Blob Store。



3. 创建Hosted、Proxy和Group三种类型仓库

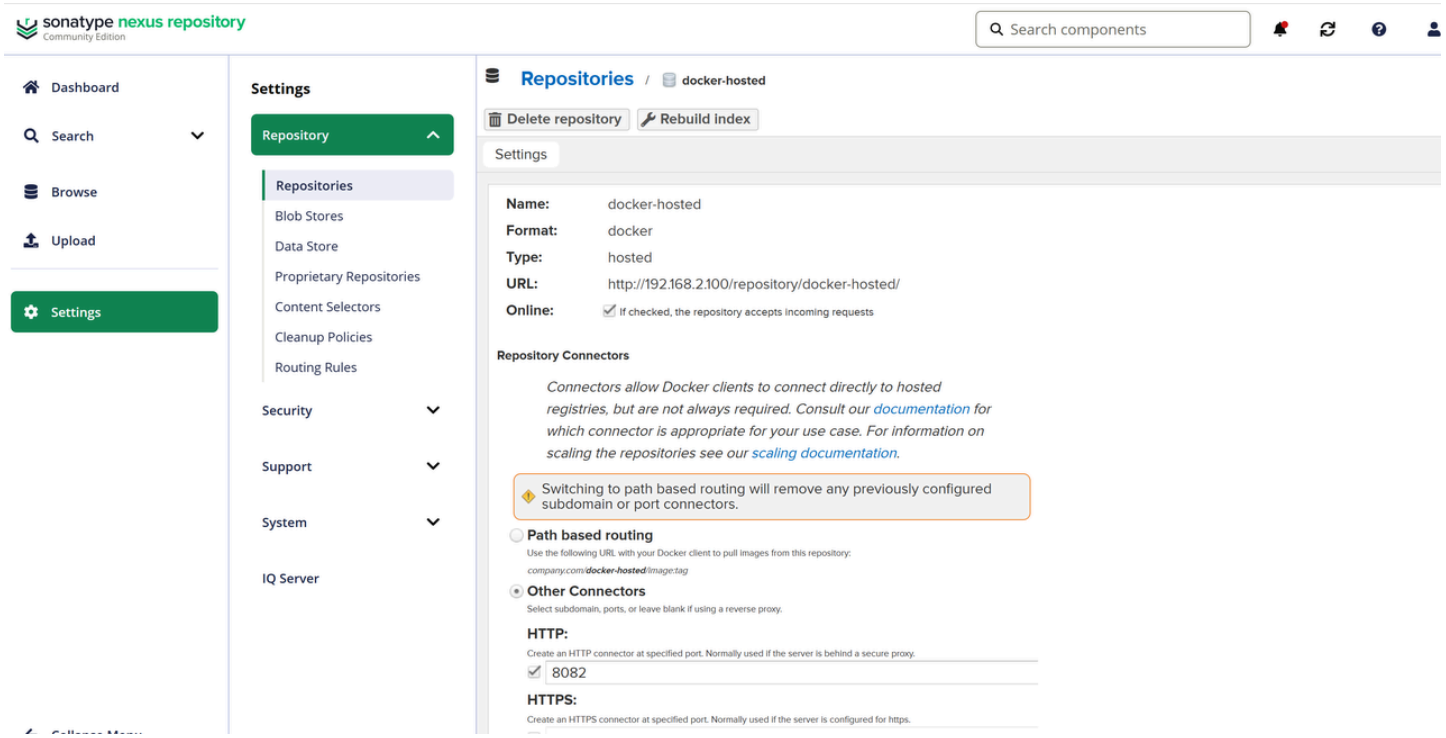
   创建 Hosted 仓库 (私有仓库)

   a. 进入 Administration → Repository → Repositories。

   b. 点击 Create repository (创建仓库)，选择类型 docker (hosted)。

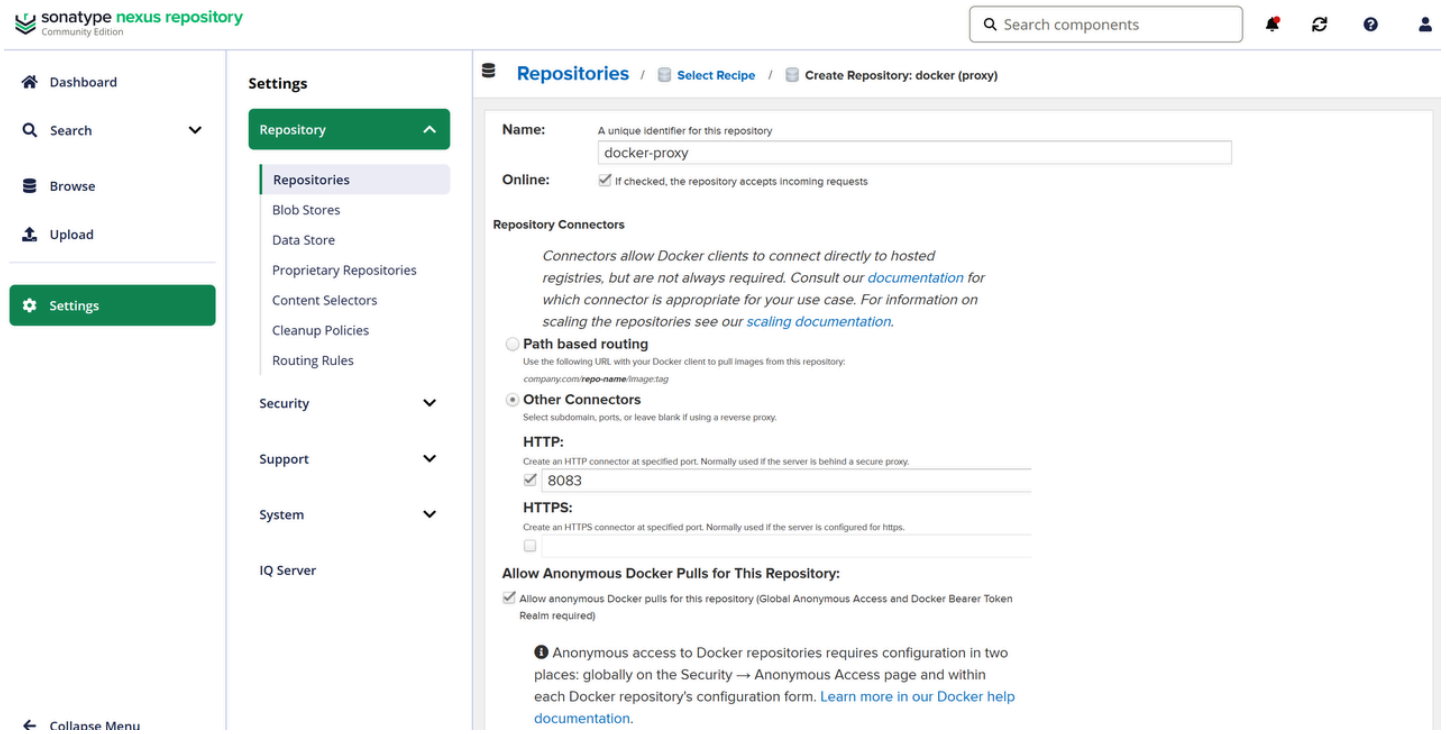
   c. 关键配置：

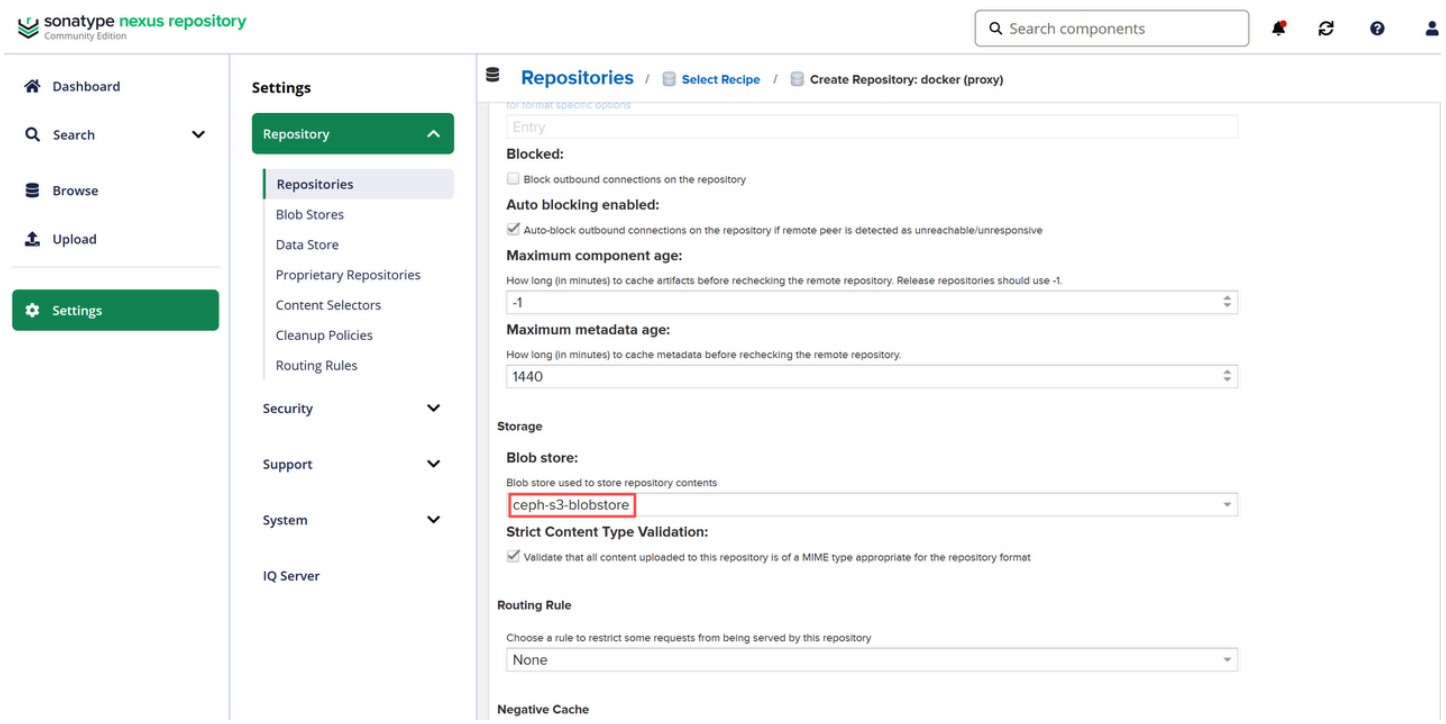
      - Name：填写仓库名称，如 `docker-hosted` 。
      - HTTP (或HTTPS)：必须勾选并设置端口，如 8082，这个端口供Docker客户端使用。（注意这里不要设置8082，不知道是否有限制， 设置该端口后始终不能正确登录，换成其他端口可以）
      - Blob store：存储空间选择上一步创建的 ceph-s3-blobstore。
      - Allow anonymous docker pull：如果您希望用户可以匿名拉取镜像，可勾选此项。但如果启用了强制基本认证，则此项会失效。



创建 Proxy 仓库 (代理仓库)

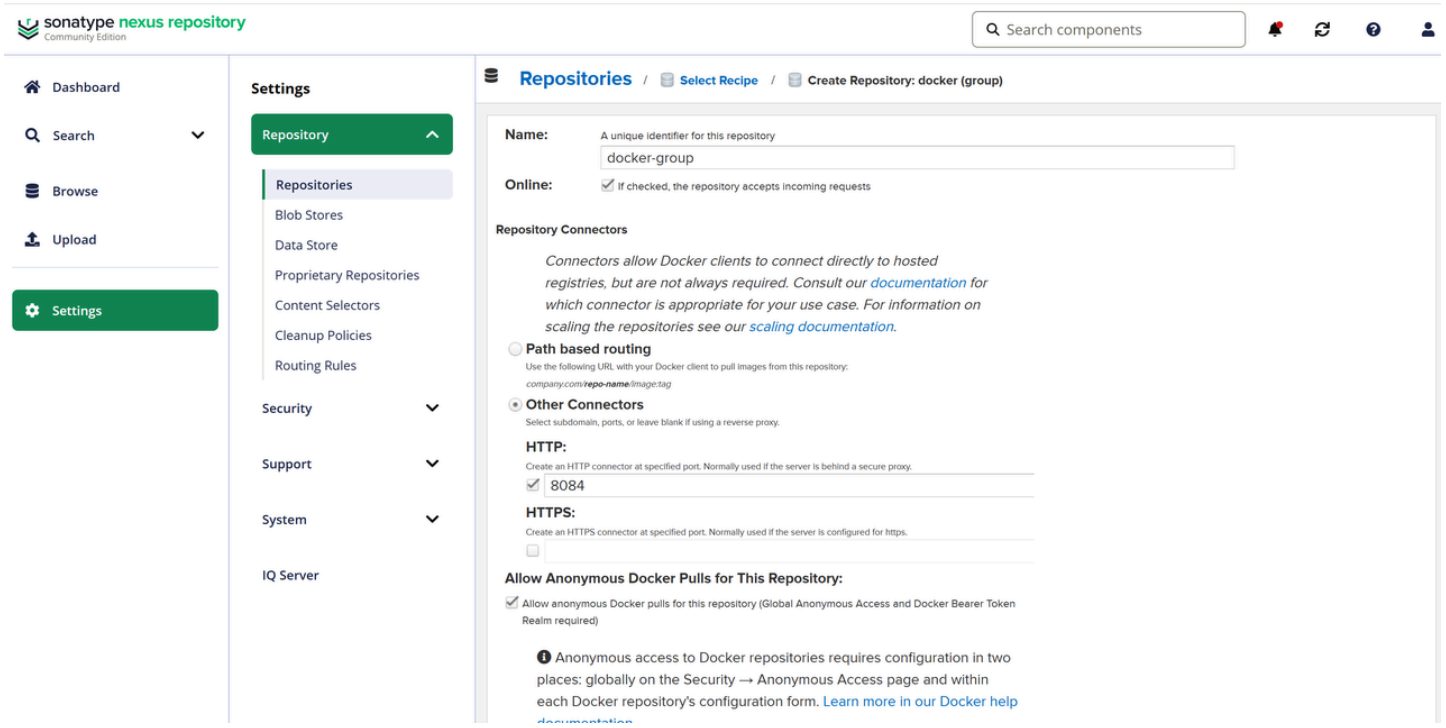
1. 点击 Create repository → 类型选择 docker (proxy)。
2. 关键配置：
   - Name：如 `docker-proxy`。
   - HTTP (或HTTPS)：设置端口，如 8083。
   - Remote storage (远程存储地址)：填写上游镜像源URL，如 https://mirrors.ustc.edu.cn/docker-ce/ (中科大docker源)。
   - Blob store：选择 ceph-s3-blobstore 或新建存储空间。

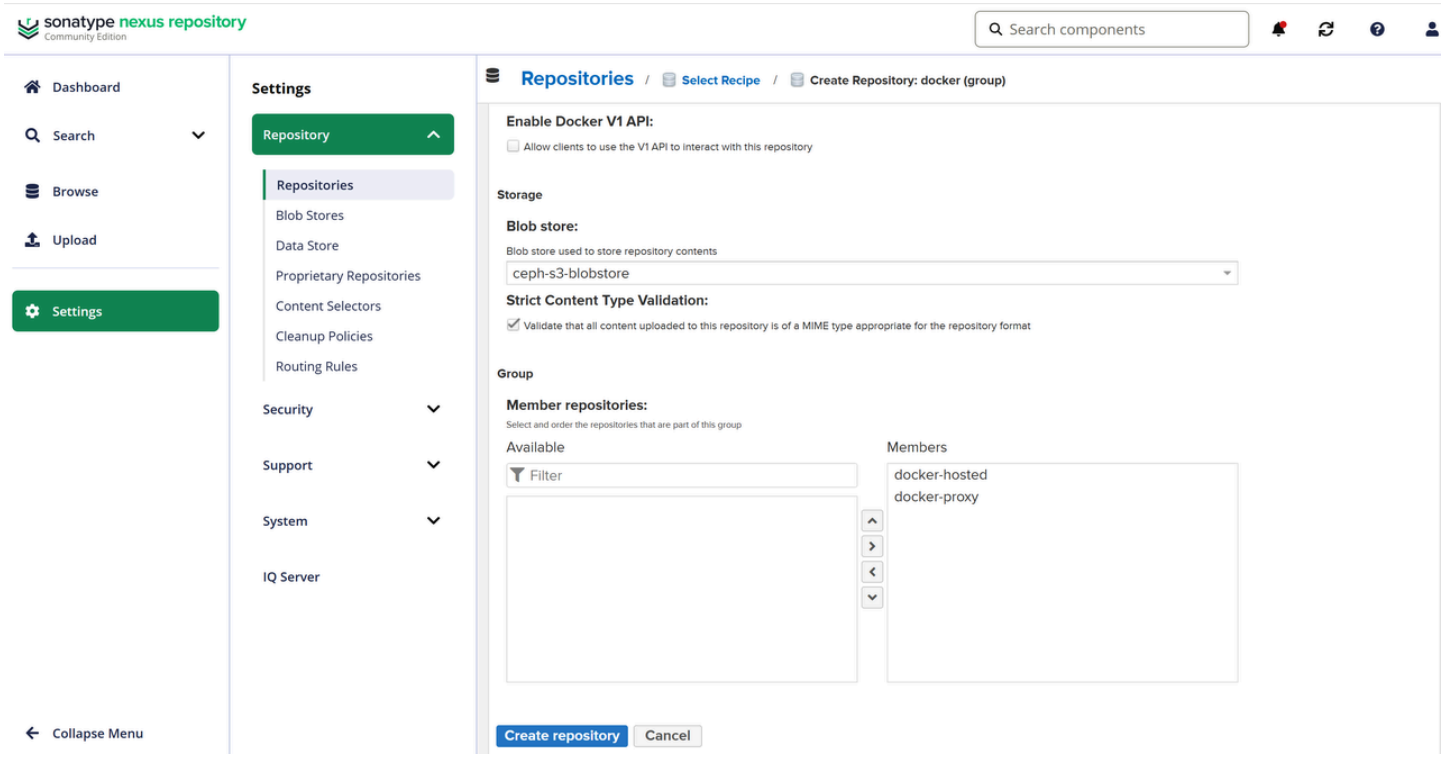


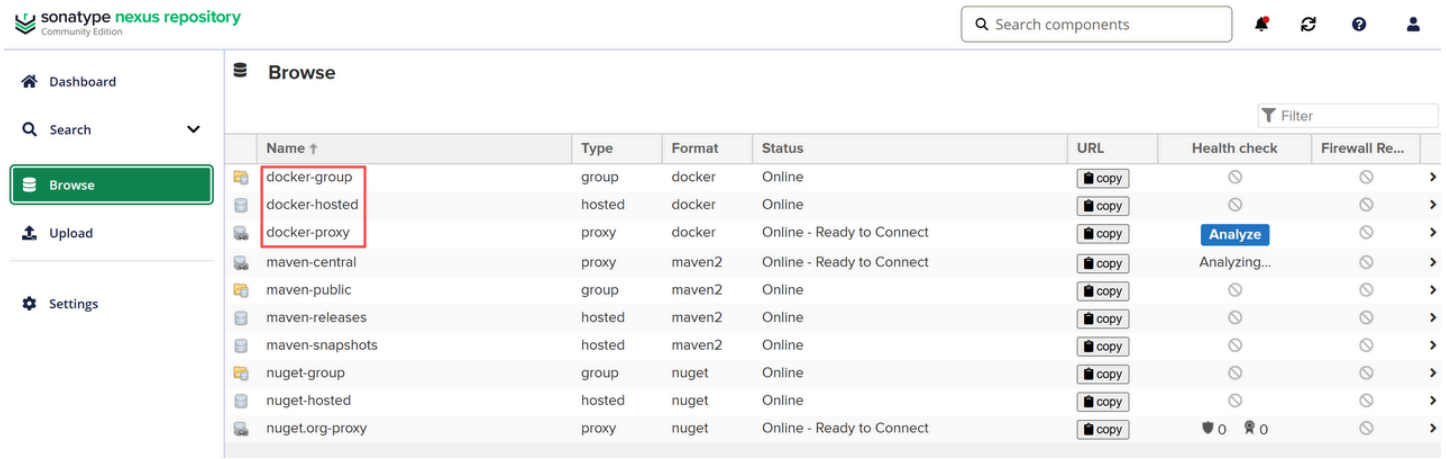


创建 Group 仓库 (聚合仓库)

1. 点击 Create repository → 类型选择 docker (group)。 （推送镜像时只能往hosted仓库推送）
2. 关键配置：
   - Name：如 `docker-group`。
   - HTTP (或HTTPS)：设置端口，如 8084。
   - Blob store：选择 ceph-s3-blobstore 或新建存储空间。
   - Member repositories (成员仓库)：将刚才创建的 `docker-hosted` (私有) 和 `docker-proxy` (代理) 从左侧移动到右侧。







3. 客户端配置

在镜像构建节点上，配置 Docker 守护进程以信任非 HTTPS 的 Nexus 仓库地址：

1. 编辑 `/etc/docker/daemon.json`，若不存在则创建。
2. 添加以下内容，将 `192.168.x.x` 和 `8881` 替换为真实地址：

```
代码块
1  {
2    "insecure-registries": ["192.168.x.x:8881", "192.168.x.x:8882",
   "192.168.x.x:8883"]
3  }
```

3. 配置保存后，重启 Docker

```
代码块
```

```
sudo systemctl daemon-reload && sudo systemctl restart docker
```

登录

docker login 192.168.x.x:8881

输入 Nexus 的用户名和密码进行认证。

拉取镜像 (`pull`)

接下来验证三种类型仓库的功能：

- 通过 Group 仓库拉取：推荐此方式，它会在私有仓库中查找失败后再尝试从代理仓库远程拉取

代码块

```
docker pull 192.168.x.x:8883/nginx:latest
```

- 通过 Proxy 仓库拉取：直接使用代理仓库拉取。

代码块

```
docker pull 192.168.x.x:8882/nginx:latest
```

推送镜像 (`push`)

需要推送到 `Hosted` 仓库。

1. 重打标签：必须将本地镜像的标签指向 `Hosted` 仓库的地址

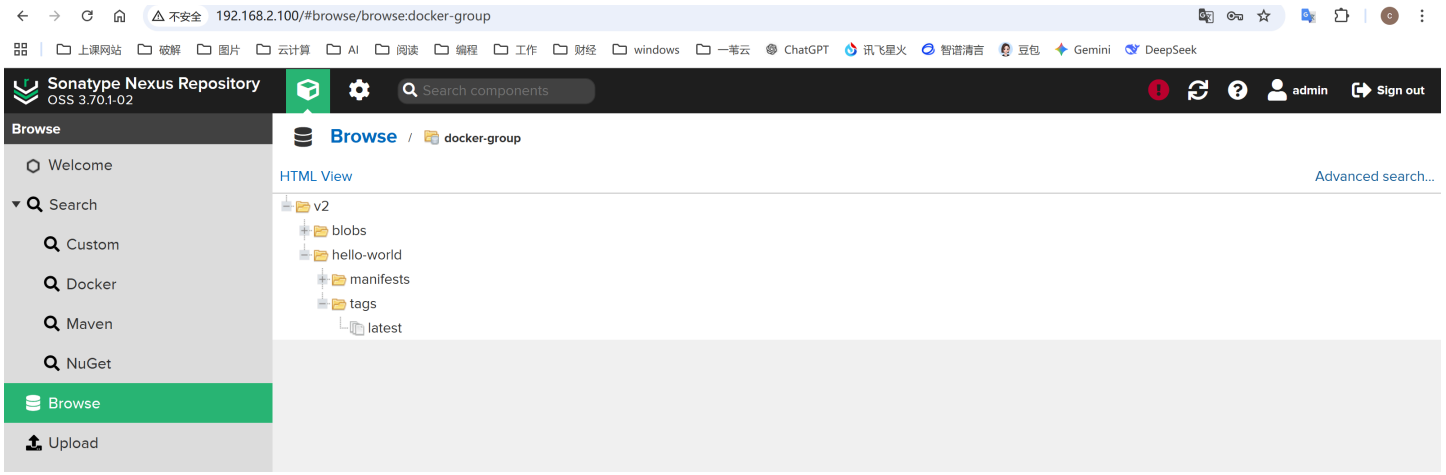
代码块

```
docker tag busybox:latest 192.168.x.x:8881/busybox:latest
```

2. 推送：

代码块

```
docker push 192.168.x.x:8881/busybox:latest
```



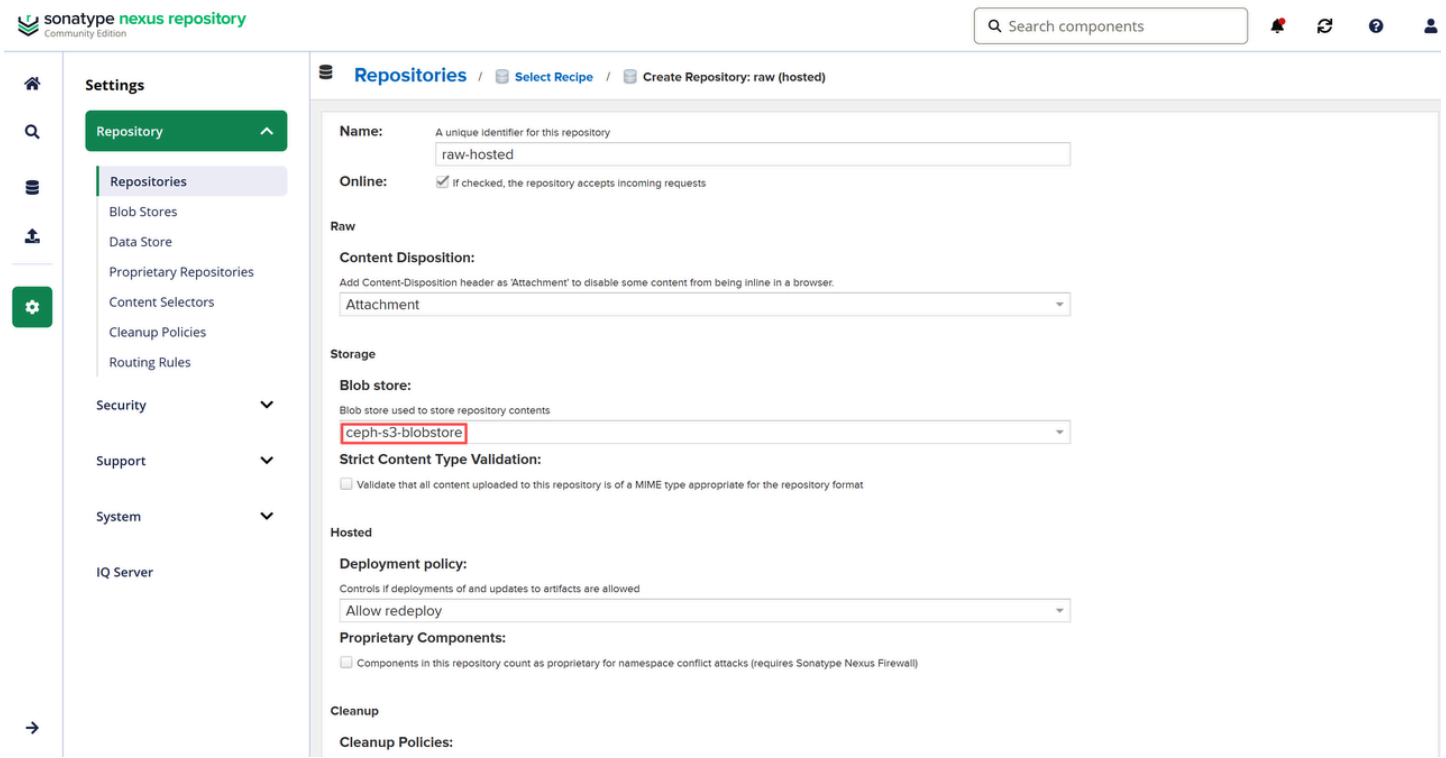
3.3.4.2 二进制文件管理

这里以ISO镜像文件管理为例

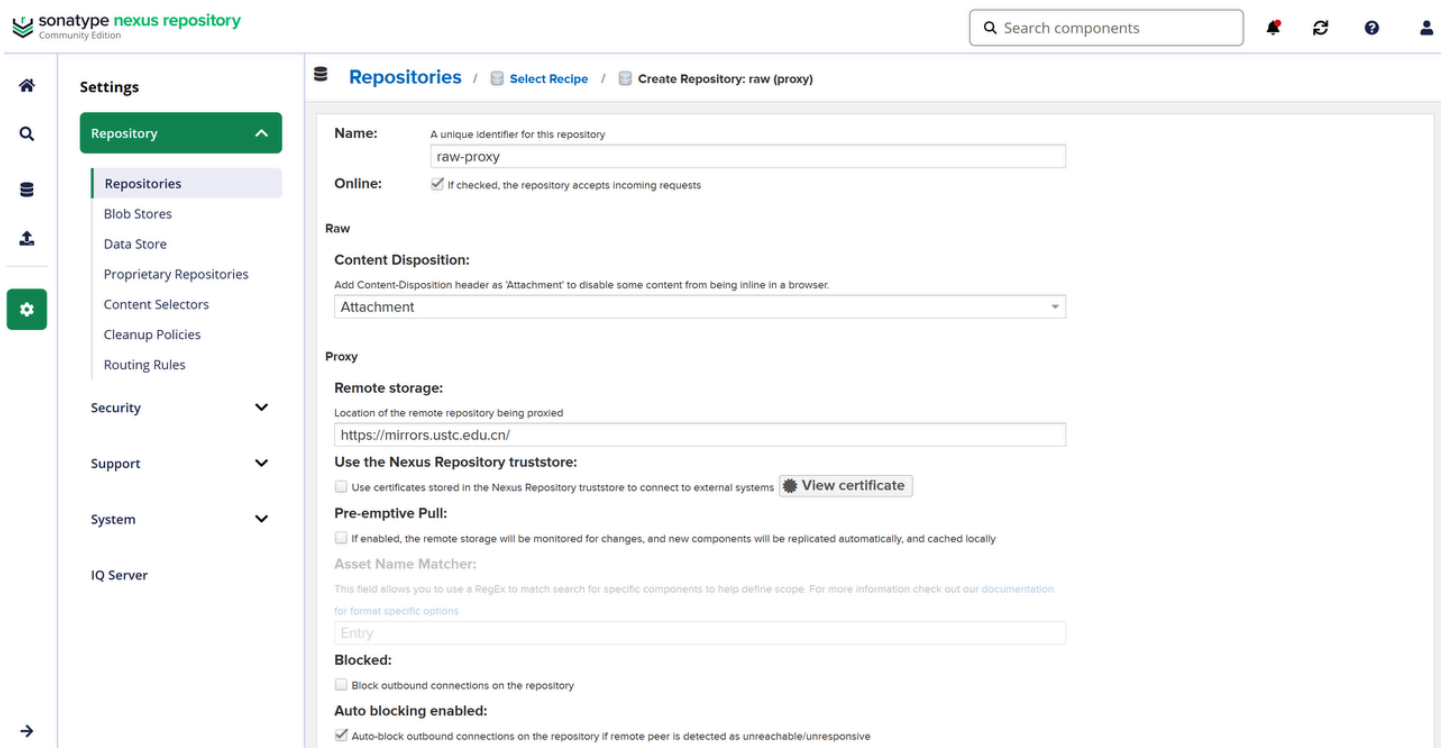
准备工作：

1. 创建存储空间（可选但推荐）：在 `Administration` → `Blob Stores` 中创建。
2. 创建目录结构：在 Nexus Web 界面上， `raw` 仓库的文件以目录结构组织。建议按ISO类型和版本创建，例如 `centos/7/isos/` 、 `ubuntu/22.04/` 。

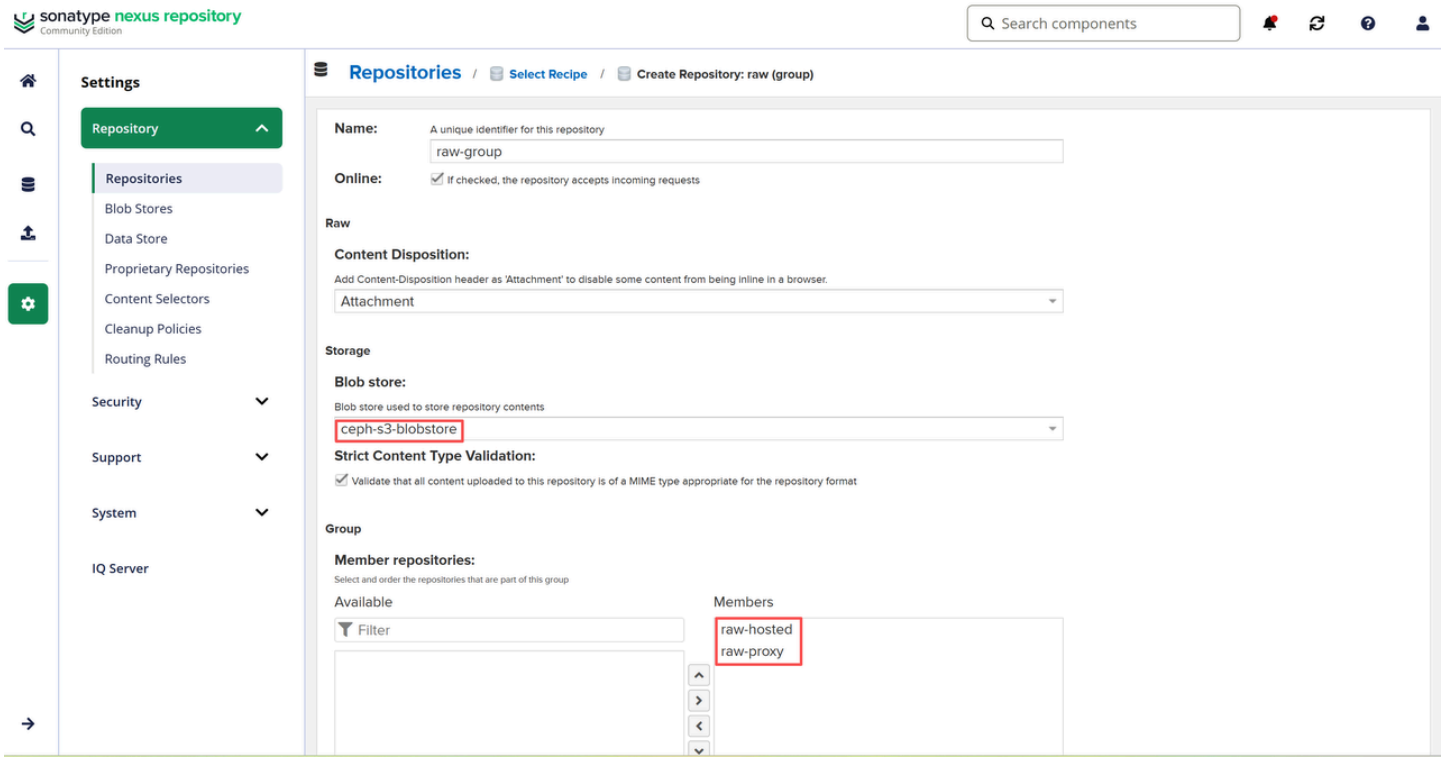
raw-hosted，存储选择ceph-s3-blobstore：



raw-proxy，存储选择ceph-s3-blobstore：



raw-group,存储选择ceph-s3-blobstore：

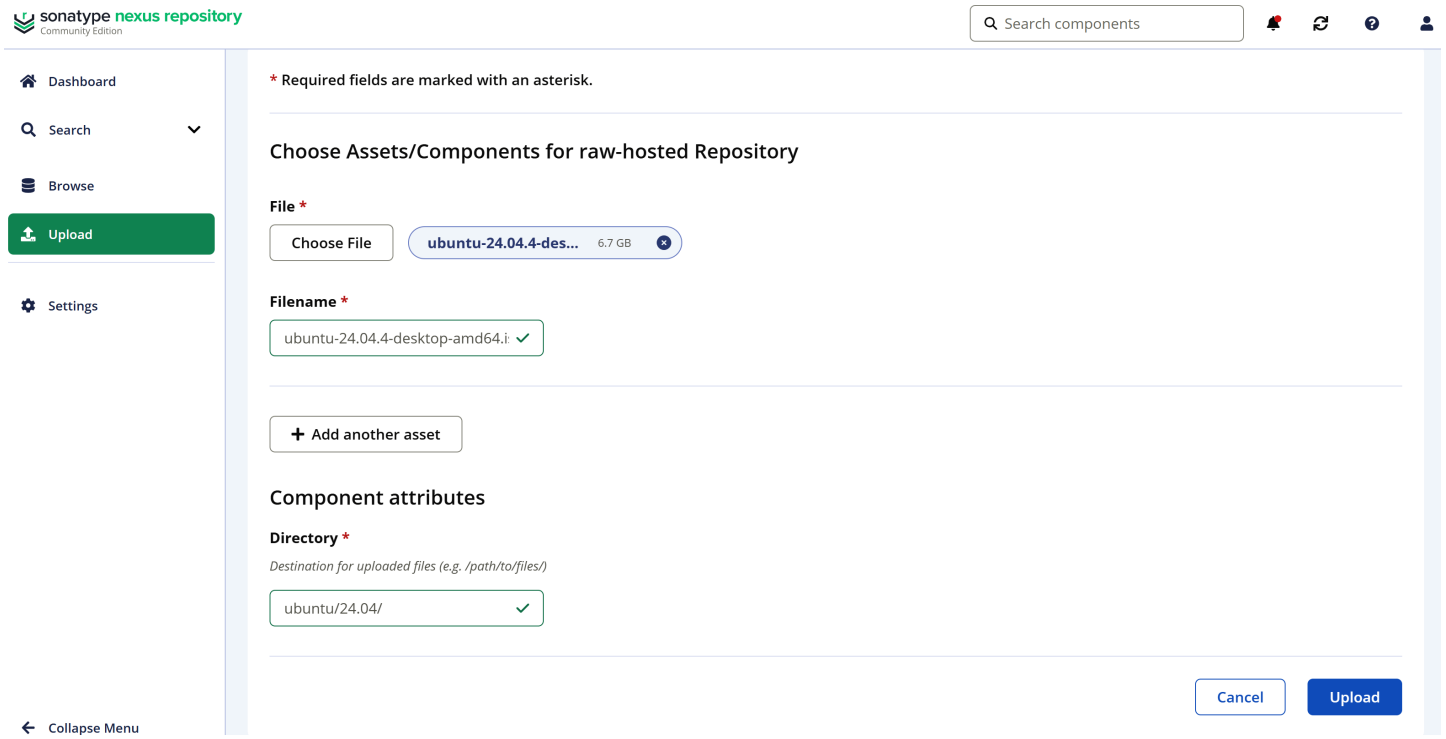


通过 Web UI 上传：

1. 导航至 `Browse` 页面，选择你刚创建的 `raw-hosted` 仓库。
2. 在页面上方点击 `Upload` 按钮。页面上会打开一个配置窗口。
3. ⚠️ 核心步骤：在弹出的上传窗口中：
   - File 字段：选择要上传的文件，这里选择ubuntu-24.04.4-desktop-amd64.iso。

- `Filename` 字段：选择文件以后，该字段会自动填充，保持默认即可。
- Component attributes Directory，上传的文件想要放到哪个目录，方便管理。

4. 点击提交完成上传。



🔐 配置访问权限

1. 在管理界面中，找到 `Security` → `Realms`，确保 `Nexus Authenticating Realm` 已处于激活状态。
2. 接下来，进入 `Security` → `Users`，创建用于访问 `raw` 仓库的专用用户（例如 `raw-user`），并为其设置密码。
   - 建议：为用户设置明确的角色，通过角色来统一管理权限。
3. 在 `Security` → `Roles` 中创建角色（例如 `raw-isos-manager`），并授予其对 `raw-isos` 仓库的 `nx-repository-view-raw-raw-isos-add`、`edit`、`read` 等权限。
4. 最后，为该角色分配创建的用户。

🐚 访问与使用 (上传或下载)

Nexus `raw` 仓库支持通过命令行工具（如 `curl`）或文件管理器，使用HTTP基本认证进行上传和下载。

下载ISO文件：

当您需要获取之前上传的ISO文件时，可以执行：

bash

curl -u raw-user:your-password \ http://localhost:8084/centos/7/isos/CentOS-7-x86_64-DVD-2009.iso \ --output /path/to/save/CentOS-7-x86_64-DVD-2009.iso

- `-u`：用于认证的`用户名:密码`。
- URL为：`http://NEXUS服务器IP:RAW仓库端口/文件在仓库中的完整路径`。

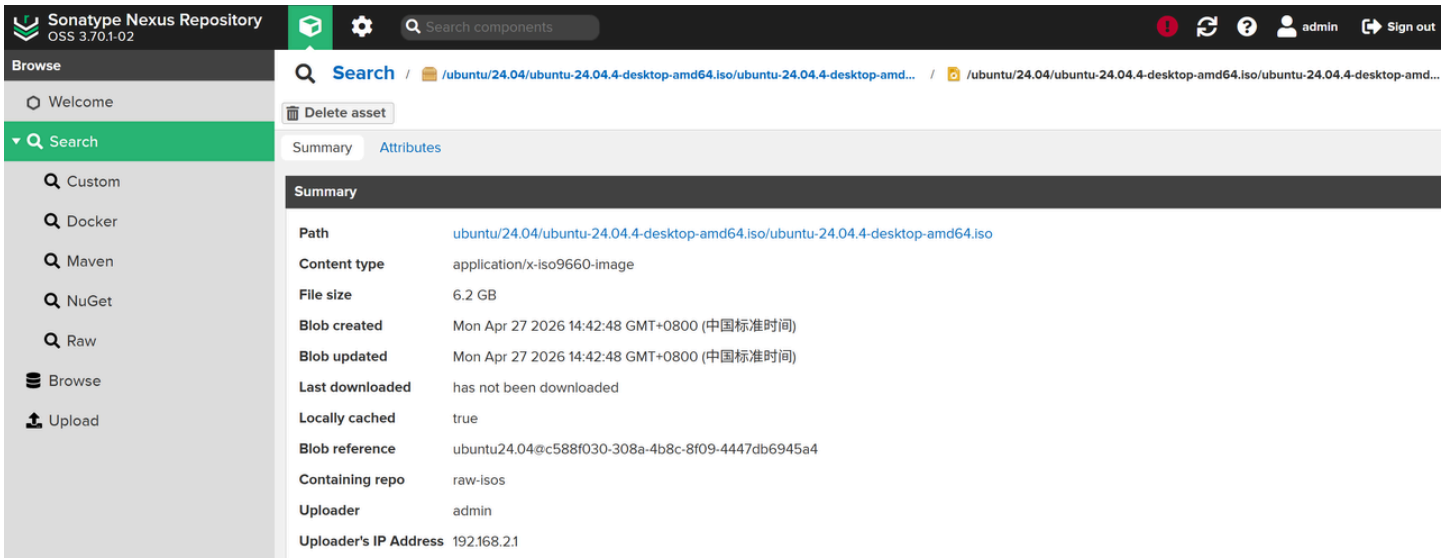
上传ISO文件：

当您有新的ISO文件需要上传时，可以执行：

bash

curl -u raw-user:your-password \ --upload-file LocalCentOS.iso \ http://localhost:8084/centos/8/isos/CentOS-8.iso

- `--upload-file`：指定本地文件路径。
- URL同样指定了文件最终在仓库中的存储位置。



3.3.5 备份与恢复

3.3.5.1 备份

Nexus 数据默认在 `nexus-data` 目录（Docker 映射目录或物理机 `sonatype-work/nexus3`）。

- 直接打包整个 `nexus-data` 目录：`tar -zcvf nexus-data-backup.tar.gz /path/to/nexus-data`。

3.3.5.2 恢复

1. 停止 Nexus 服务。
2. 清空新环境的 `nexus-data` 目录。
3. 解压备份包到 `nexus-data`。
4. 启动 Nexus，验证仓库、用户、配置均恢复。

3.3.6 高可用

3.3.6.1 nginx和nexus高可用

高可用逻辑

组件角色

组件	作用
Keepalived	VRRP 协议管理 VIP，通过健康检查决定 VIP 归属
Nginx	反向代理，监听 80 端口，转发到本机 Nexus:8081
Nexus	制品仓库，监听 8081 端口
VIP	对外统一入口，漂移到健康节点

Nginx 和 Nexus 在 **U1 和 U2 上始终运行**，VIP 决定流量走哪个节点。

正常状态

客户端 → VIP:80 → U1 Nginx(:80) → U1 Nexus(:8081)

U1: 优先级 110，持有 VIP，对外提供服务

U2: 优先级 100，BACKUP，Nginx + Nexus 同样运行但不接收外部流量

健康检查机制

check_service.sh 每 5 秒执行一次，同时检查两项：

1. Nginx 进程存在（nginx -t + kill -0 检查 PID）

2. Nexus HTTP 响应（curl http://127.0.0.1:8081/）

两项均正常 → 返回 0，优先级不变

任意一项故障 → 返回 1

连续 3 次失败（15 秒）→ 优先级 -20

连续 2 次成功（10 秒）→ 优先级恢复

故障切换流程

U1 Nginx 或 Nexus 故障：

check_service 失败 × 3（15s）

→ U1 优先级 110 - 20 = 90

→ U2 优先级 100 > U1 的 90

→ U2 抢占 VIP

→ VIP 漂移到 U2

→ 客户端流量走 U2 Nginx → U2 Nexus

→ 服务恢复（U2 Nexus 一直在运行，立即可用）

U1 恢复后：

check_service 成功 × 2（10s）

→ U1 优先级恢复 110

→ U1 优先级 110 > U2 的 100

→ U1 重新抢占 VIP

→ VIP 漂移回 U1

优先级变化汇总

| 状态 | U1 优先级 | U2 优先级 | VIP |

|---|---|---|---|

| 两节点正常 | 110 | 100 | U1 |

| U1 故障 | 90 | 100 | → U2 |

| U1 恢复 | 110 | 100 | → U1 |

| U2 故障（VIP 在 U2） | 110 | 80 | → U1 |

| U2 恢复（VIP 在 U1） | 110 | 100 | U1 |

切换时间

阶段	时间
故障检测（5s × 3次）	15 秒
VIP 切换	< 1 秒
服务恢复	立即（Nexus 已在运行）
总计	**约 15 秒**

访问链路

客户端

→ http://192.168.2.100:80（VIP）

→ 当前 MASTER 节点 Nginx(:80)

→ 本机 Nexus(:8081)

→ 本机 HAProxy(:5000)

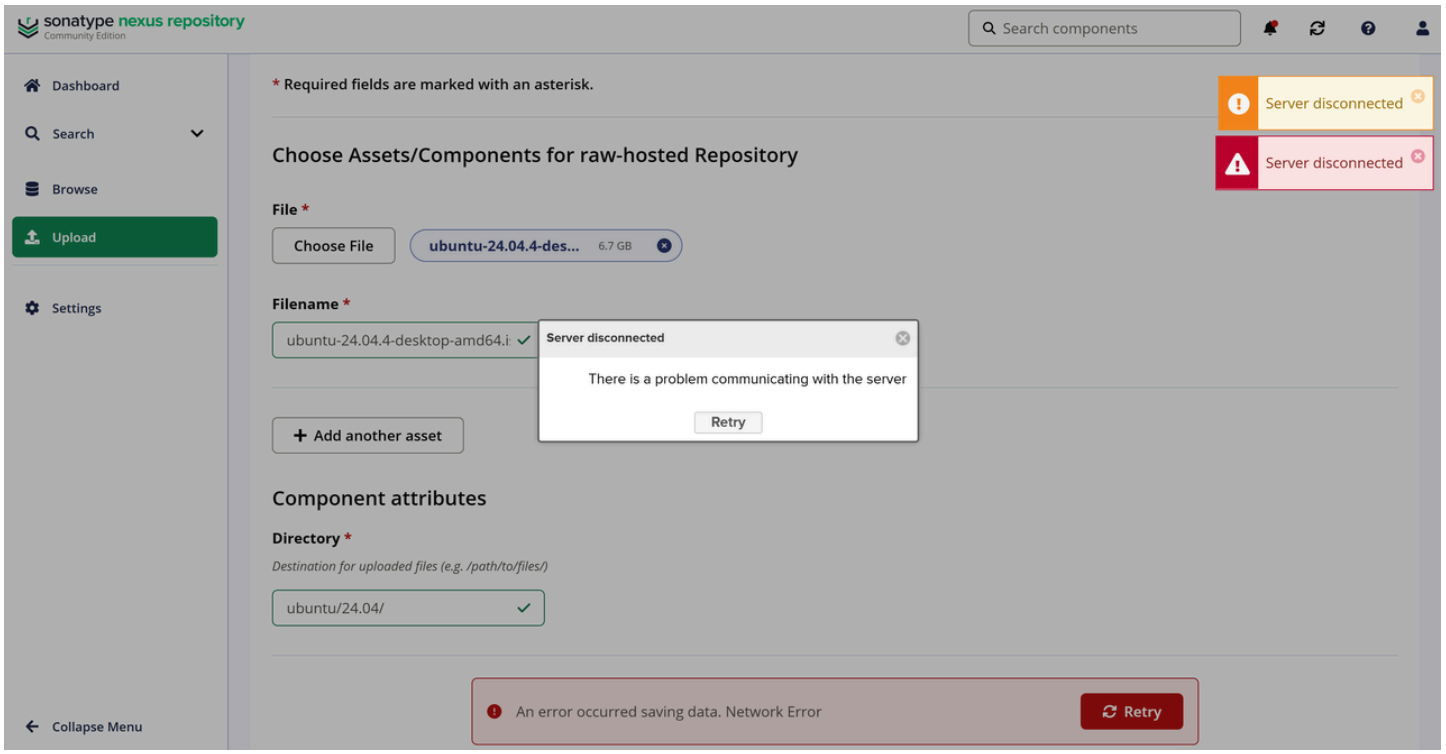
→ PostgreSQL Leader（当前在 U1 或 U2）

→ Nexus S3 Blob Store → http://192.168.2.100:8080（VIP）

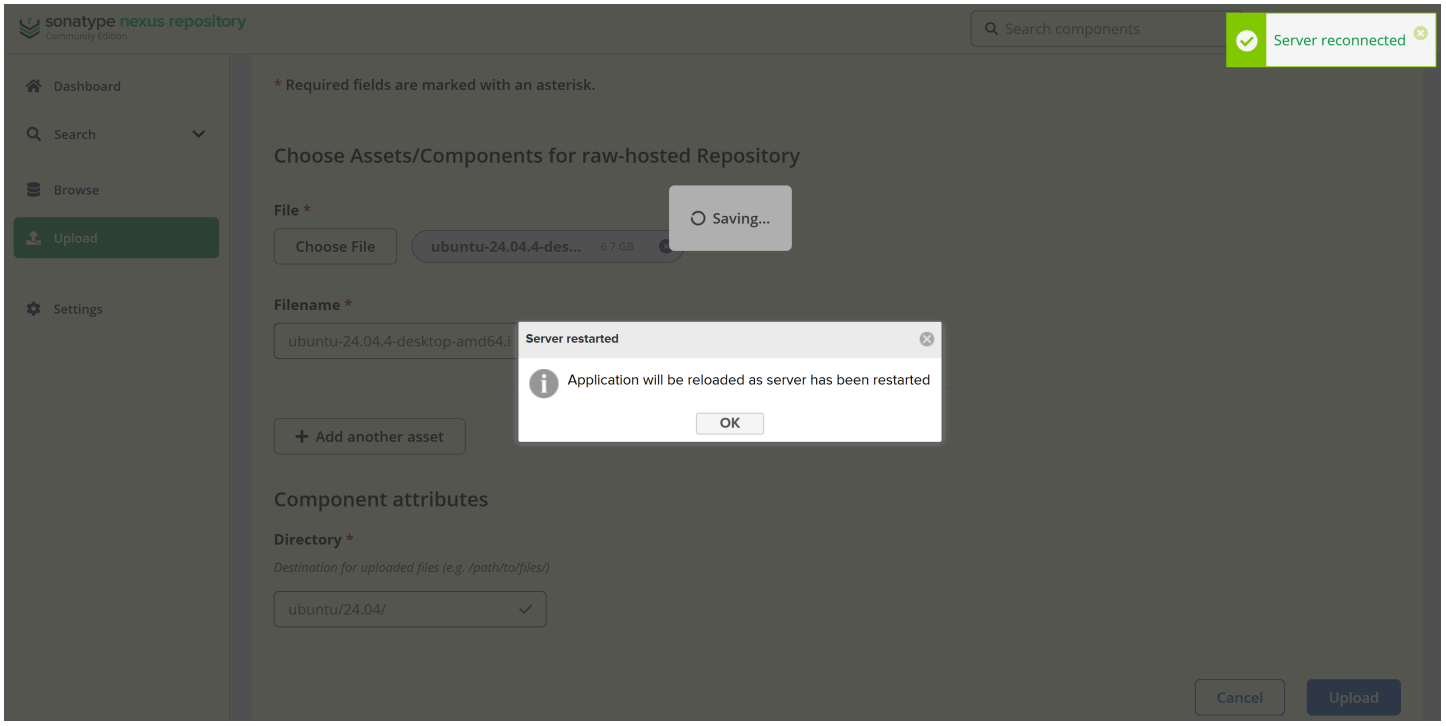
→ HAProxy(:8080) → RGW × 3

3.3.6.1.1 nginx服务高可用

在master 执行systemctl stop nginx，出现如下画面



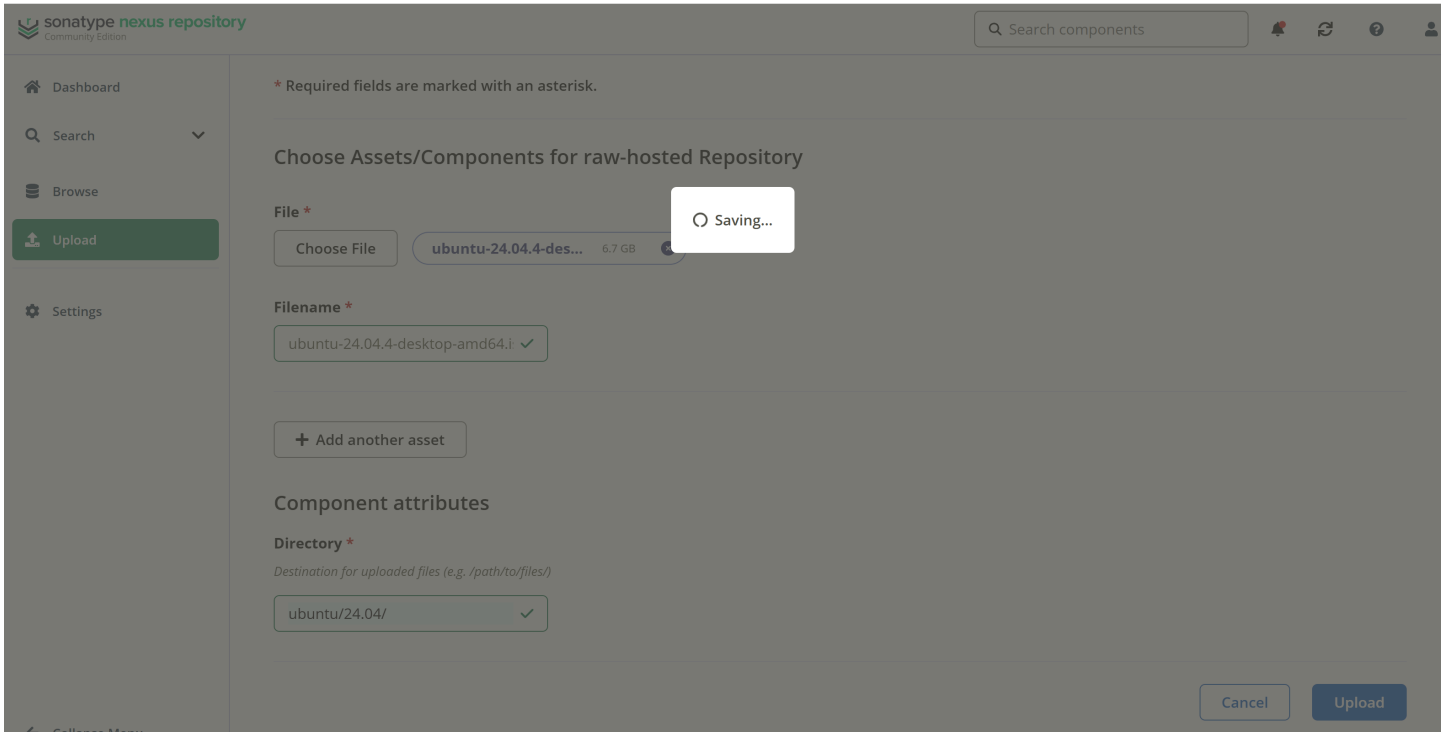
大约20s neuxs服务恢复，上传文件操作恢复传输



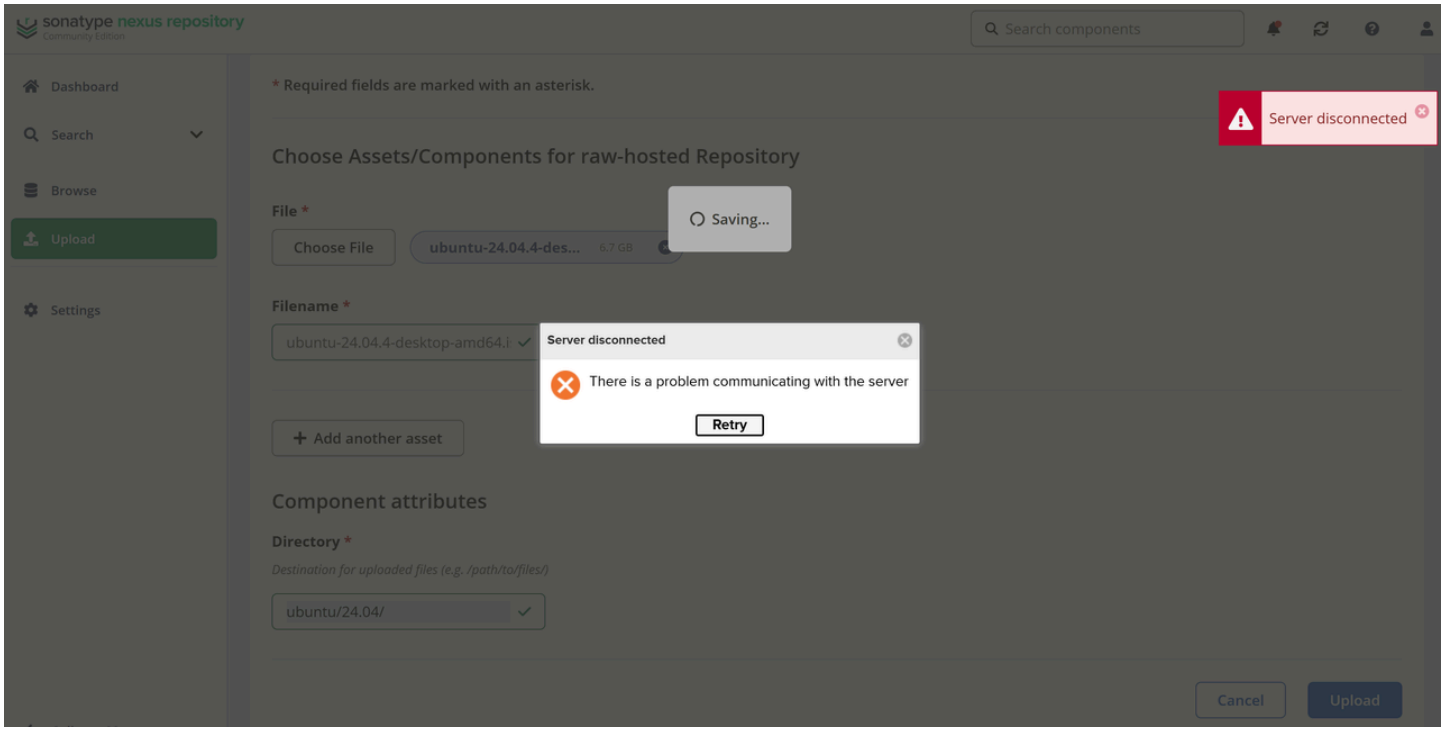
但当点击上图中OK后，neuxs服务重新加载，登录后文件需要重新上传，没有保存

3.3.6.1.2 neuxs服务高可用

所有服务运行正常，上传ubuntu IOS镜像（带业务测试比较耗资源，系统经常出问题）



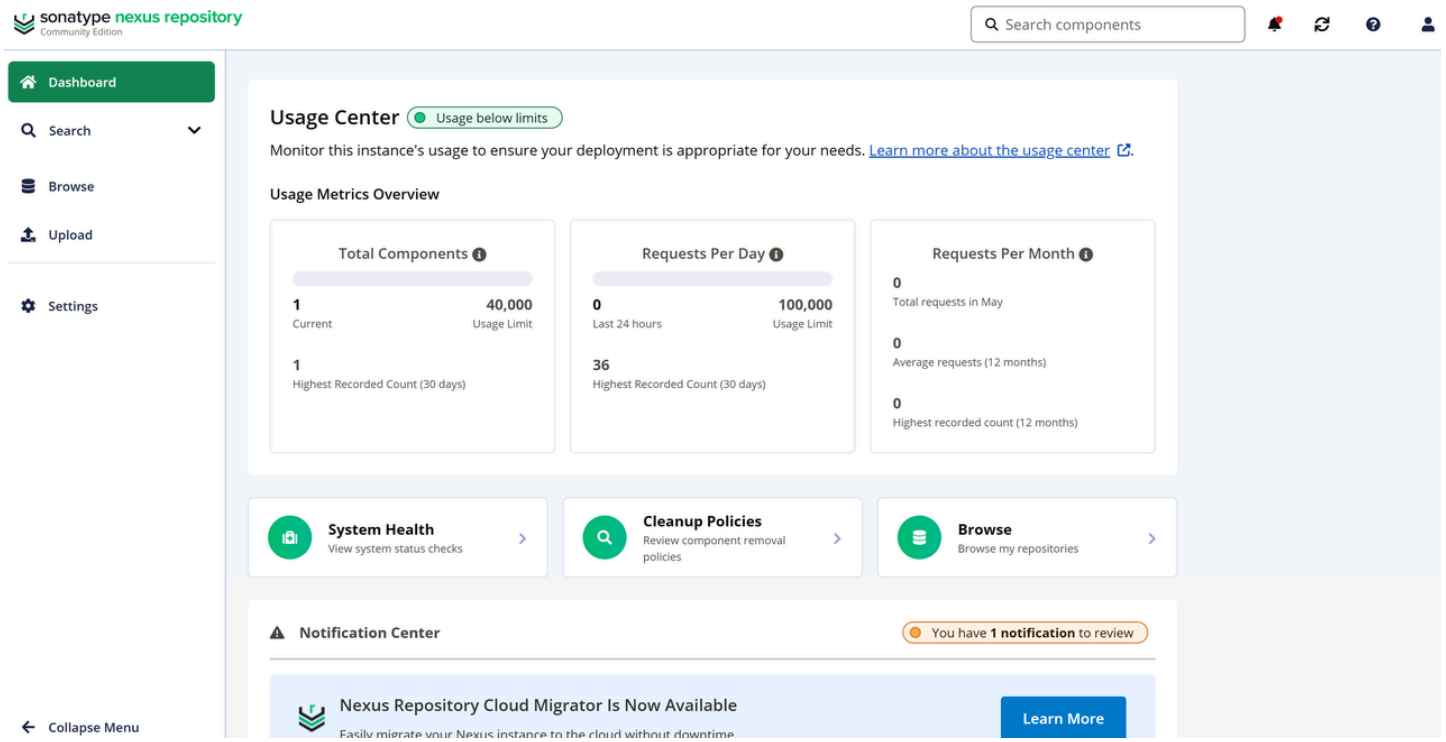
执行systemctl stop nexus.service



切换VIP

```
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq state UP group default qlen 1000
    link/ether 00:0c:29:68:34:2c brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.2.8/24 brd 192.168.2.255 scope global noprefixroute ens34
       valid_lft forever preferred_lft forever
    inet 192.168.2.100/24 scope global secondary ens34:vip
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe68:342c/64 scope link
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group defaul
    link/ether f6:73:fd:80:0a:da brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
root@U2:~#
```

nexus访问正常（大约15s钟恢复正常）



3.3.6.2 postgresql高可用

PostgreSQL 高可用逻辑

组件角色

组件	作用
etcd	分布式协调存储，Patroni 用它做 Leader 选举和集群状态存储

Patroni	管理 PostgreSQL 主从复制和自动故障切换
HAProxy	监听 5000 端口，通过健康检查只把流量路由到当前 Leader
Nexus	连接 127.0.0.1:5000（本机 HAProxy），不直连 PostgreSQL

正常状态

Nexus(U1) → 127.0.0.1:5000 → HAProxy → U1:5432 (Leader)

Nexus(U2) → 127.0.0.1:5000 → HAProxy → U1:5432 (Leader)

U1: Leader（读写）

U2: Replica（流复制，实时同步）

U3: Replica（流复制，实时同步）

两个节点的 Nexus 都通过本机 HAProxy 连接，HAProxy 通过 Patroni REST API（端口 8008）做健康检查：

- GET /master 返回 200 → 是 Leader，接受连接
- GET /master 返回 503 → 是 Replica，拒绝连接

postgresql主从间数据同步

本方案使用 **PostgreSQL 流复制（Streaming Replication）**，数据同步是**实时连续**的，没有固定的时间间隔。

具体来说：

- **流复制**：Standby 节点通过 WAL sender/receiver 持续从 Primary 接收 WAL 日志，延迟通常在毫秒级（取决于网络）。

- **同步模式**：配置为异步复制（synchronous_commit 未显式设置，默认 on 但无 synchronous_standby_names），即 Primary 提交后不等待 Standby 确认，Standby 异步追赶。

Patroni 相关的时间参数（影响的是**故障检测和切换**，不是数据同步间隔）：

| 参数 | 值 | 含义 |

|---|---|

| loop_wait | 10s | Patroni 健康检查循环间隔 |

| ttl | 30s | Leader 租约超时（Primary 失联后多久触发选举） |

| retry_timeout | 10s | 操作重试超时 |

| maximum_lag_on_failover | 1MB | 允许 Standby 落后 Primary 的最大 WAL 量，超过则不参与选举 |

Postgresql实例本身可以通过patroni实现自愈：

某个挂掉的 PostgreSQL 实例本身能否“自愈”。答案是：可以的，并且这正是 Patroni 的核心功能之一。它通过 自动重启 和 自动重建 这两种机制来实现。

恢复方式	适用场景	核心原理
自动重启	PostgreSQL 进程崩溃、OOM 或被误杀。	Patroni 作为守护进程，检测到数据库进程启动它。
自动重建	数据损坏、pg_rewind 失败，或节点因 WAL 日志问题无法追上主库。	Patroni 会清空本地数据目录，然后从主库一次全量物理备份重新搭建备库。

核心工作流程

这一“自愈”过程的核心在于 Patroni 的主动故障处理：

1. 不断“问诊”：Patroni 守护进程会定期通过 `pg_isready` 检查本地 PostgreSQL 的健康状况。
2. 尝试“急救”：一旦发现进程消失，Patroni 会立即尝试重新启动（Restart）数据库进程。
3. 执行“修复”：
    - 自动重启成功：数据库恢复正常。这通常只需几秒钟，业务基本无感知。
    - 自动重启失败：如果多次重启都无法恢复，会依据角色进行不同处理。
        - 对于备库：Patroni 会判定该节点彻底损坏，转而进入自动重建流程。它会清空本地数据目录，然后使用 `pg_basebackup` 从主库重新复制一份完整的数据副本，待同步完成后，该节点将作为一个全新的备库自动加入集群。
        - 对于主库：Patroni 会放弃尝试恢复，将 Leader Key 在 DCS 中释放，从而触发集群的故障转移。此时，一个健康的备库会被提升为新的主库。而原主库在自身恢复后（可能需要人工重启主机），其上的 Patroni 也会检测到“主库”身份已失，从而自动将自己降级为备库并重建，重新加入集群。

⚠️ **何时无法自动恢复？**

尽管设计初衷是“自愈”，但在极少数情况下，自动恢复也会失败，需要人工介入。

- 情况一：`pg_rewind` 失败且未配置兜底策略
  在故障转移后，原主库尝试加入集群时，Patroni 会优先使用 `pg_rewind` 来快速与新主库同步。如果 `pg_rewind` 失败（例如，因 WAL 日志缺失或数据目录严重损坏），且未配置自动重建，节点就会卡住无法恢复。
- 情况二：全集群彻底宕机
  如果集群的所有节点（包括 Patroni 和 etcd）都完全崩溃，当节点重新启动时，它们可能无法就“谁是主库”达成一致，从而陷入无主状态。

📝 **关键配置参考**

要确保自动恢复能有效工作，你可以在 Patroni 的 YAML 文件中进行如下配置：

yaml

*# 自动重启相关配置*loop_wait: 10 *# 健康检查的循环周期，默认10秒 [reference:5]*primary_start_timeout: 10 *# 主库挂掉后，尝试重启的超时时间，超时则触发故障转移[reference:6]*retry_timeout: 10 *# 数据库操作的重试超时时间[reference:7]# 自动重建与pg_rewind相关配置（关键！）*use_pg_rewind: true *# 开启快速回切 [reference:8]*remove_data_directory_on_rewind_failure: true *# 若pg_rewind失败，自动清理数据目录，触发重建[reference:9]*wal_log_hints: on *# pg_rewind的依赖项，建议开启 [reference:10]# 全集群彻底宕机恢复配置*no_leader: true *# 在集群全毁后，允许通过备份创建一个新的主节点[reference:11]*

验证

建议根据上面的配置，通过模拟故障来验证你当前的集群是否具备足够的“自愈”能力。

1. 本地验证：通过 `kill -9` 杀掉 PostgreSQL 进程，观察其是否能被 Patroni 自动重启，以及日志输出。
2. 模拟破坏性故障：删除 `data` 目录下的某个关键文件，模拟数据损坏场景，观察 Patroni 是否能够触发自动重建。
3. 检查 `pg_hba.conf`：确保所有节点的 `pg_hba.conf` 配置正确，允许复制用户从其他节点发起 `pg_basebackup` 和流复制，这是自动重建成功的基础。

故障切换流程

U1 PostgreSQL 故障：

1. Patroni 检测到 Leader 心跳丢失（ttl=30s）

2. etcd 上的 Leader key 过期
3. U2/U3 发起选举，复制进度最新的节点当选新 Leader
4. 新 Leader 执行 pg_promote()，升级为可读写
5. HAProxy 健康检查：
    - U1:8008/master → 503（不再是 Leader）
    - U2:8008/master → 200（新 Leader）
6. HAProxy 自动将流量切到 U2:5432
7. Nexus 连接短暂中断后自动重连，业务恢复

切换时间：

- Leader 失效检测：ttl = **30 秒**
- HAProxy 检测新 Leader：inter 3s × fall 3 = **9 秒**
- 总计约 **30~40 秒**

U1 恢复后

1. Patroni 启动，发现集群已有 Leader（U2）
2. U1 自动以 Replica 身份加入，从新 Leader 同步数据
3. HAProxy 健康检查 U1:8008/master → 503，不路由流量给 U1
4. 集群恢复三节点状态

U1 恢复后**不会自动抢回 Leader**，需要手动执行：

patronictl -c /etc/patroni/patroni.yml switchover --master U2 --candidate U1

与 Keepalived VIP 的关系

两套 HA 完全独立：

| 层 | 故障切换 | 切换时间 |

|----|------|

| 应用层（Keepalived） | Nginx/Nexus 故障 → VIP 漂移 | ~15 秒 |

| 数据库层（Patroni） | PostgreSQL 故障 → Leader 切换 | ~30-40 秒 |

Nexus 连的是 127.0.0.1:5000（本机 HAProxy），无论 VIP 在哪个节点，数据库连接都走本机 HAProxy，两套切换互不干扰。

测试注意：

方式一：停 Patroni 服务（推荐测试）

systemctl stop patroni

方式二：通过 patronictl 触发切换（生产推荐）

patronictl -c /etc/patroni/patroni.yml switchover --leader U1 --candidate U2 --force

测试

执行命令

patronictl -c /etc/patroni/patroni.yml switchover --leader U1 --candidate U2 --force

```
root@U1:~# patronictl -c /etc/patroni/patroni.yml switchover --leader U1 --candidate U2 --force
2026-05-07 18:08:52,557 - ERROR - Failed to get list of machines from http://192.168.2.8:2379/v3
on (Caused by NewConnectionError("HTTPConnection(host=\'192.168.2.8\', port=2379): Failed to est
Current cluster topology
+ Cluster: nexus-pg-cluster (7631991574234965931) +-------------+-----+------------+-----+
| Member | Host        | Role    | State     | TL | Receive LSN | Lag | Replay LSN | Lag |
+--------+-------------+---------+-----------+----+-------------+-----+------------+-----+
| U1     | 192.168.2.2 | Leader  | running   | 93 |             |     |            |     |
| U2     | 192.168.2.8 | Replica | streaming | 93 |  0/1010D3D0 |   0 | 0/1010D3D0 |   0 |
| U3     | 192.168.2.9 | Replica | streaming | 93 |  0/1010D3D0 |   0 | 0/1010D3D0 |   0 |
+--------+-------------+---------+-----------+----+-------------+-----+------------+-----+
2026-05-07 18:08:54.75990 Successfully switched over to "U2"
+ Cluster: nexus-pg-cluster (7631991574234965931) ------------+-----+------------+-----+
| Member | Host        | Role    | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+--------+-------------+---------+---------+----+-------------+-----+------------+-----+
| U1     | 192.168.2.2 | Replica | stopped |    |     unknown |     |    unknown |     |
| U2     | 192.168.2.8 | Leader  | running | 93 |             |     |            |     |
| U3     | 192.168.2.9 | Replica | running | 93 |  0/1010D530 |   0 | 0/1010D530 |   0 |
+--------+-------------+---------+---------+----+-------------+-----+------------+-----+
```

刷新nexus页面一直可访问，无中断。

3.3.6.3 ceph高可用

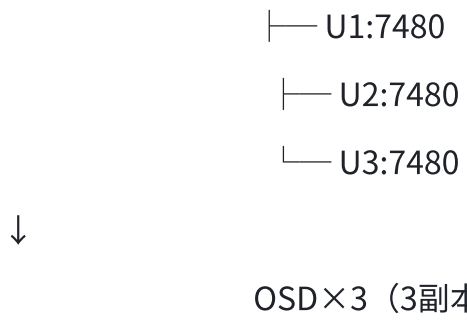
Ceph RGW 高可用逻辑

组件角色

组件	作用
RGW	S3 网关，运行在 U1/U2/U3 各一个，端口 7480
HAProxy	监听 8080，TCP 模式轮询转发到三个 RGW
Ceph OSD	数据存储，每份数据写 3 副本
VIP	Nexus 通过 http://192.168.2.100:8080 访问，随 Keepalived 漂移

正常状态

Nexus → VIP:8080 → HAProxy(U1或U2) → RGW×3(轮询)



HAProxy 使用 **TCP 模式**（不能用 HTTP 模式），原因：

- S3v4 签名把 Host header 绑定到请求中
- HTTP 模式会重写 Host header → 签名验证失败（SignatureDoesNotMatch）
- TCP 模式透传原始请求，Host header 保持不变

  RGW 配置了 rgw_dns_name = 192.168.2.100，使 RGW 识别 VIP:8080 为自身端点，而不是把它当作 bucket 名（否则返回 NoSuchBucket）。

单个 RGW 节点故障

U1 RGW 故障（U1:7480 不响应）

→ HAProxy tcp-check 检测失败

→ 自动摘除 U1:7480

→ 流量只走 U2:7480 和 U3:7480

→ Nexus S3 访问不中断

检测时间：inter 5s × fall 3 = **15 秒**（当前配置）

三个 RGW 任意挂两个，只剩一个也能正常服务（数据在 OSD 上，RGW 只是网关）。

OSD 节点故障（数据层）

一个 OSD 故障（如 U1 宕机）

→ 该 OSD 上的数据副本不可用

→ Ceph 自动从其他两个 OSD 的副本提供读写

→ 数据访问不中断（min_size=2，只需 2 副本即可读写）

→ 后台自动补充副本到其他健康 OSD

所有 RGW 存储池副本数均为 **size=3，min_size=2**，即：

- 3 个 OSD 全健康：正常读写
- 1 个 OSD 故障：仍可读写，后台修复
- 2 个 OSD 同时故障：集群进入只读保护，拒绝写入

VIP 切换对 RGW 的影响

VIP 从 U1 切到 U2

→ HAProxy 从 U1 的实例切到 U2 的实例

→ U2 的 HAProxy 同样连接 U1/U2/U3 三个 RGW

→ RGW 服务不受影响，数据完全一致（共享同一套 OSD）

VIP 切换只影响 HAProxy 入口，RGW 和 OSD 完全无感知。

测试：

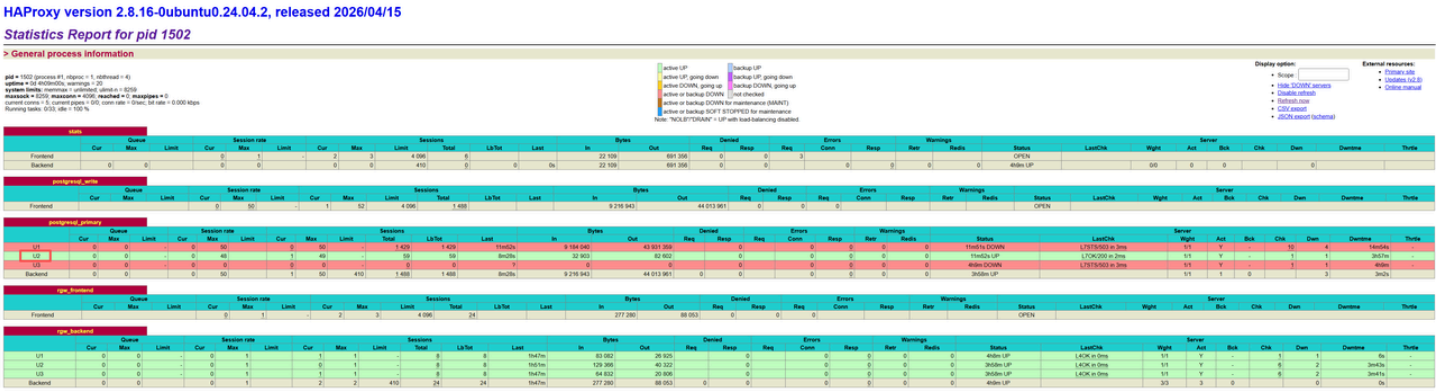
1 HAProxy 健康状态确认

查看 HAProxy stats，确认三个 RGW 都是 UP

curl -s http://192.168.2.100:7000/haproxy | grep -i rgw

或直接访问 stats 页面

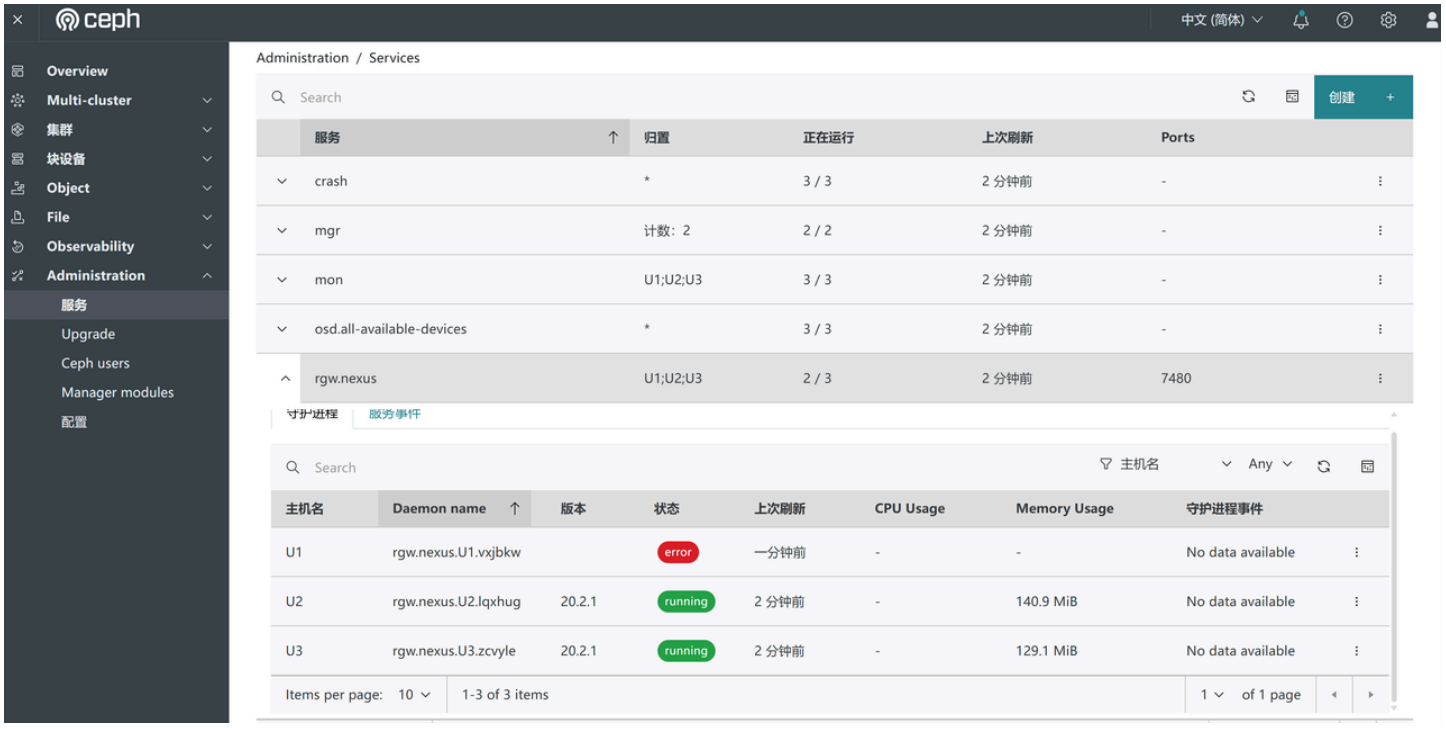
http://192.168.2.100:7000/haproxy



2 停掉单个 RGW，验证请求自动路由到其他节点

停掉 U1 的 RGW

systemctl stop 'ceph-*@rgw*' 2>/dev/null || systemctl stop $(systemctl list-units 'ceph*rgw*' --no-pager -q | awk '{print \$1}' | head -1)



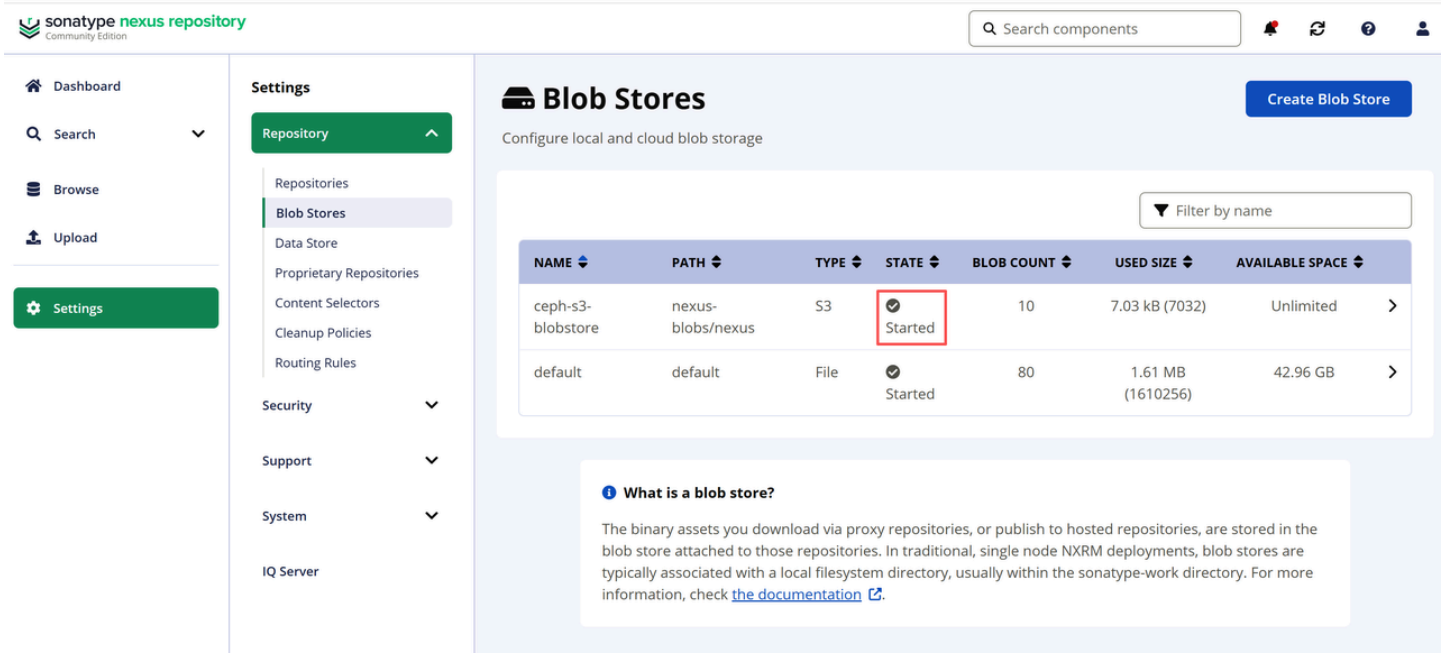
停掉后验证 S3 仍然可用：

用 aws cli 测试（从任意节点）未找到该工具

aws s3 ls s3://nexus-blobs \

--endpoint-url http://192.168.2.100:8080 \

--no-verify-ssl 2>&1 | head -5 查看nexus访问仍正常

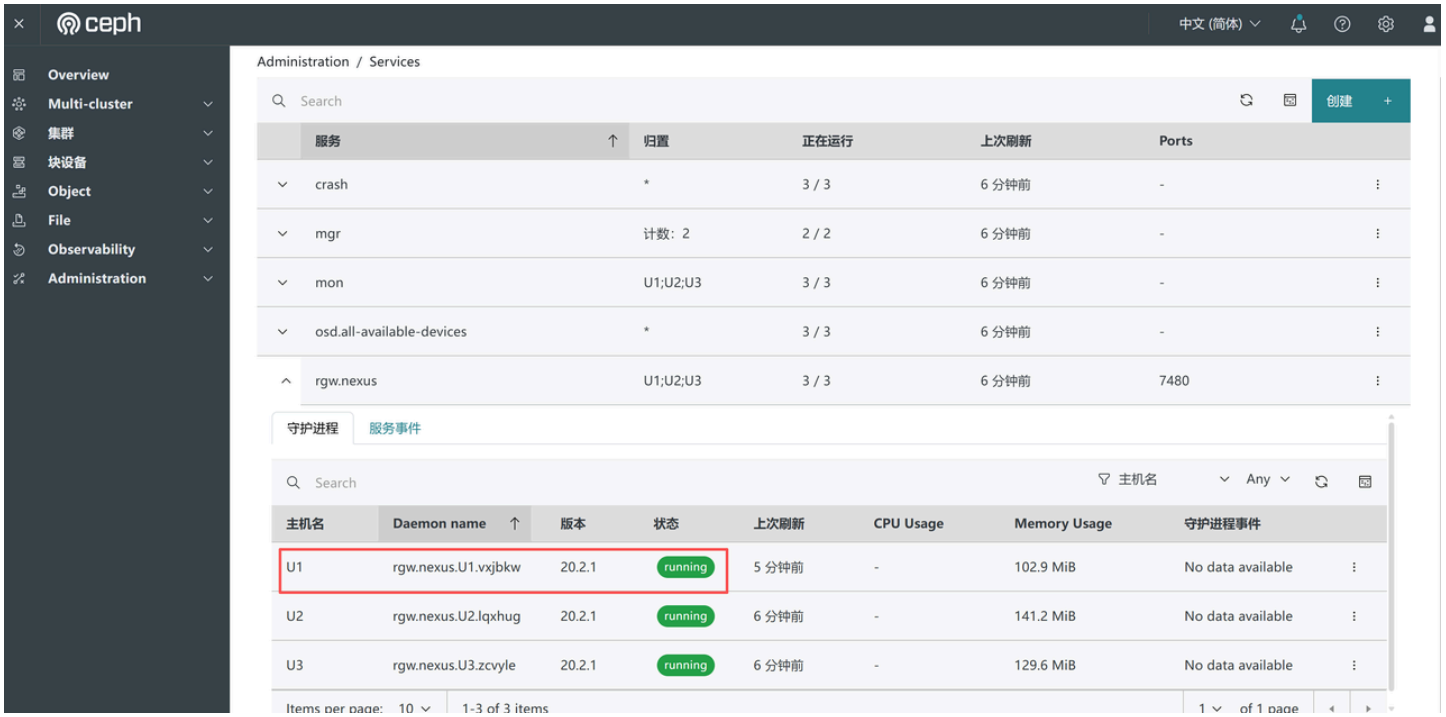


恢复：

systemctl start $(systemctl list-units 'ceph*rgw*' --no-pager -q | awk '{print $2}' | head -1)

查看恢复

ps -ef | grep radosgw



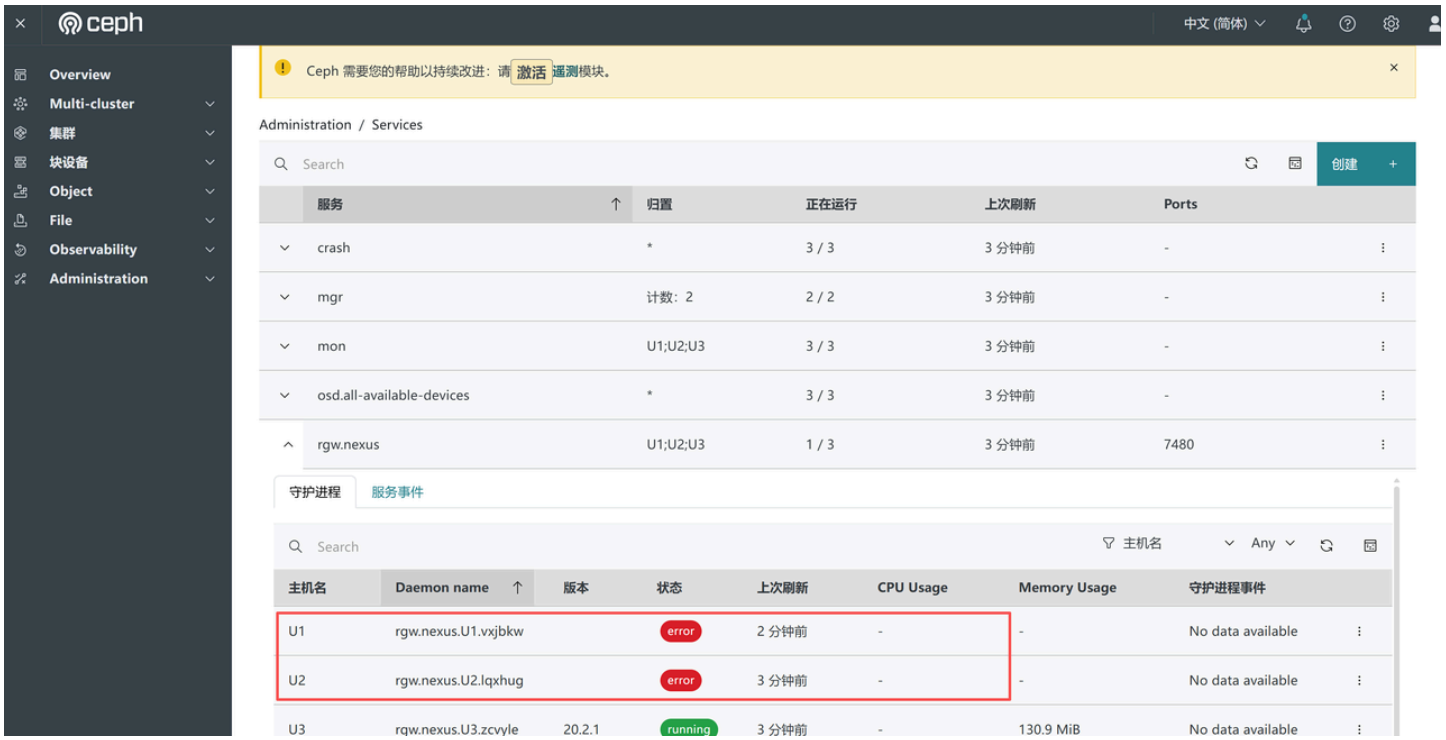
3 停掉两个 RGW，验证单节点仍可服务

同时停 U1 和 U2 的 RGW，只留 U3,分别在U1、U2执行

systemctl stop 'ceph-*@rgw*' 2>/dev/null || systemctl stop $(systemctl list-units 'ceph*rgw*' --no-pager -q | awk '{print \$1}' | head -1)

```
root@U1:~# ps -ef | grep radosgw
root       70766   13323  0 18:52 pts/0    00:00:00 grep --color=auto radosgw
root@U1:~#
```

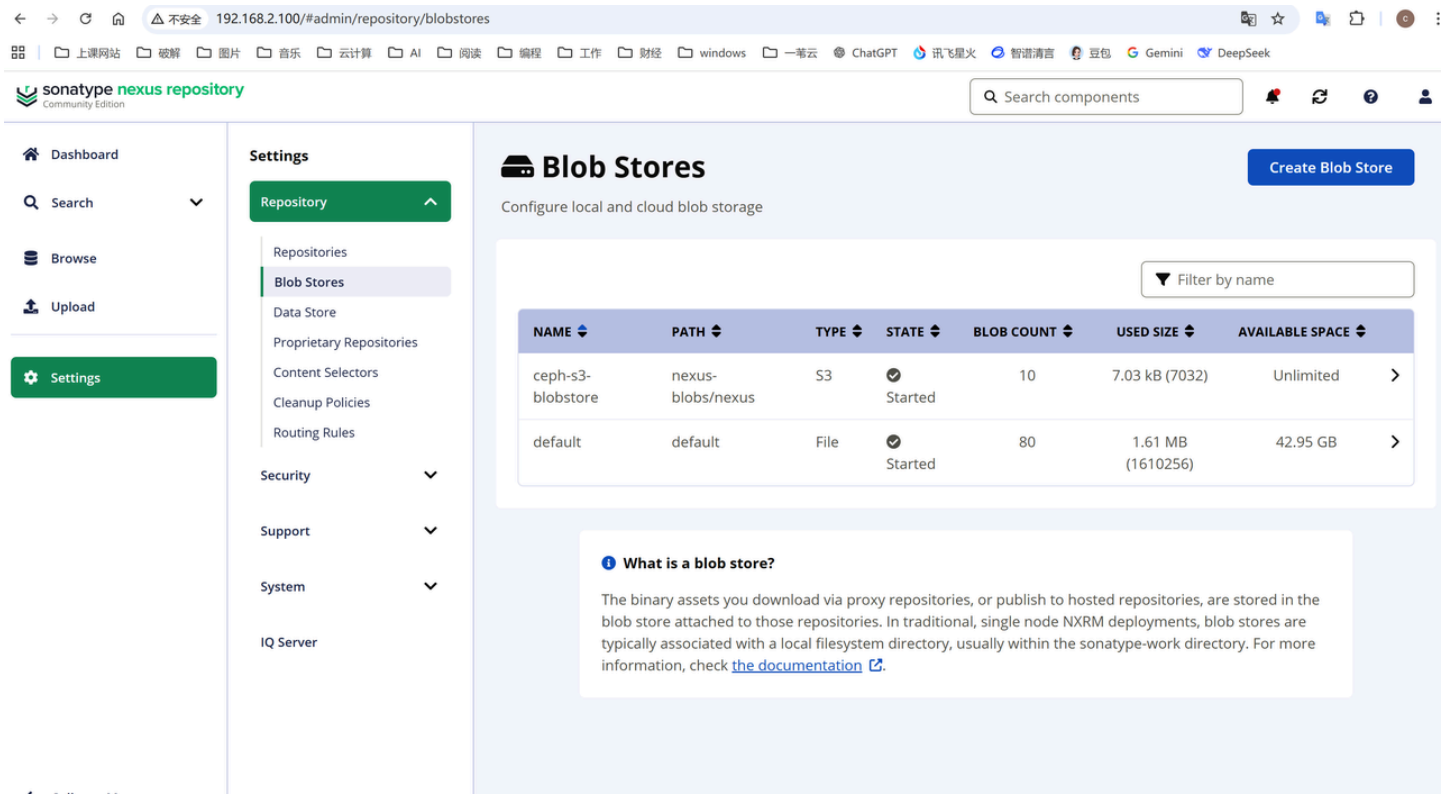
```
root@U2:~# ps -ef | grep radosgw
root      113357    6830  0 18:52 pts/0    00:00:00 grep --color=auto radosgw
root@U2:~#
```



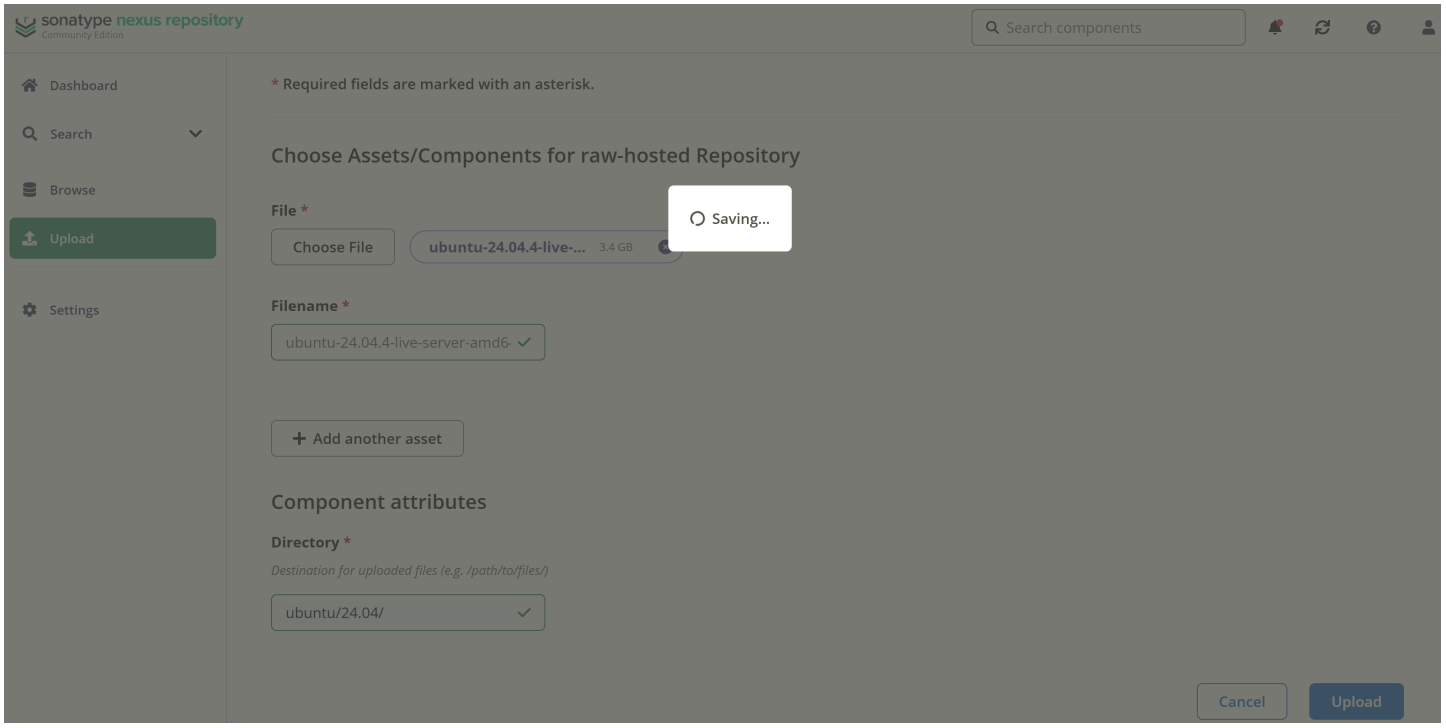
验证 S3 仍通（走 U3）

aws s3 ls s3://nexus-blobs --endpoint-url http://192.168.2.100:8080

服务访问正常



文件正常上传中



4. 组件和代码维护

4.1 nexus源码编译机制

分析对象

- 源码目录: d:\code\nexus-public-release-3.91.1-04
- 版本标识: 3.91.1-04
- 分析目标: 解释该版本源码如何组织、Maven 如何驱动整仓编译、前端资源如何参与构建、最终运行包如何装配，以及当前这个“发布源码包”与完整 Git 仓库在编译上的差异。

1. 总体介绍

Nexus 3.91.1-04 的编译机制本质上是“单仓多模块 Maven 聚合构建 + 前端 Yarn 工作区构建 + 部分模块 AspectJ 编织 + 最终 Spring Boot uber-jar 与 assembly 分发包装配”。

它不是单纯的 Java Maven 项目，也不是单纯的前后端分离项目，而是以下几层叠加：

- 根 POM 负责版本、依赖、插件管理、构建规则和 profile。
- public/common/components 下的大量模块先各自编译成 jar。
- 部分 UI 模块在 Maven 生命周期中调用 frontend-maven-plugin 执行 Yarn 脚本，生成静态资源或前端产物。
- 传统 ExtJS/Rapture 资源由自定义 extjs-maven-plugin 和 yuicompressor-maven-plugin 参与聚合、压缩。
- selfhosted/assemblies/nexus-repository-core 最后把前面生成的所有组件和插件装进一个可执行 Spring Boot jar，再把 jar 与 overlay 目录一起装配成可运行发行包。

2. 源码树的构建分层

   a. **根层**

根 pom.xml 只有一个直接模块:

- public

根 README 给出的官方命令是:

- corepack enable
- corepack yarn install
- ./mvnw clean install -Ppublic

说明官方默认编译流程分两步:

- 先初始化前端 Yarn workspace
- 再执行 Maven 聚合构建

b. **public 层**

public/pom.xml 下分成两大块:

- common
- selfhosted

含义是:

- common: 公共基础组件、格式支持、通用插件、基础装配、构建支持、第三方补丁模块
- selfhosted: 自托管发行形态专属组件和最终发行装配

c. common 层

public/common/pom.xml 下主要有:

- assemblies
- buildsupport
- components
- maven-plugins

职责分别是:

- assemblies: 提供基础 overlay 包
- buildsupport: 统一 BOM/依赖管理/测试支持等
- components: 绝大多数业务组件、格式实现、UI 插件
- maven-plugins: 自定义构建插件，例如 extjs-maven-plugin

d. selfhosted 层

public/selfhosted/pom.xml 下主要有:

- assemblies
- components

其中:

- components 提供自托管特有能力，如 nexus-self-hosted、nexus-repository-core-edition、selfhosted API
- assemblies/nexus-repository-core 负责最终可运行产物

3. 根 POM 如何控制整仓编译

    a. 聚合入口

根 pom.xml:

- packaging = pom
- modules 只有 public

所以根 POM 不产出业务 jar，它只是总调度器。

    b. 基础环境约束

根 POM 的 enforcer 明确要求:

- Maven >= 3.9.6
- Java >= 21

同时还启用了:

- banDuplicatePomDependencyVersions
- bannedDependencies 规则

这说明该版本构建非常依赖新版本 Maven 与 Java 21，低版本环境通常会直接在 validate/early phase 失败，而不是编译中途才报错。

c. Java 编译参数

根 POM 中定义:

- maven.compiler.release = 21
- maven.compiler.source = 21
- maven.compiler.target = 21

并在 maven-compiler-plugin 中启用:

- parameters = true

这意味着:

- 所有模块默认按 Java 21 编译
- 编译后保留方法参数名，方便 Spring、Jackson、反射和 AOP 场景

d. 测试执行策略

根 POM 的 surefire 配置比较重，关键点有:

- 单 fork 进程
- useSystemClassLoader = false
- 追加多组 --add-opens / --add-exports
- 注入 Mockito javaagent
- MaxDirectMemorySize=2G
- 默认排除 Abstract*、内部类、*Tests
- 通过 groups / excludedGroups 控制测试分类

这反映出项目测试并不只是普通单元测试，而是包含较多反射、动态代理、旧框架兼容和数据库分类测试。

默认排除组里包括:

- Perf
- PostgresTestGroup

因此默认构建更偏向 H2 / 常规单测路径。

e. AspectJ 编织是默认构建的一部分

根 POM 的 pluginManagement 定义了 aspectj-maven-plugin 1.14.1，关键配置:

- showWeaveInfo = true
- verbose = true
- source/target = 21
- aspectLibraries 默认包含:
  - nexus-transaction
  - nexus-common
  - nexus-security
  - nexus-validation

common/components/pom.xml 直接启用了该插件，因此 common 体系下很多组件在编译阶段会执行 AspectJ compile，而不是纯 javac。

这说明 Nexus 的事务、安全、校验等横切逻辑部分通过 AspectJ 注入，而不是完全靠 Spring AOP 代理。

f. Spring Boot 重打包

根 POM 统一管理 spring-boot-maven-plugin，selfhosted 最终发行模块在 package 阶段执行 repackage，生成可执行 uber-jar。

4. 前端构建如何嵌入 Maven 生命周期

a. 根层的前端统一配置

根 POM 通过 frontend-maven-plugin 统一定义:

- node.version = v22.19.0
- yarn.version = v1.22.22
- generate-resources 阶段:
  - install-node-and-yarn
  - yarn install
- compile 阶段:
  - yarn run build

但这里有一个关键注释:

- frontend-maven-plugin 下载的是 Yarn 1.22.22
- 实际运行时使用的是 .yarnrc.yml 指向的 Yarn 4.9.1

也就是说，官方完整仓库的真实工作模式是:

- Maven 负责准备 Node/Yarn 启动器
- 实际 workspace 安装与脚本执行依赖 Yarn 4

b. 根 package.json 是 Yarn workspace 根

根 package.json 定义 workspaces:

- public/common/components/nexus-ui-plugin
- public/common/components/nexus-rapture
- public/common/components/nexus-coreui-plugin
- private/cloud/components/nexus-cloudui-plugin

对于 public 版真正相关的是前三个模块:

- nexus-ui-plugin
- nexus-rapture
- nexus-coreui-plugin

这表明前端不是每个模块自带独立 node_modules 的传统方式，而是 Yarn workspace 统一管理依赖。

c. 前端模块的职责分工

c.1 nexus-ui-plugin

这是 React 共享 UI 能力模块，package.json 里可见:

- compile-js 用 Babel 把 src/frontend/src 编译到 target/dist
- copy-scss / copy-package-json 复制资源到 target/dist
- test/test-no-lint 执行 jest

它更像“共享 React 组件库/前端基础库”。

c.2 nexus-coreui-plugin

这是主控制台前端模块之一，package.json 显示:

- build-dev = rspack --config rspack.dev.js
- build-prod = rspack --config rspack.prod.js
- build-all = format + build-dev + build-prod
- test-coverage = jest

它明显是现代 React/Rspack 主界面模块。

同时这个模块还启用了 extjs-maven-plugin 和 yuicompressor-maven-plugin，说明它不是纯 React 项目，而是“老的 ExtJS/Rapture 资源 + 新的 React 构建”混合态。

c.3 nexus-rapture

同样使用 Rspack 构建:

- build-dev
- build-prod

并通过 extjs-maven-plugin 聚合传统 rapture 静态资源，同时为 loading-debug.css 生成 loading-prod.css。

Rapture 可以理解为 Nexus 旧 UI 体系/壳层资源核心。

d. Maven 中前端测试与打包的嵌入点

各前端模块的 pom.xml 普遍具有两类逻辑:

第一类: frontend-maven-plugin

- test 阶段执行 yarn test 或 test-no-lint
- 依赖根工作区提前初始化

第二类: maven-antrun-plugin

- package 阶段把 src/frontend、配置文件、rspack 配置等打进 sources.jar

这说明 Maven 不仅把前端产物当静态文件消费，还希望源码包中保留前端源代码，方便源码发布和二次构建。

5. 为什么 public profile 很关键

a. 根 profile public 的作用

根 POM 中 public profile 激活后会把:

- npm.install 从 install --immutable 改成 install

这个变化看似小，实际意义很大:

- install --immutable 更适合完整 workspace、锁文件齐全的 CI/正式仓库
- install 更宽松，允许公开发布源码构建时少一些严格锁定限制

b. 前端子模块 public profile 的作用

nexus-ui-plugin、nexus-coreui-plugin、nexus-rapture 在 public profile 下都会把以下 execution 关闭:

- yarn install
- yarn run build

POM 注释写得很明确:

- Skip yarn install - workspaces must be initialized at root first
- Skip yarn build - not needed for OSS release, frontend already built by Copybara

这里透露出两个非常关键的编译机制事实:

第一，子模块不应该各自安装前端依赖。

- 必须在仓库根做一次 Yarn workspace 初始化。

第二，公开 OSS release 的前端资源通常已经由上游流程预构建或预同步。

- Maven 在 public profile 下默认不重复构建这些前端资源。
- 也就是说，公开源码包更偏向“消费已准备好的前端结果”而不是“在本地完整重建所有前端产物”。

这也是 Nexus 公开发布源码和内部完整开发仓库之间的重要差异。

6. 传统 UI 资源的构建链

Nexus 3.91.1 仍然保留一条传统 UI 资源流水线。

   a. extjs-maven-plugin

这个插件位于:

- public/common/maven-plugins/extjs-maven-plugin

它在根 pluginManagement 中被统一配置为:

- goal = aggregate-js

典型用途:

- 从 src/main/resources/static/rapture 下聚合 JS
- 生成 xxx-debug.js
- 在某些模块中按 namespace 组织，如:
  - nexus-rapture: namespace = NX
  - nexus-coreui-plugin: namespace = NX.coreui

本质上它服务的是 ExtJS/Rapture 时代的资源拼接与组织。

b. yuicompressor-maven-plugin

该插件在多个模块中把 debug 资源进一步转成 prod 资源:

- JS 聚合压缩生成 xxx-prod.js
- CSS 聚合压缩生成 xxx-prod.css

根 POM 还提供 disable-optimize profile，可关闭压缩/混淆优化，方便调试。

所以传统 UI 的构建顺序可以概括为:

- 先通过 extjs-maven-plugin 聚合 debug 版 JS
- 再通过 yuicompressor 产出 prod 版 JS/CSS

7. Java 组件层是如何编译并被最终发行包消费的

a. common/components 是真正的主干

public/common/components/pom.xml 下聚合了几十个模块，例如:

- nexus-base
- nexus-common

- nexus-core
- nexus-datastore
- nexus-repository-services
- nexus-repository-view
- nexus-blobstore 系列
- nexus-rapture
- nexus-ui-plugin
- nexus-coreui-plugin
- formats/nexus-repository-maven
- formats/nexus-repository-apt
- formats/nexus-repository-raw
- 安全、脚本、升级、webhooks 等

这层先把所有业务能力拆成细颗粒 jar。

b. dependencyManagement 统一锁版本

common/components 父 POM 里几乎把所有组件版本都显式锁成 3.91.1-04，并导入 buildsupport BOM。

所以子模块间互相依赖时:

- 基本不需要各自再写版本
- 构建时版本统一性由父 POM 保证

c. selfhosted/components 再做“发行形态增强”

例如:

- nexus-self-hosted: 自托管特有整合模块
- nexus-repository-core-edition: core 发行版特有增强，并额外指定 aspectj aspectLibrary
- selfhosted API 模块: 提供自托管相关 API 封装

也就是说，public/common/components 更像“公共能力层”，selfhosted/components 更像“发行版拼装增强层”。

8. 最终产物为什么是“jar + assembly 目录”的双层装配

    a. 最终发行模块

真正的最终发行模块是:

- public/selfhosted/assemblies/nexus-repository-core

它的 packaging 是:

- jar

这一步先生成 Spring Boot 可执行 jar。

    b. 这个模块依赖了几乎整个产品栈

它直接依赖:

- 基础组件
- 数据存储组件
- UI 组件
- 各类 repository format
- 各类 plugin
- selfhosted 组件
- API 模块

因此它本质上是“最终产品聚合器”。

    b. spring-boot-maven-plugin 的作用

该模块在 package 阶段执行:

- spring-boot:repackage

结果是:

- 把本模块及其依赖重打包成可执行 uber-jar
- 这是最终启动核心

c. 为什么还要 overlay/assembly

仅有 uber-jar 还不够，因为 Nexus 运行还需要:

- bin 启动脚本
- 配置模板
- 目录结构
- sonatype-work 初始内容

因此它又依赖:

- org.sonatype.nexus.common.assemblies:nexus-base-overlay:zip

这个 zip 由 public/common/assemblies/nexus-base-overlay 生成。

9. overlay 装配机制

a. nexus-base-overlay 的作用

public/common/assemblies/nexus-base-overlay:

- packaging = pom
- prepare-package 阶段:
  - 把 src/main/resources/overlay 复制到 target/assembly
  - 把 src/main/resources/sonatype-work 复制到 target/sonatype-work
  - 修正 bin 下脚本的换行符
- package 阶段:
  - 用 maven-assembly-plugin 根据 bundle.xml 打成 zip

这个 zip 是“基础运行骨架包”，包含:

- 基础目录结构
- 脚本
- 初始工作目录模板

b. nexus-repository-core 如何消费 overlay

最终发行模块在 package 阶段执行自定义 antrun:

- 先把 nexus-base-overlay.zip 解压到 target/assembly
  - 排除 lib
  - 排除 sonatype-work
- 再把其中 sonatype-work 解压到 target/sonatype-work
- 再叠加本模块自己的 overlay
- 再修正脚本换行符
- 最后把刚刚 repackage 出来的 uber-jar 复制到 assembly/bin

这一步是整个 Nexus 发行包装配的核心。

c. 最终 assembly 描述文件做了什么

public/selfhosted/assemblies/nexus-repository-core/src/main/assembly/bundle.xml 定义:

- 把 target/assembly 复制到 ${project.build.finalName}
- 把 target/sonatype-work 放到最终 zip 的 sonatype-work
- 把最终 jar 单独放到 ${project.build.finalName}/bin

这说明最终 zip 包内部不是只有一个 jar，而是:

- 一套可启动目录树
- 一个 bin 下的可执行 jar
- 一个 sonatype-work 初始目录

10. 运行机制与编译机制的耦合点

    a. 运行入口

README 指出编译后可运行目录在:

- public/selfhosted/assemblies/nexus-repository-core/target/assembly

运行方式:

- java -jar bin/nexus-repository-core-*.jar

这说明:

- `target/assembly` 是运行目录
- `bin/*.jar` 是启动核心

    b. exec-maven-plugin 的开发运行支持

最终发行模块配置了 exec-maven-plugin:

- executable = java
- workingDirectory = target/assembly
- arguments = -jar target/${finalName}.jar

并设置环境变量:

- NEXUS_RESOURCE_DIRS = 多个本地资源目录和 target/classes
- NEXUS_SECURITY_RANDOMPASSWORD = false

这说明开发态运行支持两件事:

- 允许 `?debug` 模式直接从文件系统加载资源
- 关闭随机初始密码，方便开发

所以该项目不是“编译完再手工整理运行目录”，而是 Maven 已经内置了开发运行模式。

11. 源码包与完整仓库的实际差异

这次分析对象是一个发布源码包，不是完整 Git checkout。这个差异对“能否按 README 原样编译”影响很大。

已确认当前目录缺少:

- mvnw
- mvnw.cmd
- .mvn/
- .yarnrc.yml
- yarn.lock

但 README 和根 POM 又明确提到了:

- Maven wrapper
- .yarnrc.yml 指向 Yarn 4.9.1
- 根工作区初始化

这说明当前包是“面向发布/审计/源码公开”的裁剪版，而不是完整开发工作树。

它带来的直接影响是:

a. README 中的 `./mvnw clean install -Ppublic` 在当前目录不能原样执行

- 因为 wrapper 文件不存在

b. 根工作区 Yarn 4 的真实运行配置当前包中不可见

- 根 POM 注释说明实际依赖 .yarnrc.yml
- 但当前发布包里缺失该文件

c. 子模块 public profile 又默认跳过本地前端 build

- 这进一步说明 public release 偏向使用“已经准备好的前端构建结果”

因此，当前源码包更准确的理解应该是:

- 它完整表达了 Maven 模块结构和装配机制
- 但不一定完整携带“从零重建所有前端依赖和官方开发工作流”的全部辅助文件

12. 按机制还原一次完整构建流程

如果站在完整官方仓库视角，标准编译链路可以还原为:

第 1 步，准备环境

- JDK 21
- Maven 3.9.6+
- Node 22
- Corepack

第 2 步，初始化前端工作区

- 在仓库根执行 corepack enable
- 执行 yarn install

目的:

- 为三大 workspace 模块统一安装依赖
- 建立 workspace 软链接和解析关系

第 3 步，启动 Maven 聚合构建

- clean install -Ppublic

Maven 内部大致顺序:

- validate
  - enforcer 校验 Maven/Java/依赖规则
  - 可能执行 spotless apply 等 profile 逻辑
- initialize
  - buildnumber/timestamp 生成构建标识

    - rapture 等模块生成 resource timestamp
- generate-resources
    - frontend-maven-plugin 准备 Node/Yarn
    - 根级 Yarn install execution 可参与资源准备
- compile
    - Java 模块编译
    - AspectJ weave
    - 部分前端模块 Yarn build
    - extjs 资源聚合
- test
    - surefire 执行 Java 单测
    - 前端模块 yarn test/jest
- package
    - 普通组件打 jar
    - sources.jar 打包前端源
    - base overlay 打 zip
    - nexus-repository-core 进行 spring-boot repackage
    - 解压 overlay
    - 复制 uber-jar
    - 组装 assembly 目录
    - 最终发行 zip 生成
- install
    - 所有模块安装到本地 Maven 仓库

第 4 步，运行最终产物

- 进入 public/selfhosted/assemblies/nexus-repository-core/target/assembly
- 执行 java -jar bin/nexus-repository-core-3.91.1-04.jar

13. 如果站在当前“发布源码包”视角，实际可执行策略是什么

由于当前目录缺失 wrapper 和部分 Yarn 根配置，直接照 README 原样跑存在落差。

更现实的理解是:

a. 它依然足够用来分析完整编译机制

- 因为关键 POM、assembly 描述、package.json 都齐全

b. 但如果要在当前包上做“零修补构建”，需要额外补齐

- Maven wrapper 或直接使用本机 Maven
- 可能需要补齐 yarn.lock / .yarnrc.yml / 其他 workspace 根配置

c. public profile 设计本身已经在降低公开源码包的前端重建要求

- 子模块跳过 yarn install/build
- 更偏向复用上游已准备资源

因此，当前源码包更适合:

- 审计构建逻辑
- 研究模块依赖关系
- 做定制改造前的结构分析

如果要严格做全量重编译，最好基于官方完整 Git 分支而不是仅靠这个 release zip。

14. 编译机制里的几个关键设计特征

a. Maven 是绝对主控

- 即使有前端构建，调度中心仍是 Maven，不是 npm/yarn 脚本。

b. 前端不是独立部署，而是嵌入 Java 组件

- React/Rspack、ExtJS、Rapture 产物最终都作为静态资源进入 Java 模块或最终发行包。

c. 项目处于新旧前端技术混合阶段

- 既有 React 19 + Rspack + Jest

- 也保留 ExtJS 聚合与 YUI Compressor 流程

d. 最终发行采用“应用 uber-jar + 目录 overlay”模式

- 这种方式兼顾 Spring Boot 启动便利性和传统产品目录结构需求。

e. AOP/横切能力较重

- AspectJ 不是可有可无的附属品，而是核心编译链的一部分。

f. public profile 不等于“完整源码开发模式”

- 它更像“公开版/发布版构建兼容模式”。

15. 对实际编译的建议

如果目的是“理解机制”，当前源码包已经足够。

如果目的是“本地二次开发并稳定编译”，建议按优先级处理:

a. 优先获取完整 Git 仓库对应分支

- 例如 release-3.91.1-04

b. 使用 Java 21 + Maven 3.9.6+

c. 按仓库根工作区初始化前端

- 不要进入子模块各自 npm install

d. 明确自己是在走哪条路线

- 发布版兼容路线: `-Ppublic`
- 内部/完整开发路线: 可能需要完整 Yarn 4 工作区文件和未裁剪资源

e. 如果只关注后端改动

- 也要注意某些 UI 模块、AspectJ 模块、assembly 模块仍然可能影响最终 package

16. 最终总结

Nexus 3.91.1-04 的源码编译机制可以概括为:

- 用根 Maven POM 统一管理版本、依赖、插件和 profile。
- 用多级聚合模块把 common 能力、自托管能力、最终发行装配分层组织。
- 用 frontend-maven-plugin 把 Node/Yarn 工作流嵌入 Maven 生命周期。
- 用 Yarn workspace 统一管理前端依赖。
- 用 extjs-maven-plugin 与 yuicompressor 维持传统 Rapture/ExtJS 资源流水线。
- 用 AspectJ 在编译阶段织入事务、安全、校验等横切能力。
- 用 spring-boot-maven-plugin 产出可执行 uber-jar。
- 用 overlay + assembly 把 jar、脚本、配置模板、sonatype-work 组装成最终可运行发行包。

对当前这个 release 源码包来说，最重要的现实判断是:

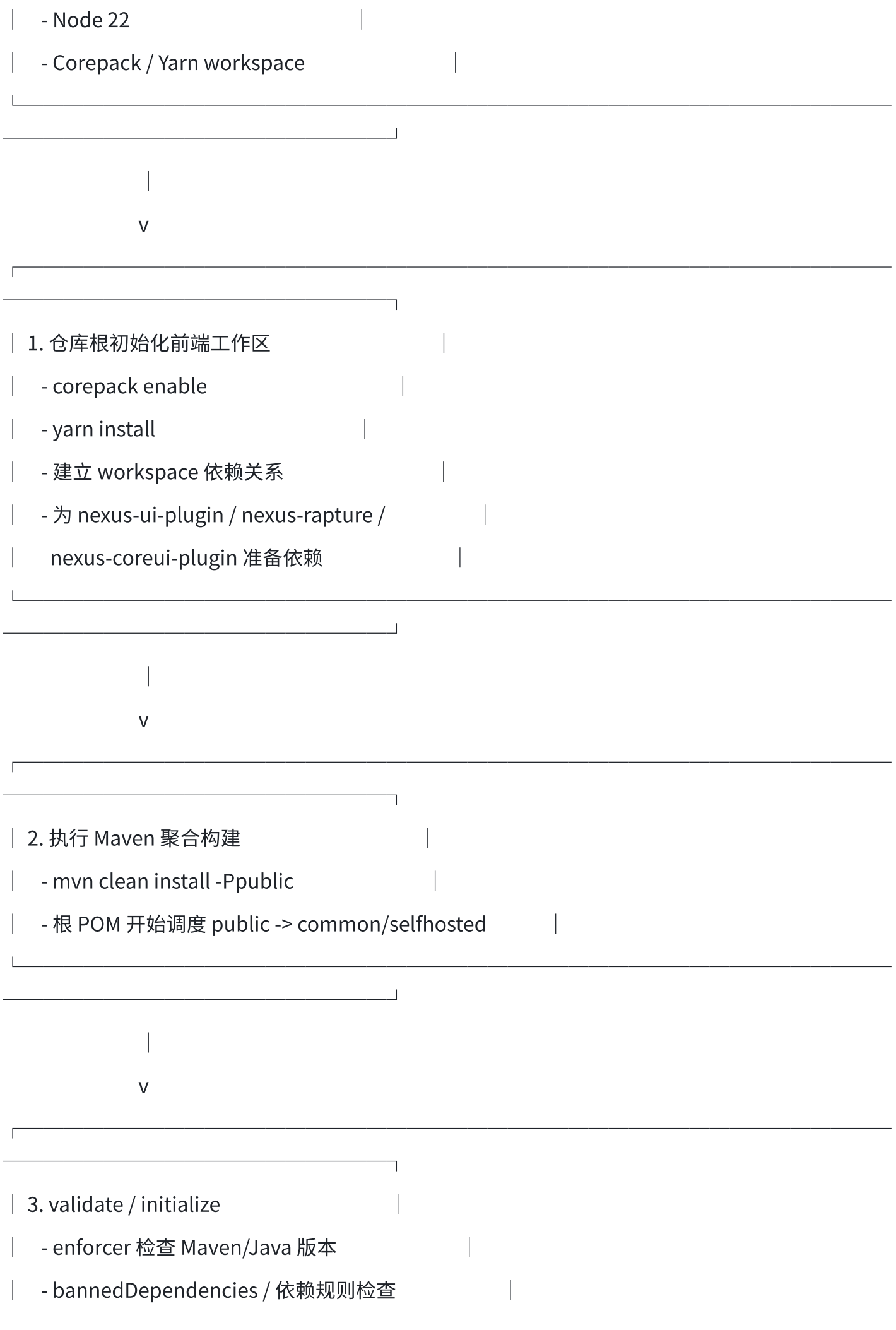
- 它完整展示了“编译机制怎么设计”
- 但不完全等价于“官方完整开发仓库”
- 因此前端根配置和 wrapper 缺失会让 README 命令无法 100% 原样落地
- 真正做全量可重复构建时，应尽量基于官方完整分支而不是仅依赖该 zip 包

17. 编译流程图

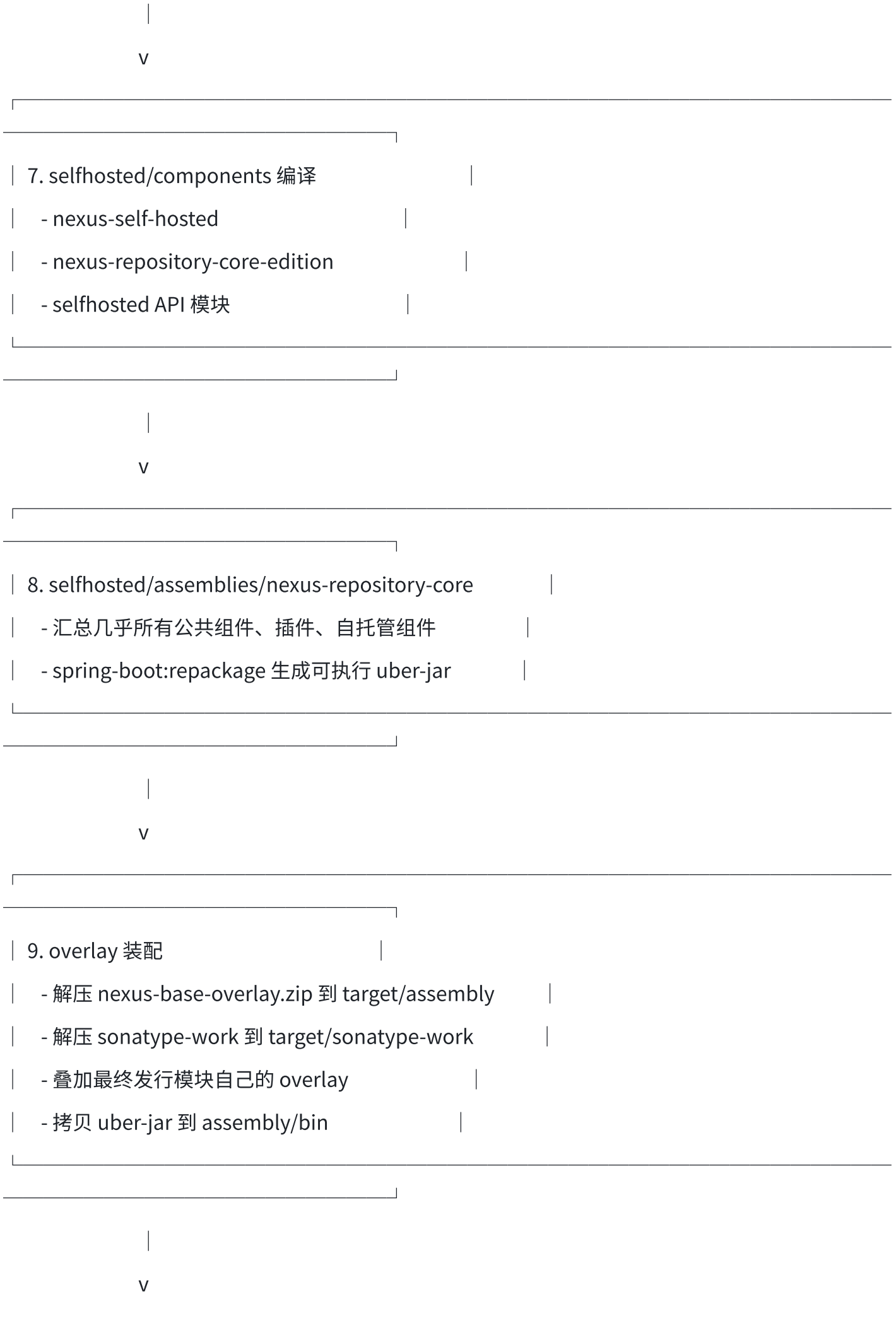
下面给出一版按“完整官方仓库视角”整理的编译流程图。

1. 总体编译流程图

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ 0. 准备环境                                                                  │
│    - JDK 21                                                                  │
│    - Maven 3.9.6+                                                            │
```


- Node 22
- Corepack / Yarn workspace
v
1. 仓库根初始化前端工作区
- corepack enable
- yarn install
- 建立 workspace 依赖关系
- 为 nexus-ui-plugin / nexus-rapture /
nexus-coreui-plugin 准备依赖
v
2. 执行 Maven 聚合构建
- mvn clean install -Ppublic
- 根 POM 开始调度 public -> common/selfhosted
v
3. validate / initialize
- enforcer 检查 Maven/Java 版本
- bannedDependencies / 依赖规则检查

```
|   - buildnumber / timestamp 生成构建标识                |
└──────────────────────────────────────────────────────────┘
                          |
                          v
┌──────────────────────────────────────────────────────────┐
| 4. common/buildsupport / common/maven-plugins            |
|   - 导入 BOM / 统一依赖管理                              |
|   - 编译 extjs-maven-plugin 等自定义构建插件             |
└──────────────────────────────────────────────────────────┘
                          |
                          v
┌──────────────────────────────────────────────────────────┐
| 5. common/components 编译                                |
|   - 大量 Java 组件编译成 jar                             |
|   - AspectJ weave 注入事务/安全/校验逻辑                 |
|   - 部分模块执行 extjs 聚合 / yui 压缩                   |
|   - 前端模块执行 test 或消费已准备资源                   |
└──────────────────────────────────────────────────────────┘
                          |
                          v
┌──────────────────────────────────────────────────────────┐
| 6. common/assemblies 编译                                |
|   - 生成 nexus-base-overlay.zip                          |
|   - 内含基础目录结构、bin 脚本、sonatype-work 模板       |
└──────────────────────────────────────────────────────────┘
```


7. selfhosted/components 编译
- nexus-self-hosted
- nexus-repository-core-edition
- selfhosted API 模块
8. selfhosted/assemblies/nexus-repository-core
- 汇总几乎所有公共组件、插件、自托管组件
- spring-boot:repackage 生成可执行 uber-jar
9. overlay 装配
- 解压 nexus-base-overlay.zip 到 target/assembly
- 解压 sonatype-work 到 target/sonatype-work
- 叠加最终发行模块自己的 overlay
- 拷贝 uber-jar 到 assembly/bin

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 10. maven-assembly-plugin 打最终分发包                                                                                                        │
│    输出内容:                                                                                                                                  │
│    - target/assembly 运行目录                                                                                                                 │
│    - bin/nexus-repository-core-3.91.1-04.jar                                                                                                  │
│    - sonatype-work 初始目录                                                                                                                   │
│    - 最终 zip 分发包                                                                                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

4.2 nexus发布形式

1. 二进制压缩包

   Linux/Unix：`nexus-3.91.4-01-unix.tar.gz`（64 位，内置 JRE）

   Windows：`nexus-3.91.4-01-win64.zip`（64 位，内置 JRE）

   macOS：`nexus-3.91.4-01-mac.tgz`（64 位，适配 Apple Silicon/Intel）

   特点：绿色免安装、解压即用、内置 Jetty 容器、数据目录独立（`sonatype-work`）

2. 容器镜像

官方镜像：`sonatype/nexus3:3.91.4`（Alpine 基础镜像，体积约 500MB）

部署方式：Docker Hub 拉取、自定义 Dockerfile 构建、K8s StatefulSet 部署

特点：环境隔离、一键启停、数据卷挂载（`/nexus-data`）、支持集群部署

3. 系统安装包（Linux 包管理）

RPM（CentOS/RHEL）：`nexus-repository-manager-3.91.4-01.x86_64.rpm`

DEB（Ubuntu/Debian）：`nexus-repository-manager_3.91.4-01_amd64.deb`

特点：系统级安装、服务托管（systemd）、自动配置环境变量、便于统一运维

4.3 nexus代码模块（版本3.91.1-04 ）

4.3.1 Nexus 整体代码模块（共 187 个 Maven 模块）

4.3.1.1 模块总数统计

- 顶级模块：12 个（根 pom 聚合）

- 核心功能模块：43 个（OSGi Bundle）
- 插件模块：76 个（格式支持 / 扩展功能）
- 构建支持模块：21 个（依赖管理 / 编译工具）
- 装配模块：8 个（发行包组装）
- 测试模块：27 个（单元 / 集成 / 性能测试）

4.3.1.2 模块架构分层（自上而下）

（1）装配层（assemblies，8 个模块）

- `nexus-repository-core`：主发行包组装（tar/zip）Maven Central
- `nexus-repository-docker`：Docker 镜像构建配置
- `nexus-repository-rpm/deb`：系统安装包打包

（2）核心层（components，43 个模块，OSGi 核心）

- 基础核心（7 个）
  - `nexus-core`：公共工具、异常、模型、Guice 依赖注入
  - `nexus-common`：通用组件（缓存、IO、安全、日志）
  - `nexus-osgi`：Felix 框架封装、Bundle 生命周期管理
- 服务核心（12 个）
  - `nexus-repository-content`：仓库内容管理（CRUD、存储抽象）
  - `nexus-repository-security`：权限、认证、用户管理（Realm 链）
  - `nexus-repository-task`：异步任务调度（定时、后台作业）
  - `nexus-repository-search`：Elasticsearch 全文检索
- 存储核心（8 个）
  - `nexus-blobstore`：二进制存储抽象（文件 / S3 / 阿里云）
  - `nexus-blobstore-file`：本地文件存储实现
  - `nexus-blobstore-s3`：对象存储适配
- Web 核心（10 个）
  - `nexus-coreui-plugin`：前端 UI（React 主界面）
  - `nexus-rest-api`：REST API（OpenAPI 3.0）
  - `nexus-jetty`：嵌入式 Jetty 容器配置

（3）插件层（plugins，76 个模块，格式 / 扩展）

- 仓库格式插件（22 个）：Maven、Docker、npm、PyPI、Helm、Raw、Go 等

- 认证插件（8 个）：LDAP、OIDC、SAML、Token、CAS
- 健康检查 / 监控插件（6 个）：Prometheus、Metrics、Health Check
- 其他扩展插件（40 个）：清理、备份、审计、Webhook、集群同步

（4）构建支持层（buildsupport，21 个模块）

- `nexus-buildsupport-parent`：全局依赖管理（统一版本）
- `nexus-buildsupport-maven`：Maven 插件配置
- `nexus-buildsupport-rest`：REST 接口规范
- `nexus-buildsupport-db`：数据库（H2/PostgreSQL）适配

4.3.2 模块依赖情况（核心依赖链）

4.3.2.1 依赖原则

- 单向依赖：下层模块不依赖上层，核心模块被插件 / 装配模块依赖
- OSGi 解耦：通过接口（Export-Package）依赖，而非硬编码引用
- 版本统一：所有模块继承 `buildsupport` 依赖管理，版本一致

4.3.2.2 核心依赖链（简化）

plaintext

```
装配模块 → 核心服务模块 → 基础核心模块
    ↑
插件模块（格式/认证）→ 核心服务模块
```

4.3.2.3 关键模块依赖明细

- `nexus-repository-content`（内容管理）
  - 依赖：`nexus-core`、`nexus-blobstore`、`nexus-repository-security`
  - 被依赖：所有格式插件、`nexus-rest-api`
- `nexus-repository-security`（安全）
  - 依赖：`nexus-core`、`nexus-common`
  - 被依赖：核心服务、所有插件、UI 模块
- `nexus-coreui-plugin`（前端）
  - 依赖：`nexus-rest-api`、`nexus-core`（API 客户端）

- 被依赖：无（独立 OSGi Bundle，前端入口）

4.3.2.4 典型模块依赖树（Maven 命令查看）

bash

运行

```
代码块
1   ./mvnw dependency:tree -pl public/common/components/nexus-repository-content
```

输出摘要：

plaintext

```
代码块
1   org.sonatype.nexus.common.components:nexus-repository-content:jar:3.91.4-01
2   +- org.sonatype.nexus.common.components:nexus-core:jar:3.91.4-01:compile
3   +- org.sonatype.nexus.common.components:nexus-blobstore:jar:3.91.4-01:compile
4   +- org.sonatype.nexus.common.components:nexus-repository-security:jar:3.91.4-
    01:compile
5   +- com.google.inject:guice:jar:5.1.0:compile
```

4.4 开发支持OIDC协议认证分析

4.4.1 需求分析

4.4.1.1 功能需求

4.4.1.1.1 OIDC 身份认证（浏览器场景）

编号	需求	优先级	描述	防御性要求
F-01	Authorization Code Flow + PKCE	P0	标准授权码流程，强制 PKCE（S256）	code_verifier 使用 `SecureRandom` 生成 43-128 字符；拒绝 plain 方法
F-02	Well-Known 自动发现	P0	解析 `/.well-known/openid-configuration`	**缓存元数据**（TTL 24h）；解析失败使用缓存副本；首次加载失败阻止 Realm 启用并明确报错

F-03	ID Token 验证	P0	签名验证 + Claims 校验	禁止 `alg:none`；校验 iss/aud/exp/iat/nonce/azp；签名失败**自动刷新一次 JWKS** 再重试
F-04	OIDC 会话状态管理	P0	state/nonce 生命周期	绑定 session + 5分钟 TTL + 使用后立即销毁；`SecureRandom` ⩾32 字节
F-05	UserInfo 端点调用	P1	补充用户信息	可选调用；**失败不阻塞登录**（ID Token Claims 兜底）；超时 5s
F-06	Token Refresh	P1	Refresh Token 静默续期	刷新失败不中断当前会话；并发刷新合并为一次（参考 `Cooperation2`）
F-07	单点登出	P2	Front/Back-channel Logout	清除本地 session；**不撤销 PAT**（长期凭据不应被 SLO 影响）

4.4.1.1.2 个人访问令牌，PAT（CLI认证）

1. 生命周期

编号	需求	优先级	描述	防御性要求
F-08	PAT 创建	P0	登录用户创建 PAT，完整 Token 仅返回一次	accessKey 碰撞：`DuplicateKeyException` → 自动重试（≤3 次），复用 `ApiKeyServiceImpl` 模式
F-09	PAT 格式	P0	`nxpat-<accessKey 20字符>-<secret 40字符>`	`SecureRandom` + Base62；前缀 `nxpat-` 便于 Secret Scanning 检测
F-10	PAT 命名与描述	P0	每个 PAT 设置名称（如 "Jenkins CI"）	名称 ≤200 字符；描述 ≤500 字符；XSS 过滤
F-11	PAT 有效期	P0	创建时设置过期时间	过期检查在 `PatRealm.doGetAuthenticationInfo()` 中执行，**不依赖清理任务**；清理仅回收存储
F-12	PAT 列表查看	P0	查看自己所有 PAT（名称、创建时间、到期、最后使用）	**永不返回 Token 值**；仅显示 accessKey 前 6 位 + 掩码
F-13	PAT 撤销	P0	用户/管理员撤销	`api_key_v2` 删除 + `pat_info` 状态更新**同一事务**；撤销后清除 Shiro 认证缓存
F-14	过期 PAT 自动清理	P1	Quartz 定时任务	仅清理过期超 7 天的记录（审计窗口）；含孤立记录修复逻辑

2. 认证集成

编号	需求	优先级	描述	防御性要求
F-15	Bearer Token 认证	P0	`Authorization: Bearer nxpat-xxx`	`PatExtractor` 匹配 `nxpat-` 前缀；格式不合法返回 null（交给下一个 Extractor）
F-16	Basic Auth 兼容	P0	`username:nxpat-xxx`	`PatBasicAuthRealm` 仅在 password 以 `nxpat-` 开头时介入；**强制校验 username == PAT 绑定用户**
F-17	Docker Login	P0	`docker login -u user -p nxpat-xxx`	通过 F-16 Basic Auth 兼容透明支持
F-18	Maven/Gradle	P0	settings.xml 密码字段填 PAT	同 F-16
F-19	npm/NuGet/PyPI	P1	各格式客户端 Token/Auth	npm `_authToken` → F-15；NuGet/PyPI → F-16

3. 权限与策略

编号	需求	优先级	描述	防御性要求
F-20	PAT 权限继承	P0	实时查询绑定用户当前权限	**不缓存权限快照到 PAT**——始终实时获取，角色变更立即生效
F-21	全局 PAT 策略	P1	允许/禁止、最大有效期、每用户上限	存于 `OidcConfiguration`；超限时返回明确错误
F-22	Scoped PAT	P2	限定仓库/只读	`pat_info.scopes` JSON 字段；权限与 scope 取交集

4.4.1.1.3 用户与角色管理

编号	需求		描述	防御性要求

		优先级		
F-23	OIDC 用户自动创建	P0	首次登录自动创建	**幂等**：并发创建捕获 `DuplicateKeyException` 转为查询
F-24	用户属性同步	P1	每次登录同步 email/displayName	同步失败不阻塞登录（WARN 日志）
F-25	Claims → 角色映射	P0	可配置映射规则	**默认无角色**；变更触发事件清除授权缓存
F-26	OIDC 用户状态同步	P0	IdP 禁用/删除用户时 Nexus 响应	被动检测：OIDC 登录时检查；PAT 路径通过 `OidcUserManager.getUser()` 检查状态
F-27	用户 ID 唯一性	P0	不同来源不冲突	ID 格式 `oidc:<sub>`；source 标记 `"oidc"`

4.4.1.1.4 配置管理

编号	需求	优先级	描述	防御性要求
F-28	OIDC Provider 配置	P0	Issuer URL、Client ID/Secret、Scopes、Claim 映射	Secret 用 `SecretsService` 加密；变更后自动刷新缓存
F-29	PAT 策略配置	P1	允许/禁止、有效期上限、数量上限	与 F-28 同一配置域
F-30	Capability 集成	P0	运行时启停	禁用时 OIDC 登录隐藏；已有 PAT 是否继续有效**可配置**
F-31	连接测试	P1	验证 IdP 连通性	独立超时 10s；返回详细诊断（DNS/TLS/HTTP 状态/Token 端点）
F-32	配置 REST API	P0	`GET/PUT /api/v1/security/oidc/config`	Secret 脱敏；空 Secret 表示不修改

4.4.1.1.5 异常处理

编号	需求	优先级	描述	实现参考

F-33	IdP 通信超时与重试	P0	所有 IdP HTTP 调用设超时和有限重试	连接超时 5s，读超时 10s，重试 1 次；参考 `DefaultsCustomizer`
F-34	IdP 熔断器	P0	连续失败后快速失败	阈值 5 次；半开窗口 60s；熔断期间 OIDC 登录立即返回 503
F-35	JWKS 缓存与容错刷新	P0	本地缓存 + 验证失败触发刷新	TTL 24h；失败保留旧缓存；**并发刷新合并**为一次网络请求
F-36	Provider 元数据缓存	P0	Well-Known 响应缓存	TTL 24h；刷新失败沿用旧值 + WARN；启动时异步预热
F-37	PAT 验证零外部调用	P0	全本地完成	DB 查询 + 缓存；**IdP 宕机不影响 PAT**
F-38	认证缓存	P0	短期缓存认证结果	复用 Shiro `AuthenticationCache`；状态变更时清除
F-39	数据一致性	P0	双表一致	同事务操作；清理任务含孤立记录检测
F-40	幂等性	P0	关键操作幂等	用户创建去重；PAT 撤销幂等；state 消费后拒绝重放

4.4.1.1.6 Prometheus 监控

- 指标清单：

1. 认证指标

编号	指标名称	类型	标签	描述	需求关联
M-01	`nexus.oidc.auth.total`	Meter	`method={oidc,pat_bearer,pat_basic}`，`result={success,failure}`	认证请求总量与速率	F-01,F-15,F-16
M-02	`nexus.oidc.auth.failure`	Meter	`method=...`，`reason={expired_token,revoked_token,invalid_signature,invalid_credentials,user_disabled,user_not_found,idp_unavailable,unknown}`	认证失败分类计数	F-01~F-18
M-03	`nexus.oidc.auth.duration`	Timer	`method={oidc,pat_bearer,pat_basic}`	认证处理耗时分布（P50/P95/P99）	全局

2. IdP 通信指标

编号	指标名称	类型	标签	描述	需求关联
M-04	`nexus.oidc.idp.request.total`	Meter	`endpoint={token,userinfo,jwks,well_known}`，`result={success,failure,timeout}`	IdP HTTP 请求量与速率	F-01~F-06,F-33
M-05	`nexus.oidc.idp.request.duration`	Timer	`endpoint={token,userinfo,jwks,well_known}`	IdP 请求耗时分布	F-33
M-06	`nexus.oidc.idp.circuit_breaker.state`	Gauge	—	熔断器状态：0=关闭，1=半开，2=打开	F-34
M-07	`nexus.oidc.idp.circuit_breaker.trip.total`	Meter	—	熔断器触发次数	F-34

3. JWKS 缓存指标

编号	指标名称	类型	标签	描述	需求关联
M-08	`nexus.oidc.jwks.cache.hit`	Meter	—	JWKS 缓存命中次数	F-35
M-09	`nexus.oidc.jwks.cache.miss`	Meter	—	JWKS 缓存未命中（触发刷新）次数	F-35
M-10	`nexus.oidc.jwks.refresh.total`	Meter	`result={success,failure}`	JWKS 刷新结果	F-35
M-11	`nexus.oidc.jwks.cache.age`	Gauge	—	当前缓存的 JWKS 距上次成功刷新的秒数	F-35

4. PAT 管理指标

编号	指标名称	类型	标签	描述	需求关联
M-12	`nexus.oidc.pat.active.count`	Gauge（CachedGauge，60s）	—	当前有效 PAT 总数	F-08~F-14

M-13	`nexus.oidc.pat.created.total`	Meter	—	PAT 创建累计量	F-08
M-14	`nexus.oidc.pat.revoked.total`	Meter	—	PAT 撤销累计量	F-13
M-15	`nexus.oidc.pat.expired.total`	Meter	—	PAT 认证时因过期被拒绝的次数	F-11
M-16	`nexus.oidc.pat.cleanup.total`	Meter	`type={expired,orphaned}`	清理任务删除的记录数	F-14,F-39

5. 用户管理指标

编号	指标名称	类型	标签	描述	需求关联
M-17	`nexus.oidc.user.active.count`	Gauge（CachedGauge，300s）	—	OIDC 用户总数	F-23
M-18	`nexus.oidc.user.sync.total`	Meter	`result={created,updated,failed}`	用户同步事件计数	F-23,F-24
M-19	`nexus.oidc.user.login.total`	Meter	—	OIDC 登录成功次数（不含 PAT）	F-01

- 健康检查：

编号	需求	优先级	描述	实现
F-41	IdP 可达性健康检查	P0	检查 Well-Known 端点和 JWKS 端点是否可达	实现 `HealthCheck`，注册到 `HealthCheckRegistry`；暴露在 `/service/rest/v1/status/check` 和 `/metrics/prometheus`
F-42	PAT 存储健康检查	P1	检查 `pat_info` 表可读写	简单的 SELECT 1 探测

- 推荐告警规则

告警名称	条件	严重级	说明
`OidcIdpDown`	`nexus_oidc_idp_circuit_breaker_state == 2` 持续 > 2min	Critical	IdP 不可用，OIDC 登录中断
`OidcAuthFailureSpike`	`rate(nexus_oidc_auth_failure_total[5m])` 突增 > 3x 基线	Warning	认证失败异常飙升，可能是攻击或配置问题
`OidcJwksCacheStale`	`nexus_oidc_jwks_cache_age > 172800`	Warning	JWKS 超过 48h 未刷新，密钥轮换可能失效
`OidcPatExpiredSpike`	`rate(nexus_oidc_pat_expired_total[5m]) > 10`	Info	大量过期 PAT 仍被使用，提醒用户更新
`OidcIdpLatencyHigh`	`nexus_oidc_idp_request_duration{quantile="0.99"} > 5`	Warning	IdP 响应延迟过高，登录体验受影响
`OidcPatStorageUnhealthy`	HealthCheck `oidc-pat-store` = unhealthy	Critical	PAT 存储不可用，所有 PAT 认证中断

4.4.1.1.7 审计日志

编号	需求	优先级	描述	实现参考
F-43	认证事件审计	P1	记录所有认证事件	复用 `NexusAuthenticationEvent` + `AuthenticationEventSubscriber`；增加 `authMethod`（oidc/pat_bearer/pat_basic）、`patName` 字段
F-44	PAT 管理审计	P1	记录 PAT 创建/撤销/清理	参考 `UserTokenAuditor` 的 domain + attributes 模式
F-45	配置变更审计	P1	OIDC 配置修改记录	Capability 变更事件

4.4.1.1.8 REST API

端点	方法	功能	权限	优先级
`/api/v1/security/oidc/config`	GET	获取配置（Secret 脱敏）	`security:oidc:read`	P0
`/api/v1/security/oidc/config`	PUT	更新配置	`security:oidc:update`	P0
`/api/v1/security/oidc/test`	POST	测试 IdP 连通性	`security:oidc:update`	P1
`/api/v1/security/pat`	POST	创建 PAT（返回完整 Token）	已认证用户	P0
`/api/v1/security/pat`	GET	列出当前用户 PAT	已认证用户	P0
`/api/v1/security/pat/{id}`	DELETE	撤销 PAT	自己的或 `security:pat:delete`	P0
`/api/v1/security/pat/admin`	GET	管理员查看所有 PAT	`security:pat:admin`	P1
`/api/v1/security/pat/admin/{userId}`	DELETE	管理员批量撤销	`security:pat:admin`	P1

4.4.1.2 非功能需求

编号	类别	需求	指标
NF-01	性能	PAT 认证延迟	< 5ms（缓存）/ < 20ms（DB）
NF-02	性能	OIDC 登录延迟	< 3s（不含 IdP 时间）
NF-03	性能	JWKS 缓存内存	< 1MB
NF-04	性能	Metrics 采集开销	< 1% CPU；Gauge 查询 < 50ms
NF-05	可用性	IdP 宕机时 PAT	100% 不受影响
NF-06	可用性	熔断器响应速度	熔断后 OIDC 请求 < 100ms 返回
NF-07	安全性	PAT 熵	≥ 120 bit
NF-08	安全性	Secret 存储	加密（SecretsService），明文不落盘

NF-09	兼容性	IdP 支持	Keycloak / Azure AD / Okta / Google / ADFS
NF-10	兼容性	CLI 客户端	Maven / Gradle / Docker / npm / NuGet / PyPI / curl
NF-11	兼容性	现有认证	Basic Auth / 原有 API Key / LDAP 无影响
NF-12	可观测性	Prometheus 指标	19 个指标覆盖认证、IdP、缓存、PAT、用户 5 个域
NF-13	可观测性	健康检查	IdP + PAT 存储注册到 HealthCheckRegistry
NF-14	可观测性	审计日志	所有安全事件结构化记录
NF-15	可维护性	模块独立	单一 Maven 模块，Capability 启停，指标随 Capability 注册/注销

4.4.2 现状分析

1. 认证架构现状

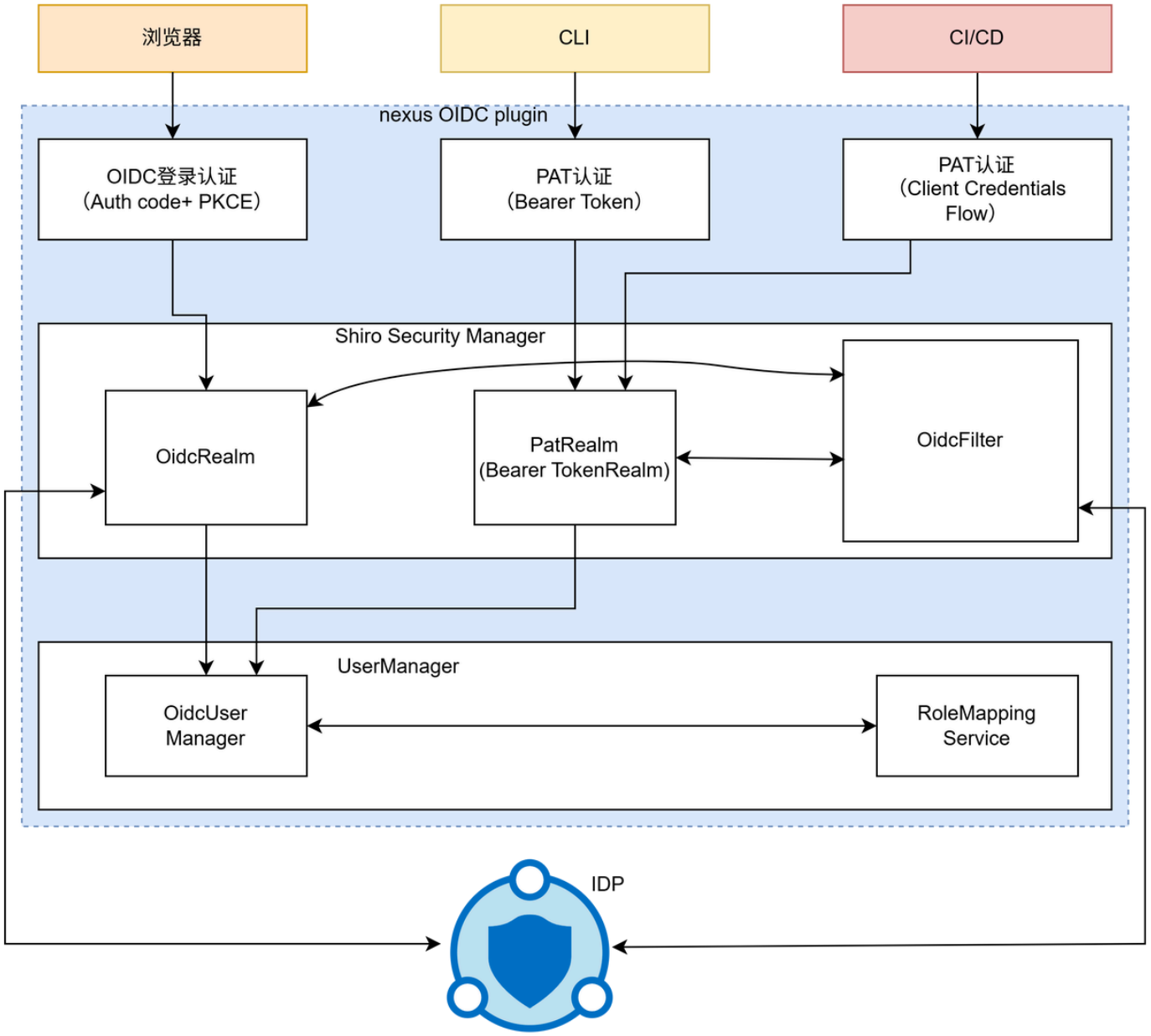
层级	现状	关键类
框架	Apache Shiro，多 Realm 插拔，First-Success 策略	`RealmManagerImpl`，`FirstSuccessfulModularRealmAuthenticator`
浏览器认证	Session Cookie（NXSESSIONID）/ JWT（NX_JWT）	`NexusWebSessionManager`，`JwtHelper`
CLI 认证	Basic Auth + API Key（Header 提取）	`NexusBasicHttpAuthenticationFilter`，`ApiKeyAuthenticationFilter`
Token 体系	V2 API Key 系统（accessKey + 加密 secret，双字段分离存储）	`ApiKeyStoreV2Impl`，`SecretsService`
Bearer Token	抽象框架已就绪（format-based 路由）	`BearerTokenRealm`，`BearerTokenManager`，`BearerToken`
用户管理	`UserManager` 接口，支持多 source（本地/LDAP/预留 SAML/OAuth2）	`ConfiguredUsersUserManager`

OIDC/PAT	OSS 版无实现	仅有 `OAUTH2_SOURCE` 常量占位

2. Metrics / Prometheus 基础设施现状

项目	现状
核心框架	Dropwizard Metrics（`MetricRegistry("nexus")`）
Prometheus 桥接	`simpleclient_dropwizard` → `DropwizardExports` 自动转换
Prometheus 端点	`/service/rest/v1/metrics/prometheus`（`PrometheusMetricsResource`）
注解支持	`@Timed`（方法计时）、`@ExceptionMetered`（异常计数）、`@Gauge`、`@CachedGauge`
手动注册	`registry.meter()` / `registry.timer()` / `registry.gauge()`
健康检查	`HealthCheckRegistry` → `/service/rest/v1/status/check`
权限	所有指标端点需 `nexus:metrics:read` 权限
认证相关指标	**无** — 目前没有任何认证相关的 Prometheus 指标

4.4.3 整体技术架构



OIDC 授权流：nexus OIDC插件向idp发送唯一代码。然后，idp将唯一代码发送回 nexus，以交换令牌。使用此方法，OpenID 提供程序可以在发送令牌之前验证信赖方。浏览器无法在此方法中看到令牌，这有助于确保令牌安全。

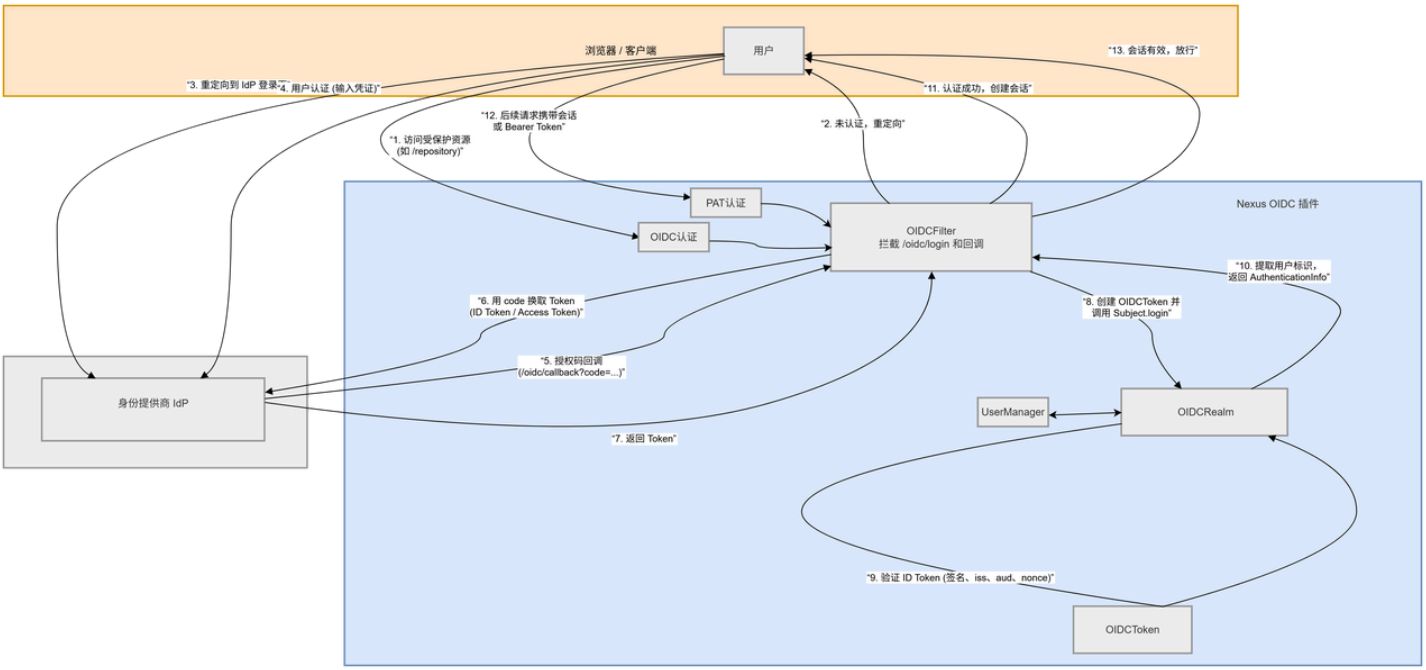
具有 PKCE 扩展的 OIDC 授权流：此流与 OIDC 授权流相同，只不过它使用代码交换公钥 (PKCE) 扩展以哈希方式发送通信。这样可以减少令牌被拦截的可能性。

客户端凭据：此流通过使用应用程序本身的标识提供对 Web API 的访问权限。它通常用于服务器到服务器的通信和无需用户交互的自动化脚本。

OIDCFilter：扩展 Shiro 的 AuthenticatingFilter。这个自定义过滤器会拦截所有 /oidc/login 等特定请求，在其中处理向 IdP 发起授权请求、接收回调并解析 Token 的逻辑。

OIDCRealm：这个 Realm 将接管 Nexus 的认证流程。

4.4.4 数据流图



1. `OIDCToken`：认证凭证

在 Shiro 中，`AuthenticationToken` 封装了用户的认证凭证。对于 OIDC，`OIDCToken` 主要用来携带从 IdP 回调获得的授权码（code）。

2. `OIDCFilter`：流程控制

`OIDCFilter` 继承自 `AuthenticatingFilter`，负责拦截并驱动整个 OIDC 授权流程：

- 拦截认证请求：在 `onAccessDenied` 方法中，它会从请求中提取 `code` 参数。
- 触发认证：一旦获取到 `code`，它会创建 `OIDCToken`，然后调用 `executeLogin(request, response)` 将 token 传递给 `SecurityManager`，进而触发 `Realm` 的认证逻辑。

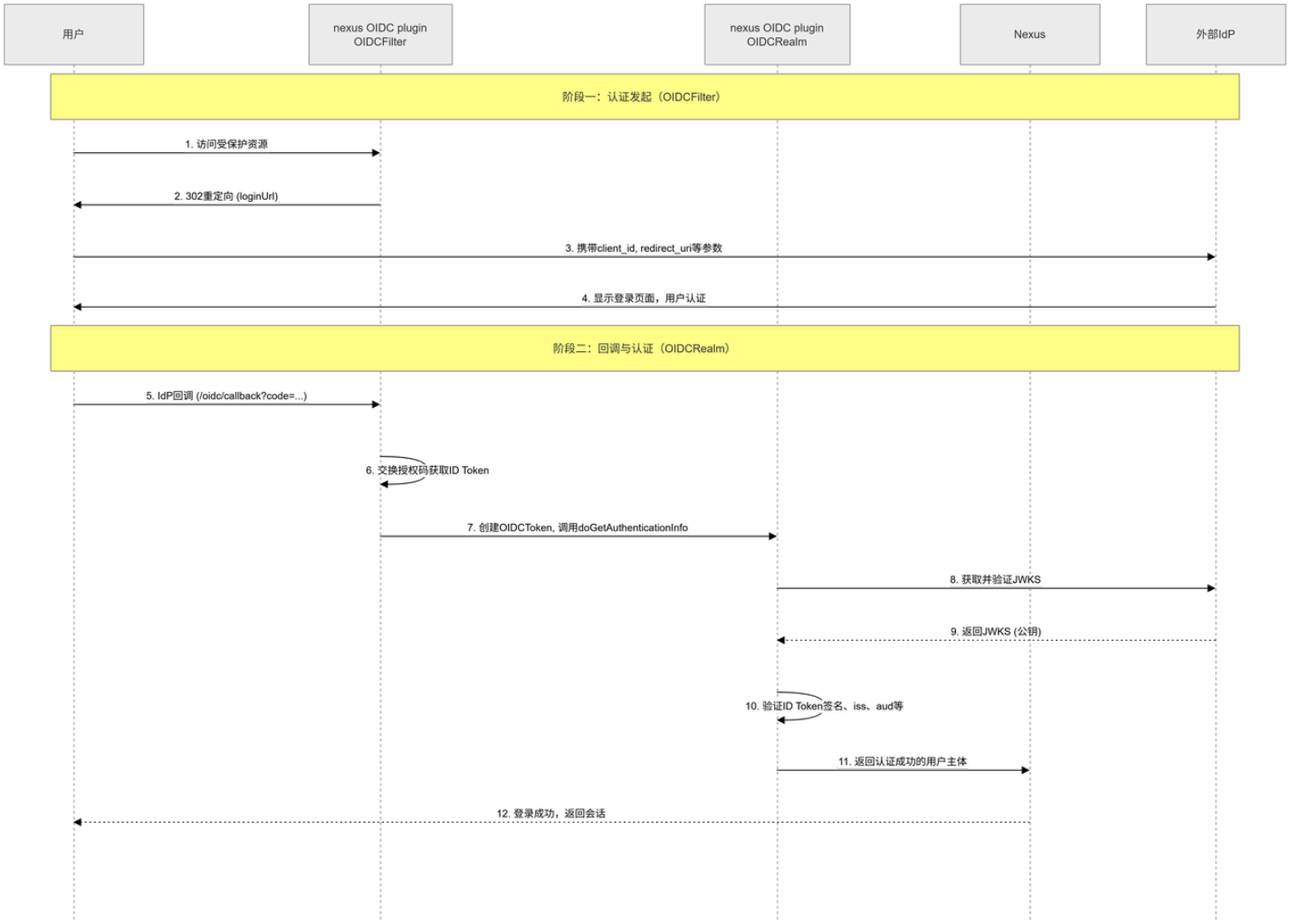
3. `OIDCRealm`：负责认证

`OIDCRealm` 是整个方案的核心，它必须继承 `AuthorizingRealm` 并重写 `doGetAuthenticationInfo` 方法。

非授权码模式的支持

如果你的客户端架构不同，获取 Token 的方式也不同：

- 授权码模式（Authorization Code Flow）：服务端 Web 应用的标准模式，就是上面代码示例的实现思路。
- PKCE 模式：移动端或单页应用必须强制使用。在回调的 Filter 中，不仅需要从 URL 中获取 `code`，还要附带 `code_verifier` 来换取 Token，以防止授权码拦截攻击。此时 `OIDCToken` 需要同时包含 `code` 和 `code_verifier`。



4.4.5 技术选型

组件	选型	理由
OIDC/JWT 库	**nimbus-oauth2-oidc-sdk** + **nimbus-jose-jwt**	最成熟；Nexus 已依赖 nimbus-jose-jwt
HTTP Client	复用 Nexus **Apache HttpClient**	`DefaultsCustomizer` 已配超时/重试/连接池
PAT 存储	复用 **ApiKeyStoreV2** + 新建 `pat_info` 表	V2 加密存储 + accessKey 索引 + 创建者追踪
配置存储	**MyBatis DAO**	与现有模式一致
定时任务	Nexus **Quartz**	PAT 清理任务
健康检查	**HealthCheckRegistry**	参考 `DefaultEncryptionKeyHealthCheck`
Metrics	**Dropwizard MetricRegistry("nexus")**	自动桥接 Prometheus；用 `@Timed` + 手动 Meter/Gauge
熔断器	简单自实现	计数器 + 时间窗口；不引入重依赖

4.4.6 需要新增的模块

新建一个独立插件模块 nexus-oidc-plugin：

代码块

```
└── src/main/java/org/sonatype/nexus/plugins/oidc/
    │
    ├── OidcPlugin.java                              ← 插件入口
    │
    ├── auth/                                        ← 认证核心
    │   ├── OidcRealm.java                           ← 浏览器 OIDC 登录
    │   ├── PatRealm.java                            ← PAT Bearer Token 认证
    │   ├── PatBasicAuthRealm.java                   ← PAT 作为 Basic Auth 密码
    │   ├── OidcAuthenticationToken.java
    │   ├── OidcCallbackFilter.java                  ← /oauth/callback
    │   ├── OidcLoginRedirectFilter.java
    │   └── OidcSessionManager.java                  ← state/nonce/PKCE 状态管理
    │
    ├── token/                                       ← PAT 管理（基于 ApiKey V2）
    │   ├── PatService.java                          ← 生命周期：创建/撤销/查询/验证
    │   ├── PatFactory.java                          ← implements ApiKeyFactory
    │   ├── PatExtractor.java                        ← implements
ApiKeyExtractor
    │   ├── PatInfo.java                             ← 元数据模型
    │   ├── PatInfoStore.java                        ← pat_info 表持久化
    │   └── PatCleanupTask.java                      ← 过期清理 + 孤立记录清理
    │
    ├── user/                                        ← 用户管理（OIDC + PAT 共用）
    │   ├── OidcUserManager.java
    │   ├── OidcUser.java
    │   └── RoleMappingService.java
    │
    ├── provider/                                    ← IdP 通信
    │   ├── OidcProviderClient.java                  ← HTTP 封装（超时/重试/熔断）
    │   ├── OidcProviderMetadata.java                ← Well-Known 解析 + 缓存
    │   ├── JwksManager.java                         ← JWKS 获取/缓存/轮换
    │   └── IdTokenValidator.java                    ← ID Token 完整性验证
    │
    ├── config/                                      ← 统一配置
    │   ├── OidcConfiguration.java
    │   ├── OidcConfigurationStore.java
    │   ├── OidcCapability.java
    │   └── OidcCapabilityDescriptor.java
    │
    ├── resilience/                                  ←异常处理
    │   ├── IdpCircuitBreaker.java                   ← IdP 熔断器
    │   └── TokenCollisionResolver.java              ← accessKey 碰撞重试
```

```
42      │
43      ├── metrics/                                    ← Prometheus 监控
44      │   ├── OidcMetrics.java                        ← 指标注册中心（统一管理所有指
    标）
45      │   ├── OidcMetricsConstants.java               ← 指标名称常量
46      │   └── IdpHealthCheck.java                     ← HealthCheck 实现
47      │
48      └── rest/
49          ├── OidcApiResource.java
50          └── PatApiResource.java
```

4.4.7 工作量分析

按 PRD 的功能域分解为 **7 个工作领域**，再拆解为原子任务：

领域	原子任务数	预估总人天
WA-1: 项目脚手架与基础设施	6	5.5
WA-2: 配置与 Capability 层	10	11
WA-3: IdP 通信	12	16
WA-4: OIDC 认证流程	11	15.5
WA-5: 用户与角色管理	8	9
WA-6: PAT 全生命周期	16	18.5
WA-7: Prometheus 监控与健康检查	8	8.5
WA-8: REST API	8	8
WA-9: 前端 UI	6	11
WA-10: 测试（集成 + 端到端）	10	17
WA-11: Phase 2 增强功能	12	16.5
WA-12: Phase 3 高级功能	5	9.5
合计	**112**	**146**

WA-1: 项目脚手架与基础设施

任务 ID	任务描述	需求关联	人天	评估理由
T-1.1	创建 `nexus-oidc-plugin` Maven 模块：pom.xml、依赖声明（nimbus-oauth2-oidc-sdk、nimbus-jose-jwt）、父 POM 集成、OSGi bundle 配置	全局	1	Nexus 插件 pom 有特殊的 OSGi/Karaf 打包要求，需参考 `nexus-default-role-plugin` 模板适配；依赖 scope 和版本锁定需与 Nexus BOM 对齐
T-1.2	创建 `OidcPlugin.java` 插件入口：Feature Flag `nexus.oidc.enabled`、条件化 Bean 加载	F-30	0.5	简单的 Spring `@Configuration` + `@ConditionalOnProperty`；参考现有 `FeatureFlags` 模式
T-1.3	创建包结构和空骨架类（auth/token/user/provider/config/resilience/metrics/rest 各包）	全局	0.5	纯结构性工作，为后续并行开发建立约定
T-1.4	编写 `pat_info` 表 DDL 和 MyBatis migration 脚本	F-08~F-14	1	需编写建表 SQL、索引、外键约束；Nexus 使用 MyBatis + Flyway 风格迁移，需符合版本编号规范；含回滚脚本
T-1.5	编写 `oidc_config` 表 DDL 和 MyBatis migration 脚本	F-28	0.5	配置表结构简单（issuer_url, client_id, client_secret, scopes, claim_mappings JSON, pat_policy JSON）；单表
T-1.6	验证模块编译集成：确保 `nexus-oidc-plugin` 作为 Nexus assembly 的一部分正确打包加载	全局	2	Nexus 的 Karaf 特性文件和 assembly 配置较复杂；需调试 OSGi bundle 加载、import/export package、Spring context 初始化；首次集成通常需要反复调试

小计：5.5 人天

WA-2: 配置与 Capability 层

任务 ID	任务描述	需求关联	人天	评估理由

T-2.1	`OidcConfiguration.java`：配置数据模型（Issuer URL、Client ID、Client Secret、Scopes、Claim 映射规则、PAT 策略）	F-28,F-29	1	需要设计嵌套结构（ClaimMapping 列表、PatPolicy 子对象）；Secret 字段需标记为加密字段
T-2.2	`OidcConfigurationDAO.java` + MyBatis Mapper XML：配置表 CRUD	F-28	1	参考现有 DAO 模式（如 `CUserDAO`）；Mapper XML 含 JSON 字段的序列化/反序列化处理
T-2.3	`OidcConfigurationStore.java`：配置存储服务（含 `SecretsService` 加密 Client Secret 的读写逻辑）	F-28	1.5	需要集成 `SecretsService` 对 Client Secret 加密/解密；参考 `ApiKeyStoreV2Impl` 的 Secret 处理模式；含单元测试
T-2.4	`OidcCapabilityDescriptor.java`：Capability 描述（配置表单字段定义）	F-30	1	定义 UI 表单字段（Issuer URL、Client ID、Scopes 等）；参考 `DefaultRoleCapabilityDescriptor`
T-2.5	`OidcCapability.java`：Capability 生命周期（onCreate 加载配置、onUpdate 刷新缓存、onRemove 清理指标和缓存）	F-30	1.5	Capability 启用/禁用时需要联动：注册/注销 Realm、注册/注销 Metrics、刷新/清理 Provider 元数据缓存；逻辑分支较多
T-2.6	配置变更事件：变更后发布事件通知 Provider 元数据缓存、JWKS 缓存、Claims 映射缓存刷新	F-28	0.5	复用 Nexus EventManager 发布 `OidcConfigChangedEvent`；各缓存组件订阅该事件
T-2.7	配置校验逻辑：Issuer URL 格式校验、必填项检查、Scopes 合法性（必须含 openid）	F-28	0.5	输入校验；URL 格式 + HTTPS 检查 + 必填字段
T-2.8	Client Secret 脱敏逻辑：API 返回时替换为 `****`，PUT 时空 Secret 表示不修改	F-32	0.5	序列化/反序列化拦截；简单但需仔细处理边界（首次设置、清空、不修改）
T-2.9	`SecurityContributor`：注册 OIDC 相关权限（`security:oidc:read`、`security:oidc:update`、`security:pat:delete`、`security:pat:admin`）	4.8	0.5	参考 `MetricsSecurityContributor` 模式；声明式权限注册
T-2.10	配置层单元测试	—	2	覆盖加密读写、脱敏、校验、事件发布、Capability 生命周期

小计：10 人天（实际编码 8 + 测试 2）

WA-3: IdP 通信

任务 ID	任务描述	需求关联	人天	评估理由
T-3.1	`OidcProviderClient.java`：封装 Apache HttpClient，配置独立的连接超时（5s）、读超时（10s）、重试（1 次）	F-33	1.5	需要基于 Nexus 已有的 `HttpClientManager` 创建专用 client 实例；配置 `RequestConfig`、`RetryHandler`；确保连接池隔离不影响其他 HTTP 流量
T-3.2	`OidcProviderMetadata.java`：解析 `/.well-known/openid-configuration` 响应，提取 authorization/token/userinfo/jwks_uri 端点	F-02	1	JSON 解析 + 字段校验（必须含 authorization_endpoint、token_endpoint、jwks_uri）；使用 nimbus 的 `OIDCProviderMetadata` 类简化解析
T-3.3	Provider 元数据缓存：TTL 24h，过期后异步刷新，刷新失败沿用旧值 + WARN 日志	F-36	1.5	需要实现：① 带 TTL 的缓存容器 ② 异步刷新逻辑（不阻塞请求线程）③ 刷新失败的 fallback ④ 配置变更时强制刷新；并发安全
T-3.4	Provider 元数据启动预热：Capability 启用时异步加载，不阻塞 Nexus 启动	F-36	0.5	在 Capability `onCreate` 中提交异步任务；加载失败记录 ERROR 但不阻止插件启用（登录时会再次尝试）
T-3.5	`JwksManager.java`：获取 JWKS（JWK Set），本地缓存，TTL 24h	F-35	1.5	使用 nimbus 的 `JWKSet.load()` + 自定义 HTTP 配置；缓存逻辑同 T-3.3 但数据模型不同（JWKSet 对象）
T-3.6	JWKS 容错刷新：签名验证失败时自动刷新一次，并发刷新请求合并为一次网络调用	F-35	1.5	需要实现 single-flight 模式（参考 `Cooperation2`）：第一个请求触发实际刷新，后续并发请求等待结果；刷新失败保留旧缓存
T-3.7	`IdpCircuitBreaker.java`：连续 5 次失败打开，半开窗口 60s 允许一次探测，成功则关闭	F-34	1	状态机实现（CLOSED → OPEN → HALF_OPEN → CLOSED/OPEN）；线程安全（AtomicInteger + volatile）；可配置阈值和窗口
T-3.8	熔断器与 ProviderClient 集成：每次 IdP 调用前检查状态，调用后更新计数	F-34	0.5	在 `OidcProviderClient` 的请求方法中加入熔断器 check/update 逻辑；熔断时直接抛出自定义 `IdpUnavailableException`
T-3.9		F-03	2	

	`IdTokenValidator.java`：ID Token 签名验证（RS256/ES256）+ Claims 校验（iss/aud/exp/iat/nonce/azp）			核心安全组件，需要：① 使用 nimbus `IDTokenValidator` 或自建基于 `JWSVerifier` ② 禁止 `alg:none` ③ 支持时钟偏差容忍（30s）④ nonce 验证 ⑤ 多 audience 场景处理；安全敏感，需仔细实现
T-3.10	ID Token 验证与 JWKS 刷新联动：签名验证失败 → 调用 T-3.6 刷新 JWKS → 重试验证一次	F-03,F-35	0.5	在 `IdTokenValidator.validate()` 中加入 retry-with-refresh 逻辑；需避免无限循环（只刷新一次）
T-3.11	IdP 通信层单元测试	—	3	**最重的测试任务之一**：① MockWebServer 模拟 IdP 各种响应（正常/超时/错误/TLS 失败）② 缓存过期和刷新测试 ③ 熔断器状态转换测试 ④ JWKS 刷新并发合并测试 ⑤ Token 验证攻击向量测试（alg:none、key confusion、过期、nonce 不匹配等）
T-3.12	`TokenCollisionResolver.java`：accessKey 碰撞检测与重试（≤3 次）	F-08	1	封装 `DuplicateKeyException` 捕获 + SecureRandom 重新生成逻辑；参考 `ApiKeyServiceImpl` 的处理模式

小计：16 人天（实际编码 12 + 测试 3 + 集成调试 1）

WA-4: OIDC 认证流程

任务 ID	任务描述	需求关联	人天	评估理由
T-4.1	`OidcAuthenticationToken.java`：自定义认证令牌，携带 authorization code + redirect_uri + state	F-01	0.5	简单 POJO 实现 `AuthenticationToken` 接口
T-4.2	`OidcSessionManager.java`：state/nonce/PKCE code_verifier 的生成、存储（Session）、消费、过期管理	F-04	1.5	需要：① SecureRandom 生成 ≥32 字节随机数 ② PKCE code_verifier (43-128 chars) + code_challenge (SHA256) ③ 绑定到 HTTP Session 并设置 TTL（5min）④ 消费后标记已使用（防重放）⑤ 过期自动清理
T-4.3	`OidcLoginRedirectFilter.java`：拦截登录请求，检查熔	F-01,F-	1.5	Servlet Filter 实现：① 检查熔断器（OPEN → 503）② 调用 T-4.2 生成 state/nonce/PKCE ③ 构

	断器状态，生成授权 URL，302 重定向到 IdP	34		造 OIDC 授权请求 URL（nimbus `AuthenticationRequest`）④ 302 重定向；需要注册到 Shiro FilterChain
T-4.4	`OidcCallbackFilter.java`：处理 IdP 回调 — state 验证	F-04	1	① 提取 callback 参数（code, state, error）② 验证 state（匹配 + 未消费 + 未过期）③ 销毁 state ④ 处理 error 响应（用户拒绝授权等）
T-4.5	`OidcCallbackFilter.java`：处理 IdP 回调 — code 换取 Token	F-01	1.5	① 构造 Token 请求（nimbus `TokenRequest`）② 通过 `OidcProviderClient` 发送 ③ 解析响应（id_token, access_token, refresh_token）④ 处理各种错误响应（invalid_grant, server_error 等）⑤ 集成熔断器计数
T-4.6	`OidcCallbackFilter.java`：处理 IdP 回调 — ID Token 验证 + 用户创建/同步	F-03,F-23,F-25	1.5	① 调用 `IdTokenValidator` ② 提取 Claims ③ 调用 `OidcUserManager` 创建/同步用户 ④ 调用 `ClaimsMappingService` 映射角色 ⑤ 处理各步骤的异常
T-4.7	`OidcCallbackFilter.java`：处理 IdP 回调 — 创建 Nexus Session	F-01	1	① 构造 `PrincipalCollection` ② 通过 `SecurityUtils.getSubject().login()` 完成 Shiro 登录 ③ 创建 Session/JWT ④ 设置 Cookie ⑤ 重定向到原始请求 URL 或首页
T-4.8	`OidcRealm.java`：Shiro AuthorizingRealm 实现 — `supports()` 仅接受 `OidcAuthenticationToken`	F-01	1	① 继承 `AuthorizingRealm` ② `supports()` 类型检查 ③ `doGetAuthenticationInfo()` — 此时认证已在 Filter 中完成，Realm 主要是构造 `AuthenticationInfo` ④ `doGetAuthorizationInfo()` — 查询用户角色和权限
T-4.9	Realm 注册：将 `OidcRealm` 注册为 Spring Component + Shiro Realm，在 Capability 启用/禁用时动态添加/移除	F-30	1	需要通过 `RealmManager.setConfiguredRealmIds()` 动态管理；Capability 禁用时从 Realm 列表中移除并清缓存
T-4.10	Filter 注册：将 OIDC 相关 Filter 注册到 Shiro FilterChain（`/oauth/login`、`/oauth/callback` 路径）	F-01	1	需要通过 `DynamicFilterChainManager` 或自定义 `WebFilter` 注册路径映射；确保不与现有 Filter 冲突
T-4.11	OIDC 认证流程单元测试	—	3	① Filter 各分支测试（正常流程、state 失败、code 交换失败、Token 验证失败）② Realm supports/auth/authz 测试 ③ Session 状态管理测试（生成/消费/过期/重放）

小计：15.5 人天（编码 11.5 + 测试 3 + Filter/Realm 注册调试 1）

WA-5: 用户与claim管理

任务 ID	任务描述	需求关联	人天	评估理由
T-5.1	`OidcUser.java`：OIDC 用户数据模型（userId = `oidc:<sub>`, source = `"oidc"`, email, displayName, status, roles）	F-27	0.5	实现 `User` 接口的 POJO
T-5.2	`OidcUserManager.java` — 接口实现骨架：实现 `UserManager` 接口，声明 source= `"oidc"`、realmName= `"OidcRealm"`、supportsWrite=false	F-23	0.5	参考 `AbstractReadOnlyUserManager`；框架代码
T-5.3	`OidcUserManager.java` — 用户创建逻辑：从 ID Token Claims 提取信息创建 Nexus 用户	F-23	1.5	① 提取 sub/email/name/preferred_username claims ② 构造 `CUser` ③ 通过 `SecurityConfigurationManager` 创建 ④ 处理 `DuplicateKeyException`（幂等：捕获后转 `getUser()`）⑤ 发布 `UserCreatedEvent`
T-5.4	`OidcUserManager.java` — 用户属性同步：每次登录时更新 email/displayName	F-24	0.5	比较现有属性与 Claims，有差异则更新；更新失败仅 WARN 不阻塞
T-5.5	`OidcUserManager.java` — 用户状态检查：供 PAT 认证路径调用，检查用户是否存在且为 active	F-26	0.5	`getUser()` + status 检查；用户不存在时触发关联 PAT 清理（参考 `BearerTokenRealm` 对 `UserNotFoundException` 的处理）
T-5.6	`ClaimsMappingService.java`：根据配置规则将 IdP Claims（groups/roles）映射为 Nexus 角色 ID 列表	F-25	2	核心映射逻辑：① 支持多种映射模式（精确匹配、前缀匹配、正则）② 从配置中读取映射规则 ③ 处理多值 claim（groups 通常是数组）④ 默认无角色（无匹配规则时返回空集）⑤ 映射变更后清除授权缓存
T-5.7	`OidcUserManager.java` — 用户查询/搜索：实现 `listUsers()`、	F-12	1	查询 `CUser` 表按 source 过滤；分页支持

	`searchUsers(criteria)`，从 Nexus 用户存储中查询 source=`"oidc"` 的用户			
T-5.8	用户管理单元测试	—	2	① 用户创建幂等性测试 ② Claims 映射各种模式（精确/前缀/正则/多值/空值）③ 用户同步成功/失败分支 ④ 状态检查逻辑

小计：8.5 人天（编码 6.5 + 测试 2）

WA-6: PAT 全生命周期

任务 ID	任务描述	需求关联	人天	评估理由
T-6.1	`PatInfo.java`：PAT 元数据模型（id, username, name, description, accessKey, expiresAt, lastUsedAt, lastUsedIp, createdAt, createdRealm, status）	F-10,F-11	0.5	POJO + builder
T-6.2	`PatInfoDAO.java` + MyBatis Mapper XML：pat_info 表 CRUD（含分页查询、按用户查询、按状态查询、过期记录查询）	F-12,F-14	1.5	多个 SQL 语句：insert、selectByUsername（分页）、selectByAccessKey、updateStatus、updateLastUsed、deleteExpired、selectOrphaned
T-6.3	`PatInfoStore.java`：pat_info 存储服务，封装 DAO 调用 + 事务管理	F-39	1	`@Transactional` 封装；含孤立记录查询方法
T-6.4	`PatFactory.java`：实现 `ApiKeyFactory`（`@Qualifier("pat")`），使用 `SecureRandom` + Base62 生成 20 字符 accessKey + 40 字符 secret	F-09	1	实现 `makeApiKey(principals)` → 拼接为 `accessKey + secret`；Base62 编码器（[0-9a-zA-Z]）；确保足够熵
T-6.5	`PatExtractor.java`：实现 `ApiKeyExtractor`（`@Qualifier("pat")`），从 `Authorization: Bearer nxpat-xxx-yyy` 中提取 Token	F-15	0.5	检测 `nxpat-` 前缀 → 分离 accessKey 和 secret → 返回拼接的 credentials；格式不匹配返回 null

T-6.6	`PatService.java` — 创建 PAT：调用 `ApiKeyService.createApiKey("pat", principals)` + 创建 `PatInfo` 记录，同一事务	F-08,F-39	2	① 校验 PAT 策略（允许创建？超过上限?）② 调用 PatFactory 生成 Token ③ `ApiKeyService.createApiKey()` 写入 `api_key_v2` ④ 写入 `pat_info` ⑤ 事务管理 ⑥ 碰撞重试（调用 `TokenCollisionResolver`）⑦ 构造响应（完整 Token 仅此一次）
T-6.7	`PatService.java` — 撤销 PAT：删除 `api_key_v2` + 更新 `pat_info` 状态为 REVOKED + 清除 Shiro 认证缓存	F-13	1	同一事务内操作两表；撤销后调用 `RealmManagerImpl` 风格的缓存清除；幂等（已撤销再撤销不报错）
T-6.8	`PatService.java` — 查询 PAT 列表：按用户查询，返回 PatInfo 列表（脱敏 accessKey）	F-12	0.5	查询 + accessKey 脱敏（前 6 位 + `****`）
T-6.9	`PatService.java` — 验证 PAT：查询 PatInfo 检查过期和状态	F-11	0.5	由 PatRealm 调用；检查 expiresAt 和 status 字段
T-6.10	`PatRealm.java`：继承 `BearerTokenRealm`（`@Qualifier("pat")`），format= `"pat"`	F-15,F-20	1.5	① `supports()` 仅接受 `principal="pat"` 的 `NexusApiKeyAuthenticationToken` ② `doGetAuthenticationInfo()` — ApiKeyService 查找 + PatService 验证过期/状态 + 用户状态检查 ③ `doGetAuthorizationInfo()` — 查询绑定用户的实时权限 ④ 缓存配置
T-6.11	`PatBasicAuthRealm.java`：识别 password 以 `nxpat-` 开头的 Basic Auth 请求	F-16	1.5	① 继承 `AuthenticatingRealm` ② `supports()` 检查 `UsernamePasswordToken` + password 前缀 ③ 解析 PAT → 调用 PatRealm 相同的验证逻辑 ④ **强制校验 username == PAT.username** ⑤ 不匹配时返回明确的认证失败
T-6.12	`PatCleanupTask.java`：Quartz 定时任务，清理过期 >7 天的 PAT + 孤立记录	F-14,F-39	1.5	① 注册 Quartz Job（参考现有 Nexus Task 模式）② 查询过期 pat_info 记录 ③ 删除对应 api_key_v2 记录 ④ 查询孤立记录（pat_info 有但 api_key_v2 无，或反向）⑤ 修复孤立记录 ⑥ 可配置调度周期（默认每天）

T-6.13	PAT last_used_at 异步更新：认证成功后异步更新最后使用时间和 IP	F-12	0.5	使用 EventManager 或 Executor 异步写入，避免阻塞认证路径；高频更新场景下需要节流（如最多每 5 分钟更新一次同一 PAT）
T-6.14	PAT 输入校验：名称 ≤200 字符、描述 ≤500 字符、XSS 过滤、有效期范围检查	F-10,F-11	0.5	输入验证逻辑 + XSS sanitizer
T-6.15	PAT Realm 注册：动态添加/移除 PatRealm 和 PatBasicAuthRealm 到 Shiro	F-30	0.5	与 T-4.9 类似但处理两个 Realm
T-6.16	PAT 全生命周期单元测试	—	3	① 创建流程（正常/碰撞重试/策略拒绝）② 撤销（正常/幂等/缓存清除）③ PatRealm 认证全流程（成功/过期/撤销/用户禁用/用户不存在/secret 不匹配）④ PatBasicAuthRealm（用户名匹配/不匹配）⑤ 清理任务（过期清理/孤立修复）⑥ 异步 last_used 更新

小计：17.5 人天（编码 13.5 + 测试 3 + Realm 注册调试 1）

WA-7: Prometheus 监控与健康检查

任务 ID	任务描述	需求关联	人天	评估理由
T-7.1	`OidcMetricsConstants.java`：所有指标名称常量定义	M-01~M-19	0.5	常量类，确保命名一致性
T-7.2	`OidcMetrics.java` — 认证指标注册：M-01 auth.total、M-02 auth.failure、M-03 auth.duration（按 method 拆分）	M-01~M-03	1	注册 6 个 Meter（3 method × 2 result）+ 3 个 Timer；参考 PRD 第九章实现参考
T-7.3	`OidcMetrics.java` — IdP 通信指标注册：M-04 request.total、M-05 request.duration、M-06	M-04~M-07	1	注册 12 个 Meter（4 endpoint × 3 result）+ 4 个 Timer + 1 个 Gauge + 1 个 Meter

	circuit_breaker.state、M-07 trip.total			
T-7.4	`OidcMetrics.java` — JWKS 缓存指标：M-08~M-11	M-08~M-11	0.5	3 个 Meter + 1 个 Gauge
T-7.5	`OidcMetrics.java` — PAT 管理指标 + 用户指标：M-12~M-19	M-12~M-19	1	M-12/M-17 使用 `@CachedGauge`（需要在 PatService/OidcUserManager 上注解）；其余为 Meter
T-7.6	`OidcMetrics.java` — 生命周期管理：`start()` 注册所有指标，`stop()` 用 `removeMatching` 注销	M-全部	0.5	与 OidcCapability 生命周期联动
T-7.7	在各模块中嵌入 Metrics 采集点：OidcCallbackFilter、PatRealm、PatBasicAuthRealm、OidcProviderClient、JwksManager、PatService 中调用 OidcMetrics 记录方法	M-全部	2	分散在多个类中的采集点植入，每处需要：① 正确选择指标 ② 正确传递标签 ③ Timer 的 try-with-resources 模式 ④ 异常路径也要记录
T-7.8	`IdpHealthCheck.java`：HealthCheck 实现 + PAT 存储 HealthCheck	F-41,F-42	1	① 检查 Well-Known 端点可达性 ② 检查 JWKS 端点可达性 ③ 检查熔断器状态 ④ 组合判定逻辑 ⑤ PAT 存储简单 SELECT 探测 ⑥ 注册到 `HealthCheckRegistry`
T-7.9	监控层单元测试	—	1	验证指标注册/注销、采集点触发、HealthCheck 各状态分支

小计：8.5 人天（编码 7 + 测试 1 + 采集点调试 0.5）

WA-8: REST API

任务 ID	任务描述	需求关联	人天	评估理由
T-8.1	`OidcApiResource.java` — GET `/api/v1/security/oidc/config`：返回配置（Secret 脱敏）	F-32	0.5	查询 + 序列化 + 权限注解

T-8.2	`OidcApiResource.java` — PUT `/api/v1/security/oidc/config`：更新配置（含 Secret 处理逻辑）	F-32	1	校验 + 空 Secret 不修改逻辑 + 变更事件发布 + 审计日志
T-8.3	`PatApiResource.java` — POST `/api/v1/security/pat`：创建 PAT	F-08	1	请求体校验（名称/描述/有效期）→ 调用 PatService.create() → 返回完整 Token + 元数据
T-8.4	`PatApiResource.java` — GET `/api/v1/security/pat`：列出当前用户 PAT	F-12	0.5	调用 PatService.listByUser() → 分页 + 脱敏响应
T-8.5	`PatApiResource.java` — DELETE `/api/v1/security/pat/{id}`：撤销 PAT	F-13	0.5	权限检查（自己的直接撤销，他人的需要 admin 权限）→ 调用 PatService.revoke()
T-8.6	API 请求/响应 DTO：`OidcConfigRequest` / `Response`、`PatCreateRequest` / `Response`、`PatListResponse`	4.8	1	多个 DTO 类 + Jackson 注解 + Swagger/OpenAPI 注解
T-8.7	API 异常处理：自定义 ExceptionMapper 处理 OIDC/PAT 特定异常（PatExpiredException、PatPolicyViolationException 等）	4.8	0.5	参考 `ExceptionMapperSupport` 模式；含 fault-id
T-8.8	REST API 单元测试	—	2	各端点正常/异常路径；权限检查；输入校验；脱敏验证

小计：7.5 人天（编码 5.5 + 测试 2）

WA-9: 前端 UI

任务 ID	任务描述	需求关联	人天	评估理由
T-9.1	OIDC 登录按钮：在 Nexus 登录页面添加 "使用 OIDC 登录" 按钮，点击跳转 `/oauth/login`	F-01	2	Nexus 前端基于 ExtJS（Rapture 框架）；需要：① 找到登录组件扩展点 ② 添加按钮 ③ 按钮仅在 OIDC Capability 启用时显示（需调用后端状态接口）；ExtJS 开发效率较低
T-9.2		F-12	2.5	

	PAT 管理页面 — 列表视图：用户安全设置中新增 "个人访问令牌" 标签页，展示 PAT Grid（名称、创建时间、到期时间、最后使用、状态）			创建 ExtJS Grid Panel + Store + Model；分页加载；列渲染（过期标红、状态标签）；参考已有的 User Token 或 API Key 管理界面模式
T-9.3	PAT 管理页面 — 创建对话框：弹窗填写名称、描述、有效期（下拉选择 + 自定义日期）→ 创建后显示完整 Token 的一次性复制框	F-08,F-10,F-11	2	ExtJS Form + Dialog；关键交互：Token 创建后用 `<textarea readonly>` 展示，提供复制按钮，关闭后永远不再显示完整 Token
T-9.4	PAT 管理页面 — 撤销操作：列表行操作按钮或右键菜单，确认对话框后调用 DELETE API	F-13	1	确认对话框 + API 调用 + 列表刷新
T-9.5	OIDC Capability 配置 UI：Capability 配置表单（Issuer URL、Client ID、Client Secret、Scopes、Claim 映射规则编辑器）	F-28	2.5	Capability 表单由 `CapabilityDescriptor` 驱动但复杂字段（Claim 映射规则）可能需要自定义表单组件；Client Secret 输入框 type=password
T-9.6	前端国际化（如需要）	全局	1	提取所有 UI 文本到 i18n 资源文件

小计：11 人天

WA-10: 测试（集成测试 + 端到端测试）

任务 ID	任务描述	需求关联	人天	评估理由
T-10.1	搭建嵌入式 OIDC Provider 测试基础设施（使用 mock-oauth2-server 或 WireMock 模拟 IdP）	全局	2	配置 MockOAuth2Server：签发可验证的 ID Token、提供 JWKS 端点、支持 Authorization Code Flow 模拟；作为所有集成测试的基础设施
T-10.2	OIDC 登录集成测试：完整 Authorization Code Flow + PKCE → Token 交换 → 用户创建 → Session 建立	F-01~F-04,F-23	2	模拟浏览器重定向流程；验证 state/nonce/PKCE 全链路；验证用户创建和角色映射
T-10.3	PAT 创建 + Bearer 认证集成测试：创建 PAT → 用 PAT 访问受保护 API → 验证权限	F-08,F-15,F-20	1.5	完整的创建-使用-验证链路

T-10.4	PAT Basic Auth 集成测试：用 `username:nxpat-xxx` 方式访问 → 验证认证成功 + username 校验	F-16	1	重点测试用户名匹配/不匹配场景
T-10.5	IdP 故障场景集成测试：模拟 IdP 超时、宕机 → 验证熔断器行为 → 验证 PAT 不受影响	F-34,F-37	2	用 WireMock delay/fault 模拟各种故障；验证熔断器状态转换；验证 PAT 路径完全隔离
T-10.6	JWKS 密钥轮换集成测试：签发新 key 的 Token → 验证自动刷新 → 新旧 key 并存	F-35	1	MockServer 动态切换 JWKS 端点返回的 key
T-10.7	PAT 过期和撤销集成测试：创建 → 过期后认证拒绝 → 撤销后认证拒绝 → 缓存清除验证	F-11,F-13	1	需要控制时间（mock clock 或短过期时间）
T-10.8	Docker Login 端到端测试：`docker login` + `docker push` + `docker pull` 使用 PAT	F-17	2	需要搭建 Nexus Docker Registry + Docker 客户端环境；验证 V2 Token Auth 路径
T-10.9	Maven 端到端测试：settings.xml 配置 PAT → `mvn deploy` → `mvn dependency:resolve`	F-18	2	需要搭建 Maven Repository + 配置 settings.xml；验证 deploy 和 resolve 都正常
T-10.10	Prometheus 指标端到端测试：执行各种操作 → 查询 `/metrics/prometheus` → 验证指标值正确	M-全部	1.5	触发认证/创建/撤销操作后验证指标计数和耗时

小计：17 人天

> **理由说明**：集成测试和端到端测试是本项目工作量的重头，因为：① 涉及多个外部协议（OIDC、Docker V2 Auth、Maven/HTTP） ② 需要搭建模拟 IdP ③ Docker/Maven 端到端测试需要完整的运行环境 ④ 故障场景测试需要精确控制网络行为。

WA-11: Phase 2 增强功能

任务 ID	任务描述	需求关联	人天	评估理由
T-11.1	Token Refresh：使用 Refresh Token 静默续期 + 并发合并	F-06	2	① 存储 Refresh Token（加密）② 定时检查并刷新 ③ single-flight 并发合并 ④ 刷新失败不中断会话
T-11.2	UserInfo 端点调用：可选获取补充信息，失败不阻塞	F-05	1	HTTP 调用 + Claims 合并 + 5s 超时 + 失败容忍
T-11.3	用户属性同步增强：登录时同步 + 与 UserInfo 结合	F-24	0.5	在 T-11.2 基础上更新用户属性
T-11.4	全局 PAT 策略实现：允许/禁止 PAT 创建、最大有效期限制、每用户 PAT 数量上限	F-21	1.5	① 策略配置存储（OidcConfiguration 子对象）② PatService.create() 中校验 ③ 明确错误响应
T-11.5	PAT 策略配置 UI/API	F-29	1	扩展 Capability 配置表单 + REST API
T-11.6	管理员 PAT 管理 API：GET `/pat/admin` + DELETE `/pat/admin/{userId}`	4.8 P1	1	分页查询所有 PAT + 批量撤销
T-11.7	管理员 PAT 管理 UI：管理后台查看所有用户 PAT 的列表页	4.8 P1	2	ExtJS Grid + 过滤器（按用户/状态/过期）+ 批量操作
T-11.8	连接测试 API：POST `/oidc/test` 返回详细诊断	F-31	1.5	① DNS 解析检查 ② TLS 握手检查 ③ Well-Known 端点 HTTP 请求 ④ Token 端点可达性 ⑤ 结构化诊断响应
T-11.9	认证事件审计 + PAT 管理审计 + 配置变更审计	F-43~F-45	2	① 扩展 `NexusAuthenticationEvent` 增加 authMethod/patName ② 创建 `OidcAuditor` 参考 `UserTokenAuditor` ③ Capability 变更事件审计
T-11.10	Phase 2 Metrics 补全：M-08~M-11（JWKS 缓存）、M-13~M-19（PAT 生命周期 + 用户）	M-08~M-19	1	注册剩余指标 + 嵌入采集点
T-11.11	PAT 存储健康检查	F-42	0.5	简单 SELECT 探测 + 注册
T-11.12	Grafana 仪表盘模板 + 告警规则文档	4.6.4,4.6.5	1	编写 JSON 仪表盘模板 + Prometheus alerting rules YAML

T-11.13	Phase 2 测试	—	2	新增功能的集成测试

小计：17 人天

WA-12: Phase 3 高级功能

任务 ID	任务描述	需求关联	人天	评估理由
T-12.1	Scoped PAT — 数据模型：`pat_info.scopes` JSON 字段 + 权限交集逻辑	F-22	2.5	① 定义 scope 数据模型（仓库级别 + 操作级别）② `PatRealm.doGetAuthorizationInfo()` 中将用户权限与 scope 取交集 ③ Scope 的 UI 选择器
T-12.2	Scoped PAT — 创建 UI 和 API 扩展	F-22	1.5	创建 PAT 时选择 scope；API 请求/响应扩展
T-12.3	单点登出（SLO）：Front-channel + Back-channel Logout 处理	F-07	3	① Back-channel：接收 IdP 的 logout 通知（POST endpoint），验证 logout token，清除对应用户的 Session ② Front-channel：注册 iframe URL ③ 不撤销 PAT；协议实现较复杂
T-12.4	npm/NuGet/PyPI 客户端适配验证与修复	F-19	1.5	各格式客户端的认证方式验证；可能需要格式特定的 Extractor 或 Filter 调整
T-12.5	Phase 3 测试	—	2	Scoped PAT 权限交集测试；SLO 流程测试；各格式客户端端到端测试

小计：10.5 人天

项目排期：

排期说明：

项目	说明

T+0	项目启动日（第 1 个工作日）
T+N	距启动日的第 N 个**工作日**（不含周末节假日）
1 周	5 个工作日。T+0~T+4 = 第 1 周
1 人天	8 小时有效编码/测试时间，不含会议和 Code Review
半日任务	≤0.5d 的任务与相邻任务合并到同一天

里程碑规划：

里程碑	任务阶段	节点	交付物	验收标准
M1	WA-1: 项目脚手架与基础设施	**T+4**	可编译加载的空插件模块	`mvn clean install` 通过
M2	WA-2: 配置与 Capability 层	**T+14**	配置 CRUD + Capability 启停	管理 UI 可创建 OIDC Capability
M3	WA-3: IdP 通信	**T+30**	IdP 通信 + 熔断器 + Token 验证	MockServer 全部单元测试通过
M4	WA-5: 用户与角色管理	**T+39**	OIDC 用户自动创建 + Claims 角色映射	用户幂等创建 + 映射模式单测通过
M5	WA-4: OIDC 认证流程	**T+54**	浏览器 OIDC 登录打通	对接 idp（如Keycloak）完成首次登录
M6	WA-7: Prometheus 监控与健康检查	**T+72**	Prometheus监控功能可用	`/metrics/prometheus` 可查询
M7	WA-8: REST API	**T+87**	PAT 全功能可用	Bearer + Basic Auth 验证通过
M8	WA-9: 前端 UI	**T+98**	前端 UI 完成	登录按钮 + PAT 管理页面 + 配置表单
M9	WA-10: 集成测试 + 端到端测试	**T+114**	**Phase 1 完成**	Docker/Maven E2E 通过
M10	WA-11: 增强功能	**T+131**	**Phase 2 完成**	Token Refresh + 管理员工具 + 审计

M11	WA-12: 高级功能	**T+143**	**Phase 3 完成**	Scoped PAT + SLO + 全格式适配
M12	OSGI/Karf打包调试等	**T+162**	**项目交付**	缓冲消化 + 回归全量通过

4.4.8 风险与注意事项

风险场景	预估额外耗时	理由
OSGi/Karaf 打包调试	2-3 天	Nexus 的 OSGi 环境对新 Bundle 的 import/export 要求严格，首次集成通常需要反复调试 ClassNotFoundException
主流 IdP 兼容性适配	3-5 天	各 IdP（Keycloak/Azure AD/Okta/Google）的 OIDC 实现有细微差异（claim 名称、Token 格式、JWKS 结构），可能需要逐一适配
Docker V2 Token Auth 集成	2-3 天	Docker Registry V2 的 Token Auth 协议有特殊的 `GET /v2/token` → Bearer token 交换流程，与标准 Basic Auth 路径有差异
Shiro Filter/Realm 注册冲突	1-2 天	动态注册 Filter 和 Realm 可能与现有安全配置冲突
数据库迁移兼容	1-2 天	在现有 Nexus 实例上升级时的数据库迁移脚本兼容性
未预见问题	3-5 天	经验值

4.5 开源项目代码版本维护与升级

包括主代码仓库停维等。

4.5.1 配置并同步上游仓库

首先，Fork Sonatype 维护的官方 Nexus 社区版最新代码，然后将fork的仓库克隆到本地，并添加上游仓库地址：

代码块

```
git clone https://github.com/your-org/nexus-public.git
cd nexus-public
```

```
git remote add upstream https://github.com/sonatype/nexus-public.git
git remote -v
origin  https://github.com/ccdbgo/nexus-public.git (fetch)

origin  https://github.com/ccdbgo/nexus-public.git (push)

upstream        https://github.com/sonatype/nexus-public.git (fetch)

upstream        https://github.com/sonatype/nexus-public.git (push)

# 拉取上游全部tag
git fetch upstream --tags
# 本地切到目标版本
git checkout tags/release-3.91.1-04 -b release-3.91.1-04
#把这个版本分支推送到你的fork仓库
git push origin refs/heads/release-3.91.1-04:release-3.91.1-04
后续业务代码开发基于分支release-3.91.1-04
```

✏️ main：fork官方仓库并git clone后默认代码分支为main分支，main分支完全追踪上游社区版（只读，绝不做任何定制修改）分支上拉取上游更新，然后合并到你的 `release-3.91.1-04` 分支进行集成。

`release-3.91.1-04`：所有上游更新、新功能、修复都合并到 release-3.91.1-04，冲突永远只在 release-3.91.1-04 上解决一次，release-3.91.1-04 是真正的业务代码库，所有的工作都围绕它展开。

4.5.2 同步上游代码

- 当发现官方仓库有需要的重要功能特性或漏洞修复代码时进行同步。

代码块

```
git checkout main
# 拉取上游镜像
git pull upstream main
# 切换到开发分支
git checkout release-3.91.1-04
# 将上游更新合并到开发分支，如有冲突，在此解决（只在这一个地方解决冲突）
git merge main

```

4.5.3 开发新功能（如 OIDC 支持）

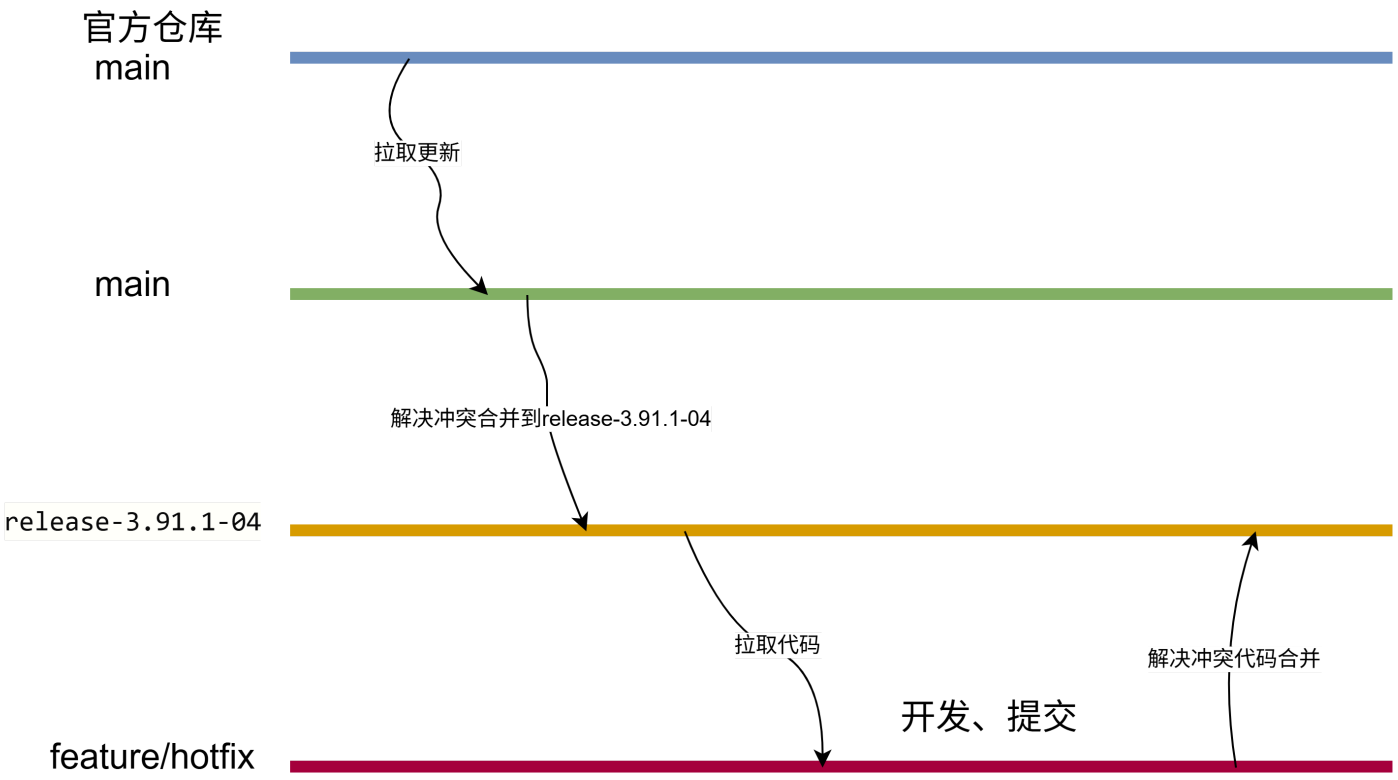
代码块

```
git checkout release-3.91.1-04
git checkout -b feature/oidc    # 临时特性分支
# ... 写代码、提交 ...
git checkout release-3.91.1-04
git merge --no-ff feature/oidc    # 合并回主线
git branch -d feature/oidc        # 删除临时分支（可选，但推荐）
```

注意：使用 `--no-ff` 合并特性分支，可以在 `git log --graph` 中清楚看到哪些提交是功能开发，哪些是上游同步。

4.5.4 紧急修复安全漏洞

- 上游发布了安全补丁：走“同步上游更新”流程，合并到 release-3.91.1-04 即可。
- 如果需要自己打补丁：直接在 release-3.91.1-04 上创建一个 `hotfix/xxx` 临时分支，修复后合并回 release-3.91.1-04，然后删除临时分支。



5. 测试建议

建议测试顺序，从低风险到高风险，从单组件到组合场景测试。破坏性操作，执行前必须确认。破坏性操作清单：

- 停止 Patroni Leader → 触发 PG 故障转移，Nexus 写中断
- 停止 etcd 多数节点 → Patroni 降级，Nexus 完全不可写
- 重启整个节点 → 影响该节点所有服务

5.1 测试目标

测试目标包括功能需求测试和非功能需求测试，目的是全面验证系统镜像存储和Docker镜像存储方案的功能正确性、性能指标、高可用能力、安全性和兼容性，确保系统满足技术方案中定义的各项需求指标。

具体测试目标包括：

1. 验证系统各功能模块的正确性，包括镜像上传/拉取、版本控制、删除清理、多租户权限等核心功能。
2. 验证系统性能指标是否满足需求，包括拉取速度≥200MB/s、推送1GB镜像≤30s、500并发支持。
3. 验证高可用架构的故障切换能力，包括Nginx/Nexus故障切换、PostgreSQL主从切换、Ceph RGW故障转移。
4. 验证系统安全性，包括RBAC权限控制、TLS加密传输、存储加密。
5. 验证系统兼容性，包括多种镜像格式支持、多后端存储适配。
6. 验证备份恢复机制的有效性。

5.2 测试范围

测试范围：

a. 所有功能需求验证

b. 高可用架构各层的故障切换测试

c. 端到端业务流程测试

d. 性能基准测试

e. 安全性测试

f. 备份恢复测试

测试范围外：

a. 第三方组件（nginx、Ceph、PostgreSQL、Nexus、Haproxy、Keppalived、etcd、patroni）自身的内部逻辑。

b. 网络基础设施（交换机、路由器）。

c. 底层硬件和操作系统层面的稳定性和性能。

5.3 测试环境

节点信息

节点	IP 地址	配置	角色	运行组件
U1	192.168.2.2	4c6g20g nvme	主节点	Nexus 8081, Nginx, Keepalived(priority=110), HAProxy(5000--postgresql/8080--ceph rgw), Ceph mon/osd/rgw/mgr, Patroni, etcd
U2	192.168.2.8	4c6g20g nvme	备节点	Nexus 8081, Nginx, Keepalived(priority=110), HAProxy(5000--postgresql/8080--ceph rgw), Ceph mon/osd/rgw/mgr, Patroni, etcd
U3	192.168.2.9	4c6g20g nvme	仲裁节点	Ceph mon/osd/rgw, etcd
	VIP 192.168.2.100		虚拟 IP	Keepalived VRRP

端口映射

端口	协议	用途
192.168.2.100:80	HTTP	Nginx → Nexus 反向代理入口
192.168.2.100:8080	TCP	HAProxy → Ceph RGW（S3）
192.168.2.100:5000	TCP	HAProxy → PostgreSQL Leader
192.168.2.100:8081	HTTP	Nexus 直接访问（经 Nginx 转发）
U1/U2/U3:7480	HTTP	Ceph RGW 实例
U1/U2:8008	HTTP	Patroni REST API（健康检查）
1/U2/U3:2379	TCP	etcd 客户端端口

数据流：

Nexus → HAProxy(127.0.0.1:5000) → PostgreSQL Leader (Patroni)

Nexus → HAProxy(192.168.2.100:8080) → Ceph RGW (U1/U2/U3:7480)

关键超时参数

参数	值	说明
Keepalived fall	3 次 × 5s = 15s	健康检查失败检测时间
Keepalived weight	-20	失败时优先级降低量
Patroni TTL	30s	Leader 租约超时
HAProxy health check interval	2s	后端健康检查间隔

配置方法

5.4 测试用例

VIP故障迁移测试

测试用例编号	测试用例名称	测试目的	前置条件	
TC-VIP-01	U1 Nginx 进程故障 → VIP 迁移至 U2	验证 U1 Nginx 停止后，Keepalived 健康检查失败，VIP 自动迁移至 U2	· VIP 持有于 U1（192.168.2.2） · U1 和 U2 Nexus 均正常运行 · 基线健康检查全部通过	1. 记录 ssh rc 2. 停止 ssh rc 3. 等待 sleep 4. 验证 ssh rc 5. 验证 curl -s 【恢复 ssh rc sleep
TC-VIP-02	U1 Nexus 进程故障 → VIP 迁移至 U2	验证 U1 Nexus 停止后（HTTP 8081 不可达），健康检查失败，VIP 迁移至 U2	· VIP 持有于 U1 · 基线健康检查全部通过	1. 停止 ssh rc 2. 等待 sleep 3. 验证 ssh rc 4. 通过 curl -s 【恢复 ssh rc sleep sleep
TC-VIP-03	U1 节点完全宕机 → VIP 迁移至 U2	验证 U1 整机宕机时，VIP 自动迁移至 U2，服务恢复	· VIP 持有于 U1 · 基线健康检查全部通过 · 确认 U2 Nexus 已连接至 PostgreSQL 并可正常写入 · ⚠️ 高风险：需维护窗口执行	1. 记录 ssh rc ceph 2. 模拟 ssh rc 3. 等待 sleep 4. 验证 ssh rc 5. 验证 curl -s

				【恢复
TC-VIP-04	VIP 双向切换验证（U1→U2→U1）	验证 VIP 可在 U1 和 U2 之间双向自动切换，且每次切换后服务均可用	· VIP 持有于 U1 · 基线健康检查全部通过	1. 停止 ssh rc sleep ssh rc 2. 恢复 ssh rc sleep ssh rc 3. 停止 ssh rc sleep ssh rc 【恢复 ssh rc
TC-VIP-05	Keepalived 进程故障 → VIP 行为验证	验证 U1 Keepalived 进程崩溃时，VIP 迁移至 U2；U1 Keepalived 恢复后 VIP 抢占回 U1	· VIP 持有于 U1 · 基线健康检查全部通过	1. 强制 ssh rc 2. 等待 sleep 3. 验证 ssh rc 4. 恢复 ssh rc sleep 5. 验证 ssh rc 【验证 journ

测试用例编号	测试用例名称	测试目的	
TC-PG-01	PostgreSQL Leader 节点故障 → 自动选举新 Leader	验证 Patroni Leader 节点停止后，etcd TTL 超时（30s），Replica 自动晋升为新 Leader	· 确认当 · 基线健 · Nexus · 高风险
TC-PG-02	Patroni Replica 节点故障 → 集群继续运行	验证 Patroni Replica 节点故障时，Leader 继续运行，Nexus 不受影响	· 确认当 · 基线健
TC-PG-03	etcd 单节点故障 → Patroni 集群继续运行	验证 etcd 单节点故障时（3 节点中 1 个），Patroni 仍可维持 Leader 租约，Nexus 不中断	· etcd 3 · 基线健

TC-PG-04	etcd 多数节点故障 → Patroni Leader 降级保护	验证 etcd 失去 quorum（2/3 节点故障）时，Patroni Leader 主动降级为只读，防止脑裂	· etcd 3 · 基线健 · 已通知 · 高风险
TC-PG-05	HAProxy PostgreSQL 后端切换验证（switchover）	验证 HAProxy 通过 Patroni REST API（port 8008 /master）正确识别 Leader，并在 Leader 切换后自动更新路由	· 基线健 · 风险等

Ceph RGW HA测试

测试用例编号	测试用例名称	测试目的	
TC-RGW-01	单个 RGW 实例故障 → HAProxy 自动剔除	验证单个 RGW 实例停止后，HAProxy 自动将其标记为 DOWN，S3 请求路由至其余实例	· 3 个 RGW 实 U3:7480） · HAProxy 8 · 基线健康检
TC-RGW-02	两个 RGW 实例故障 → 单实例继续服务	验证 3 个 RGW 中 2 个故障时，剩余 1 个实例仍可提供 S3 服务	· 3 个 RGW 实 · 基线健康检
TC-RGW-03	Ceph OSD 单节点故障 → 数据可用性验证	验证单个 OSD 故障时，Ceph 数据仍可访问（副本数 >= 2），S3 读写不中断	· Ceph HEA · 基线健康检

TC-RGW-04	HAProxy TCP 模式 S3v4 签名验证	验证 HAProxy 使用 TCP 模式转发 S3 请求时，S3v4 签名验证正确（Host 头不被改写）	· HAProxy 8 · 基线健康检 · 风险等级：

端到端组合场景测试

测试用例编号	测试用例名称	测试目的	前置条件
TC-E2E-01	U1 完全宕机 → 全栈故障转移验证	验证 U1 整机宕机时，VIP、PostgreSQL Leader、RGW 均自动故障转移，Nexus 服务在 U2 上完整恢复	· VIP 持有于 U1，PostgreSQL Leader 在 U1 · 基线健康检查全部通过 · 已通知相关人员测试窗口 · 高风险：建议在业务低峰期执行
TC-E2E-02	PostgreSQL Leader 故障 + VIP 同时在 U1 → 双重故障恢复	验证 U1 同时发生 Nexus 故障和 PostgreSQL Leader 故障时，系统能正确处理两个独立的故障转移流程	· VIP 持有于 U1，PostgreSQL Leader 在 U1 · 基线健康检查全部通过 · 高风险：Nexus 写操作中断约 30-40s
TC-E2E-03	持续负载下的故障转移（混沌测试）	在 Nexus 持续接收请求的情况下，触发 VIP 故障转移，	· VIP 持有于 U1 · 基线健康检查全部通过

		验证请求中断时间和数据一致性	· 已安装 curl 工具